

The Complete Python Book

From first install to CPython contributor

What this book is

One book, four volumes, written to be read in order or dipped into at whatever level you currently occupy.

- **Volume I — Beginner.** You have never written Python, or you have copied code without understanding it. By the end you can write and run real programs that read files, handle errors, and use the standard library.
- **Volume II — Intermediate.** You can write scripts and now need to write *software*: classes, generators, decorators, packages, tests, types, version control.
- **Volume III — Advanced.** You maintain codebases other people depend on. Memory behaviour, concurrency, performance, design, packaging, and the tooling that keeps a team honest.
- **Volume IV — Expert.** You want to know what the interpreter actually does. Descriptors and metaclasses, the type system's rules, the GIL and free-threading, bytecode and the GC, AST and import hooks, distributed systems, security, and contributing upstream.

Each volume answers three questions about every mechanism: **what is the rule**, **why does it exist**, and **where does it bite**. Volume IV goes furthest into the first; Volume I spends most of its time on the third, because beginners are not short of syntax, they are short of debugging instincts.

How to use it

Do not read passively. Every chapter ends with exercises. Do at least two. Type the code rather than copying it — the typos are the lesson.

Markers. [3.12] means "arrived in that version." ⚠ marks a footgun that has caused real outages. 🏃 marks something to run yourself.

Version. Code targets Python 3.11+ unless noted. The interpreter is moving quickly right now — free-threading, the JIT, and annotation semantics have all changed in consecutive releases — so verify version-specific claims against the release notes for the version you actually run.

Pick a track after Volume II. Web, data, or automation. The advanced topics you actually need diverge sharply from that point; Appendix B tells you which chapters matter for each.

Table of Contents

Volume I — Beginner

1. Setup: installing Python, the REPL, running scripts, pip, virtual environments
2. Syntax basics: variables, indentation, comments, `input` and `print`
3. Data types: `int`, `float`, `str`, `bool`, and conversion
4. Strings: indexing, slicing, methods, f-strings

5. Operators and expressions
6. Control flow: `if/elif/else`, `for`, `while`, `break/continue`
7. Collections: list, tuple, set, dict
8. Functions: arguments, defaults, return values, scope
9. Errors: reading tracebacks, `try/except`
10. File I/O: text files and CSV
11. Modules and the standard library: `math`, `random`, `datetime`, `os`

Volume II — Intermediate

12. Comprehensions and generator expressions
13. `*args`, `**kwargs`, keyword-only arguments, unpacking
14. Lambdas, `map/filter`, `sorted` with keys
15. Object-oriented programming: classes, inheritance, dunder, properties
16. Iterators and generators
17. Closures and decorators
18. Exceptions in depth: custom classes, `finally`, `with`
19. Modules, packages, `__name__`, and project structure
20. Working library: `pathlib`, `json`, `collections`, `itertools`, `functools`
21. Testing with `pytest` and debugging with `pdb`
22. Type hints and `mypy`
23. Git, `requirements.txt`, and `pyproject.toml`

Volume III — Advanced

24. The memory model: references, mutability, garbage collection
25. Advanced typing: generics, protocols, `TypedDict`, overloads
26. Metaclasses, descriptors, `__slots__`, and ABCs
27. Concurrency: threading, multiprocessing, and the GIL
28. `asyncio`: coroutines, the event loop, `async` context managers
29. Performance: profiling, caching, algorithmic tuning
30. Design patterns and SOLID in Python
31. Packaging and publishing to PyPI; semantic versioning
32. CI/CD, linting, formatting, and pre-commit hooks
33. Dataclasses, `Pydantic`, and serialization

Volume IV — Expert

- **Part I** — The Object Model
- **Part II** — The Type System
- **Part III** — Concurrency and Parallelism
- **Part IV** — Performance and Internals
- **Part V** — Metaprogramming, Introspection and the Import System
- **Part VI** — Iteration, Generators and Laziness
- **Part VII** — Errors, Context Managers and API Design
- **Part VIII** — Data Modeling
- **Part IX** — Packaging, Distribution and Environments
- **Part X** — Testing and Quality

- **Part XI** — Correctness-Critical Standard Library
- **Part XII** — Security
- **Part XIII** — Domain-Specific Languages and Code Generation
- **Part XIV** — Advanced Async: Custom Loops, Backpressure, Structured Concurrency
- **Part XV** — Distributed Systems: Task Queues, Brokers, Scaling Workers
- **Part XVI** — Contributing to CPython and Major Open Source

Appendices

- **A** — Version cheat sheet, 3.8 to 3.15
- **B** — Choosing a track: web, data, automation
- **C** — Study plans for all four levels
- **D** — Quick reference: the mistakes that cost the most
- **E** — Capstone projects
- **F** — Reading list

Volume I — Beginner

The goal of this volume is not to teach you every keyword. It is to get you to the point where you can write a program, watch it fail, read the failure, and fix it without help. That loop is the whole skill.

Chapter 1 — Setup

Installing Python

Check what you already have. Open a terminal (Terminal on macOS/Linux, PowerShell on Windows):

```
python3 --version    # macOS / Linux
python --version     # Windows
```

If that prints something like `Python 3.12.4`, you are ready. If not, or if it prints a 2.x version, install a current one:

- **Windows** — download the installer from python.org. ⚠ **Tick "Add python.exe to PATH"** on the first screen. Skipping it is the single most common beginner setup problem.
- **macOS** — the system Python is old and partly reserved for the OS. Install a fresh one from python.org, or `brew install python@3.12`.
- **Linux** — usually preinstalled. `sudo apt install python3 python3-pip python3-venv` fills the gaps on Debian/Ubuntu.
- **Any platform, recommended** — `uv`, a fast tool that manages interpreters and packages: `uv python install 3.12`.

The REPL

Typing `python3` with no arguments starts the interactive interpreter:

```
>>> 2 + 3
5
>>> name = "Ada"
>>> len(name)
3
>>> exit()
```

The REPL is for experiments: checking what a function returns, what an error looks like, what methods a string has (`dir("")`, `help(str.split)`). Use it constantly. Professionals use it constantly.

Running scripts

Put code in a file ending `.py` and run it:

```
# hello.py
print("Hello, world")
```

```
python3 hello.py
```

Two things beginners trip over. First, you must be in the same directory as the file, or give the full path — `cd` to the right folder first. Second, ⚠ **never name your file after a module you want to import**. A file called `random.py` will shadow the standard library's `random`, and the error message will not tell you that. Same for `json.py`, `email.py`, `test.py`.

pip and packages

The standard library is large but not everything. `pip` installs the rest, from the Python Package Index (PyPI):

```
python3 -m pip install requests
```

Use `python3 -m pip`, not bare `pip`. The `-m` form guarantees you are installing into the interpreter you just named, which eliminates an entire category of "I installed it but Python says it isn't there" confusion.

Virtual environments

A virtual environment is a private folder of packages for one project. Without them, every project shares one global set of libraries, and two projects that need different versions of the same library cannot both work.

```
python3 -m venv .venv          # create it (once per project)

source .venv/bin/activate      # macOS / Linux
.venv\Scripts\activate         # Windows
```

```
python -m pip install requests # goes into .venv, not system Python
deactivate                    # when you're done
```

Your prompt changes to show `(.venv)` when it is active. Add `.venv/` to `.gitignore` — you never commit it; you commit the *list* of what to install (Chapter 23).

Rule to internalise now: **one project, one virtual environment, always.**

Editor

VS Code with the Python extension, or PyCharm. Either gives you syntax highlighting, autocompletion, and — most importantly — red squiggles under mistakes before you run anything. Configure it to use your `.venv` interpreter (VS Code: Command Palette → "Python: Select Interpreter").

Exercises. (1) Create a project folder, make a venv, install `requests` inside it, and prove it is not installed outside. (2) Run the same script three ways: from the terminal, from your editor's run button, and by pasting it into the REPL. (3) Deliberately create `math.py` containing `print("oops")`, then `import math` from a script in the same folder. Read what happens. Delete it.

Chapter 2 — Syntax Basics

Variables

A variable is a name pointing at a value. No declaration, no type annotation required:

```
count = 10
price = 19.99
name = "Ada Lovelace"
is_active = True
```

Names may contain letters, digits and underscores, and may not start with a digit. Convention (and everyone follows it): `lower_snake_case` for variables and functions, `UPPER_SNAKE_CASE` for constants, `CapWords` for classes.

Assignment is rebinding, not copying a box. `a = b` makes both names point at the same object. For numbers and strings this never matters; for lists it matters enormously (Chapter 7).

Multiple assignment and swapping:

```
x, y = 1, 2
x, y = y, x      # swap, no temporary needed
a = b = 0        # both point at the same 0
```

Indentation is syntax

Python uses indentation where other languages use braces. A consistent indent — **four spaces, never tabs** — defines a block:

```
if count > 5:
    print("big")
    print("still inside the if")
print("outside the if")
```

⚠ Mixing tabs and spaces produces `TabError` or, worse, code that runs but does the wrong thing. Set your editor to insert spaces when you press Tab. Every Python editor has this setting; turn it on now.

Comments

```
# a single-line comment

"""
A triple-quoted string. Used as a docstring when it is the first
statement in a module, function or class — that is what help() shows.
"""
```

Write comments that explain *why*, not *what*. `count += 1 # increment count` is noise; `count += 1 # retries are capped at 3 by the API` is worth reading.

print and input

```
print("Hello")
print("a", "b", "c")           # a b c   (space-separated)
print("a", "b", sep="-")      # a-b
print("no newline", end="")    # suppress the trailing newline
```

`input()` reads a line from the user and **always returns a string**:

```
name = input("What is your name? ")
age = input("How old are you? ")
print(f"Next year {name} will be {int(age) + 1}")
```

⚠ Forgetting `int()` is the classic first bug: `age + 1` on a string raises `TypeError`, and `age * 2` silently gives `"3030"` instead of `60`.

Exercises. (1) Write a script that asks for two numbers and prints their sum, product, and average. (2) Break it by entering "abc" and read the error. (3) Write a script whose output is a formatted receipt, using `print` with `sep` and `end`.

Chapter 3 — Data Types

Four types carry most beginner code.

```
n = 42          # int    - whole numbers, unlimited size
f = 3.14        # float  - decimals, ~15-17 significant digits
s = "text"      # str    - text
b = True        # bool   - True or False
nothing = None  # NoneType - "no value"
```

`type(x)` tells you what something is; `isinstance(x, int)` asks whether it is a given type.

Integers and floats

Python ints have no size limit — `2 ** 1000` just works. Floats do not, and they are approximate:

```
0.1 + 0.2        # 0.30000000000000004
0.1 + 0.2 == 0.3  # False
```

This is not a Python bug; it is how binary fractions work in every language. For now, remember two rules: compare floats with a tolerance (`abs(a - b) < 1e-9` or `math.isclose(a, b)`), and **never use floats for money** — use whole numbers of cents, or `Decimal` (Volume IV, Part XI).

Booleans and truthiness

`True` and `False` are the only bools, but every object is usable in a condition. **Falsy**: `False`, `None`, `0`, `0.0`, `""`, `[]`, `{}`, `()`, `set()`. Everything else is truthy.

```
items = []
if items:          # idiomatic - do not write len(items) > 0
    print("has items")
if name:           # catches both "" and None
    print(name)
```

⚠ `if x:` and `if x is not None:` are different. If `0` or `""` is a legitimate value, you must use the explicit `is not None`.

None

`None` means "no value yet" or "nothing to return." Test it with `is`, never `==`:

```
result = None
if result is None:
    result = compute()
```

Conversion

```
int("42")           # 42
int("42.7")         # ValueError – int() will not parse a decimal string
int(42.7)           # 42             – truncates toward zero, does not round
float("3.14")       # 3.14
str(42)             # "42"
bool("False")       # True!  any non-empty string is truthy
list("abc")         # ['a', 'b', 'c']
```

Conversion fails loudly, which is good. Wrap user input in `try/except ValueError` (Chapter 9).

Exercises. (1) Predict then check: `bool(0.0)`, `bool("0")`, `bool([])`, `bool([[]])`. (2) Write a converter that turns a string into a number if it can, and returns `None` otherwise. (3) Show three values where `int(x)` and `round(x)` disagree.

Chapter 4 — Strings

Creating and combining

```
single = 'quotes'
double = "quotes"           # identical; pick one and be consistent
multi = """spans
lines"""
escaped = "she said \"hi\"\\n" # \\n newline, \\t tab, \\ backslash
raw = r"C:\Users\new"        # raw string – backslashes are literal
```

Strings are **immutable**. Every operation returns a new string; nothing modifies in place.

```
"Hello" + " " + "World"    # concatenation
"ab" * 3                    # 'ababab'
```

⚠ Building a long string with `+=` in a loop is $O(n^2)$. Collect pieces in a list and `"".join(parts)` at the end.

Indexing and slicing

```
s = "Python"
s[0]      # 'P'      first
s[-1]     # 'n'      last
s[0:2]    # 'Py'     start inclusive, stop exclusive
s[:2]     # 'Py'     start defaults to 0
s[2:]     # 'thon'    stop defaults to the end
s[::2]    # 'Pto'    every second character
s[::-1]   # 'nohtyP' reversed
```

The half-open convention (**stop** excluded) is everywhere in Python. It means `s[:n] + s[n:] == s` for any `n`, and `len(s[a:b]) == b - a`. Slices never raise `IndexError` — out-of-range slices just give you less. Single-index access does raise.

Methods worth knowing

```
s.upper() / s.lower() / s.title() / s.capitalize()
s.strip() / s.lstrip() / s.rstrip()      # remove surrounding whitespace
s.replace("a", "b")
s.split(",")                          # "a,b,c" -> ['a','b','c']; split() splits on whitespace
",".join(["a","b"])                   # 'a,b' - note: called on the SEPARATOR
s.startswith("Py") / s.endswith(".txt")
s.find("th")                          # index, or -1 if absent
s.index("th")                         # index, or ValueError
"th" in s                             # True - the idiomatic membership test
s.count("t")
s.isdigit() / s.isalpha() / s.isspace()
s.zfill(5) / s.ljust(10) / s.rjust(10) / s.center(10)
s.removeprefix("Py")                  # [3.9]
```

Because strings are immutable, `s.strip()` does nothing unless you assign the result: `s = s.strip()`.

f-strings

The modern way to build strings [3.6+]:

```
name, score = "Ada", 0.9137
f"{name} scored {score}"              # 'Ada scored 0.9137'
f"{name.upper()} in {2 + 2} tries"    # any expression inside the braces
f"{score:.1%}"                       # '91.4%'
f"{score:.2f}"                       # '0.91'
f"{1234567:,}"                      # '1,234,567'
f"{name:>10}|"                       # right-aligned in 10 columns
f"{name=}"                          # 'name=Ada' - [3.8] debugging shortcut
```

The format spec after `:` is worth learning: `[[fill]align][sign][width][,][.precision][type]`. Types include `d` (int), `f` (fixed), `e` (scientific), `%` (percent), `x` (hex), `b` (binary).

`f"{name=}"` is the fastest debugging tool in the language — it prints both the expression and its value.

Exercises. (1) Write a function that reverses each word in a sentence but keeps word order. (2) Given `" NAME , email@x.com , 42 "`, produce a clean tuple `("name", "email@x.com", 42)`. (3) Format a table of names and prices so the columns line up, using only f-string specs.

Chapter 5 — Operators and Expressions

Arithmetic

```

7 + 2      # 9
7 - 2      # 5
7 * 2      # 14
7 / 2      # 3.5    - true division, ALWAYS returns a float
7 // 2     # 3      - floor division (rounds toward negative infinity)
7 % 2      # 1      - remainder
7 ** 2     # 49     - power
-7 // 2    # -4     ⚠ floors, does not truncate
-7 % 2     # 1      ⚠ result takes the sign of the divisor

```

`divmod(7, 2)` gives `(3, 1)` in one call — useful for converting seconds into hours/minutes.

Comparison and chaining

```

a == b    a != b    a < b    a <= b    a > b    a >= b
0 < x < 10      # chaining - evaluates x once, means (0 < x) and (x < 10)

```

`==` compares **values**; `is` compares **identity** (are these the same object?). Use `is` only for `None`, `True`, `False`, and sentinels. ⚠ `x is 1000` may be `False` even when `x == 1000`, because small integers are cached and larger ones are not. Modern Python warns you about this.

Logical operators

```

a and b    # if a is falsy, return a; else return b
a or b     # if a is truthy, return a; else return b
not a      # True/False

```

They **short-circuit** and they return an operand, not a bool. That gives two idioms:

```

name = user_input or "anonymous"    # default for empty/None
if user is not None and user.active: # safe: the second half only runs if the
    first passes

```

⚠ `x or default` treats `0` and `""` as missing. When those are valid values, use an explicit `if x is None`.

Assignment operators

```

x += 1    x -= 1    x *= 2    x /= 2    x //= 2    x %= 3    x **= 2

```

The walrus `:=` [3.8] assigns inside an expression:

```
while (line := f.readline()):  
    process(line)  
  
if (n := len(data)) > 100:  
    print(f"too big: {n}")
```

Precedence

From tightest to loosest: `**` → unary `-` → `*` / `/` / `//` / `%` → `+` / `-` → comparisons → `not` → `and` → `or`. When in any doubt, use parentheses. Readers should never have to recall this table.

Membership and identity

```
"a" in "cat"           # True  
3 in [1, 2, 3]         # True  
"k" in {"k": 1}        # True — checks KEYS  
x is None  
x is not None
```

Exercises. (1) Write a function converting seconds to `"1h 05m 09s"` using `divmod`. (2) Explain, with examples, why `-7 // 2` is `-4`. (3) Rewrite three nested `if` statements as one chained comparison plus `and`.

Chapter 6 — Control Flow

if / elif / else

```
if score >= 90:  
    grade = "A"  
elif score >= 80:  
    grade = "B"  
elif score >= 70:  
    grade = "C"  
else:  
    grade = "F"
```

Only the first matching branch runs. Order matters — if you put `score >= 70` first, everyone gets a C.

The conditional expression, for simple cases:

```
status = "pass" if score >= 60 else "fail"
```

for

`for` iterates over anything iterable — a list, a string, a dict, a file, a range:

```
for item in ["a", "b", "c"]:
    print(item)

for i in range(5):           # 0,1,2,3,4
    print(i)

for i in range(2, 10, 2):    # 2,4,6,8 — start, stop, step
    print(i)
```

Two indispensable helpers:

```
for index, value in enumerate(items, start=1):
    print(f"{index}. {value}")

for name, score in zip(names, scores):
    print(name, score)
```

⚠ `zip` stops at the shortest input silently. Use `zip(a, b, strict=True)` [3.10] when lengths must match.

⚠ **Never modify a list while iterating over it.** Items get skipped:

```
for x in items:
    if bad(x):
        items.remove(x)    # WRONG — shifts indices under the iterator

items = [x for x in items if not bad(x)]    # right
```

`while`

```
attempts = 0
while attempts < 3:
    if try_login():
        break
    attempts += 1
else:
    print("locked out")    # runs only if the loop ended WITHOUT break
```

The `else` clause on loops (both `for` and `while`) is unique to Python: it runs when the loop finished normally. It reads badly to most people; use it for search loops where "not found" needs handling, or use a flag or a function with an early `return`.

⚠ Every `while` needs something that changes the condition. Forgetting `attempts += 1` gives you an infinite loop; Ctrl-C stops it.

break, continue, pass

- `break` — leave the innermost loop immediately.
- `continue` — skip to the next iteration.
- `pass` — do nothing; a placeholder to keep a block syntactically valid.

To break out of nested loops, put them in a function and `return`, or use a flag. Python has no labelled break.

match [3.10]

```
match command.split():
    case ["go", direction]:
        move(direction)
    case ["take", *items]:
        pick_up(items)
    case ["quit"] | ["exit"]:
        return
    case _:
        print("unknown command")
```

`match` does structural pattern matching, not just value comparison — it destructures sequences, mappings, and objects. It is not a faster `if` chain; use it when you are taking apart shapes.

Exercises. (1) FizzBuzz for 1–100. (2) A number-guessing game using `while`, `break`, and input validation. (3) Given a list of dicts, find the first entry matching a condition using `for/else`, then rewrite it with `next()` and a default.

Chapter 7 — Collections

Four built-in containers. Choosing the right one is most of the work.

	Ordered	Mutable	Duplicates	Lookup cost	Use for
<code>list</code>	yes	yes	yes	$O(n)$ search	sequences you change
<code>tuple</code>	yes	no	yes	$O(n)$ search	fixed records, dict keys
<code>set</code>	no*	yes	no	$O(1)$	membership, dedup
<code>dict</code>	yes (insertion)	yes	keys unique	$O(1)$	lookup by key

* sets have no meaningful order; do not rely on iteration order.

list



```

xs = [3, 1, 2]
xs.append(4)           # add one at the end
xs.extend([5, 6])      # add many          (xs += [5,6] is the same)
xs.insert(0, 0)        # insert at index – O(n), avoid in loops
xs.remove(1)           # remove first matching VALUE (ValueError if absent)
xs.pop()              # remove and return last
xs.pop(0)             # remove and return first – O(n)
xs.sort()              # sorts IN PLACE, returns None
xs.sort(reverse=True)
xs.reverse()          # in place
xs.index(3)           # first index of a value
xs.count(3)
xs.clear()
sorted(xs)            # returns a NEW sorted list, leaves xs alone

```

⚠ `xs = xs.sort()` sets `xs` to `None`. In-place methods return `None` by design, to stop you from chaining and losing data.

Indexing and slicing work exactly as for strings, and slices of lists are **copies**:

```
shallow = xs[:]        # a copy of the list...
```

...but only one level deep. `shallow[0]` and `xs[0]` are still the same object. For nested structures use `copy.deepcopy`.

References, the big one:

```

a = [1, 2, 3]
b = a                # NOT a copy – same list, two names
b.append(4)
a                    # [1, 2, 3, 4]

b = a.copy()         # now independent

```

This is the difference between mutable and immutable types, and it is the source of most beginner mysteries. Volume III, Chapter 24 explains the underlying model.

tuple

```

point = (3, 4)
x, y = point         # unpacking
single = (42,)       # ⚠ the comma makes it a tuple, not the parentheses

```

Immutable, so tuples can be dictionary keys and set members. Use them for fixed-size records where position has meaning (coordinates, RGB values, database rows) and for returning several values from a function.

`namedtuple` gives the same thing with names (Chapter 20).

set

```
s = {1, 2, 3}
empty = set()           # ⚠️ {} is an empty DICT, not a set
s.add(4)
s.discard(9)           # no error if absent; s.remove(9) raises KeyError
s | t                   # union
s & t                   # intersection
s - t                   # difference
s ^ t                   # symmetric difference
s <= t                  # subset
```

The killer feature is $O(1)$ membership. Checking `x in big_list` scans; `x in big_set` does not. Converting a list to a set for repeated lookups is often the single biggest speed-up a beginner program can get.

```
unique = list(dict.fromkeys(items))    # dedupe AND keep order
unique_unordered = list(set(items))    # dedupe, order lost
```

Set elements must be hashable — so no lists inside sets. Use `frozenset` or tuples.

dict

```
person = {"name": "Ada", "age": 36}
person["name"]           # 'Ada' — KeyError if missing
person.get("email")      # None — no error
person.get("email", "n/a") # default
person["email"] = "a@b.c" # add or overwrite
person.setdefault("tags", []).append("x")
del person["age"]
person.pop("age", None)   # remove with a default

person.keys() / .values() / .items()
for key, value in person.items():
    print(key, value)

"name" in person          # checks KEYS
{**a, **b}                # merge (b wins)
a | b                     # merge [3.9]
person.update(other)
```

Dicts preserve insertion order [3.7+]. Keys must be hashable (strings, numbers, tuples — not lists).

Counting idiom:

```
counts = {}
for word in words:
    counts[word] = counts.get(word, 0) + 1
# or, better:
from collections import Counter
counts = Counter(words)
```

Nesting

Real data is nested — lists of dicts, dicts of lists:

```
users = [
    {"name": "Ada", "langs": ["python", "c"]},
    {"name": "Linus", "langs": ["c"]},
]
for user in users:
    print(user["name"], ", ".join(user["langs"]))
```

Reach into deep structures defensively: `data.get("a", {}).get("b")` rather than `data["a"]["b"]` when a level might be missing.

Exercises. (1) Count word frequencies in a paragraph, print the top five. (2) Given two lists of user IDs, find those in both, in either, and in one but not the other — using sets. (3) Demonstrate the aliasing bug with a list of lists created by `[[0] * 3] * 3`, and explain it. (4) Invert a dict (values become keys), handling duplicate values.

Chapter 8 — Functions

```
def greet(name, greeting="Hello"):
    """Return a greeting for name."""
    return f"{greeting}, {name}!"

greet("Ada")           # 'Hello, Ada!'
greet("Ada", "Hi")     # positional
greet(name="Ada", greeting="Hi") # keyword — clearer at call sites
greet(greeting="Hi", name="Ada") # keyword args can be reordered
```

A function without an explicit `return` returns `None`.

Why functions

Not "to avoid repetition" — that is a side effect. The reason is that a function gives a name to an idea and a boundary to a bug. If `parse_date` is wrong, you fix it in one place and you can test it in isolation.

Guidelines: one job per function; a name that is a verb phrase; short enough to see at once; and returning a value rather than printing, so it is testable and reusable.

Default arguments

⚠ **The mutable default trap.** Defaults are evaluated **once**, when the function is defined:

```
def add_item(item, basket=[]):      # WRONG
    basket.append(item)
    return basket

add_item("a")      # ['a']
add_item("b")      # ['a', 'b']  - the same list, forever

def add_item(item, basket=None):    # RIGHT
    if basket is None:
        basket = []
    basket.append(item)
    return basket
```

This will catch you at least once. Now it has caught you here instead.

Returning multiple values

```
def min_max(xs):
    return min(xs), max(xs)      # actually returns a tuple

low, high = min_max([3, 1, 4])
```

Scope

Names inside a function are local. Reading an outer name works; assigning to it creates a new local:

```
count = 0
def increment():
    count += 1      # UnboundLocalError - assignment makes `count` local

def increment():
    global count    # works, but avoid
    count += 1

def increment(count): # better: take it in, give it back
    return count + 1
```

The lookup order is **Local** → **Enclosing** → **Global** → **Built-in** (LEGB). Prefer parameters and return values over `global`; global state makes code untestable and produces bugs that depend on execution order.

⚠ Do not shadow built-ins: naming a variable `list`, `dict`, `str`, `id`, `type`, `sum`, or `input` breaks the built-in for the rest of the scope.

Docstrings and annotations

```
def bmi(weight_kg: float, height_m: float) -> float:
    """Return Body Mass Index.

    Args:
        weight_kg: mass in kilograms.
        height_m: height in metres.

    Returns:
        BMI as kg/m².
    """
    return weight_kg / height_m ** 2
```

Annotations do not enforce anything at runtime (Chapter 22 explains what they *are* for). Docstrings are shown by `help(bmi)` and by your editor. Write them for anything another person — including future you — will call.

Exercises. (1) Write `is_palindrome(s)` that ignores case and punctuation, with a docstring and three test calls. (2) Demonstrate the mutable-default bug, then fix it. (3) Write a function that takes a list of dicts and a key name and returns the average of that key, handling missing keys sensibly. (4) Explain why `def f(): x += 1` fails when `x` is global.

Chapter 9 — Errors

Reading a traceback

Do not panic and do not skim. Tracebacks read **bottom-up for the cause, top-down for the path**:

```
Traceback (most recent call last):
  File "app.py", line 12, in <module>
    main()
  File "app.py", line 8, in main
    total = compute(values)
  File "app.py", line 4, in compute
    return sum(v["amount"] for v in values)
KeyError: 'amount'
```

The **last line** is what went wrong: a dictionary had no `'amount'` key. The line above it is where: `app.py` line 4. Everything above that is how you got there. [3.11+] the error even underlines the exact failing subexpression.

The errors you will meet first:

Exception	Means
-----------	-------

Exception	Means
<code>SyntaxError</code>	the code could not be parsed — usually a missing <code>:</code> , bracket or quote <i>on the line before</i> the one reported
<code>IndentationError</code> / <code>TabError</code>	inconsistent indentation
<code>NameError</code>	a name that does not exist — typo, or used before assignment
<code>TypeError</code>	wrong type for an operation (<code>"3" + 4</code>)
<code>ValueError</code>	right type, impossible value (<code>int("abc")</code>)
<code>IndexError</code>	list index out of range
<code>KeyError</code>	dict key missing
<code>AttributeError</code>	no such attribute/method — often " <code>NoneType</code> has no attribute X", meaning something returned <code>None</code>
<code>ZeroDivisionError</code>	self-explanatory
<code>FileNotFoundError</code>	wrong path, or wrong working directory
<code>ModuleNotFoundError</code>	not installed, or wrong virtual environment

⚠ `'NoneType' object has no attribute ...` almost always means you used the result of an in-place method: `xs = xs.sort()`, `s = s.replace(...)` forgotten, or a function that fell off the end without returning.

try / except

```
try:
    age = int(input("Age: "))
except ValueError:
    print("Please enter a whole number.")
```

Rules from day one:

```
try:
    risky()
except (ValueError, TypeError) as exc:    # catch specific types
    print(f"could not process: {exc}")
except FileNotFoundError:
    print("missing file")
else:
    print("worked")                        # runs only if nothing raised
finally:
    cleanup()                             # ALWAYS runs
```

⚠ **Never write a bare `except:` or `except Exception: pass`.** It swallows typos, keyboard interrupts, and real bugs, and turns a five-second fix into a two-day investigation. Catch what you can actually handle.

Catch narrowly and close to the problem. If you cannot do anything useful about an error, let it propagate — a crash with a traceback is far better than silently wrong output.

Raising

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError(f"amount must be positive, got {amount}")
    if amount > balance:
        raise ValueError(f"insufficient funds: {amount} > {balance}")
    return balance - amount
```

Good error messages contain the offending value. `"invalid input"` helps nobody.

Validation loop

The pattern you will write a hundred times:

```
while True:
    try:
        age = int(input("Age: "))
    except ValueError:
        print("Numbers only, please.")
        continue
    if 0 <= age <= 130:
        break
    print("That's not a plausible age.")
```

Exercises. (1) Deliberately trigger each error in the table above and read the traceback. (2) Write a robust "read a positive number from the user" function. (3) Take a script that crashes on missing files and make it report the problem usefully instead. (4) Explain why `except Exception: pass` is worse than no error handling at all.

Chapter 10 — File I/O

Reading and writing text

Always use `with` — it closes the file even if an exception is raised:

```
with open("notes.txt", "r", encoding="utf-8") as f:
    content = f.read()  # whole file as one string

with open("notes.txt", encoding="utf-8") as f:
```

```
lines = f.readlines()           # list of lines, newlines included

with open("notes.txt", encoding="utf-8") as f:
    for line in f:               # BEST: streams, constant memory
        print(line.rstrip("\n"))
```

Writing:

```
with open("out.txt", "w", encoding="utf-8") as f:    # "w" TRUNCATES the file
    f.write("first line\n")                         # write does not add \n
    f.writelines(["a\n", "b\n"])

with open("log.txt", "a", encoding="utf-8") as f:    # "a" appends
    f.write("another entry\n")
```

Modes: **r** read (default), **w** write/truncate, **a** append, **x** create-or-fail, **b** binary, **+** read-and-write.

Always pass `encoding="utf-8"`. The default depends on the operating system's locale, which is why a script that works on your Mac produces `UnicodeDecodeError` on a colleague's Windows machine.

Paths

```
from pathlib import Path

p = Path("data") / "reports" / "2026.txt"    # / joins, cross-platform
p.exists() / p.is_file() / p.is_dir()
p.parent / p.name / p.stem / p.suffix
p.parent.mkdir(parents=True, exist_ok=True)
text = p.read_text(encoding="utf-8")          # shortcut, no `with` needed
p.write_text(text, encoding="utf-8")
for f in Path("data").glob("*.csv"):
    print(f)
```

`pathlib` replaces string concatenation and `os.path`. Never build paths with `+` and `"/"` — it breaks on Windows.

⚠ `FileNotFoundError` usually means your **working directory** is not what you think. `Path.cwd()` tells you; `Path(__file__).parent` gives you the script's own folder.

CSV

```
import csv

with open("people.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["age"])    # values are STRINGS
```

```
with open("out.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "age"])
    writer.writeheader()
    writer.writerows([{"name": "Ada", "age": 36}])
```

⚠ `newline=""` is required. Without it you get blank rows between records on Windows. And note that CSV has no types — convert with `int(row["age"])` yourself.

Do not parse CSV by `line.split(",")`. Real CSV has quoted fields containing commas and newlines; the `csv` module handles them and your split does not.

JSON

```
import json

with open("config.json", encoding="utf-8") as f:
    config = json.load(f)          # file -> dict

with open("out.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2)  # dict -> file

s = json.dumps(data)              # dict -> string
d = json.loads(s)                 # string -> dict
```

`load/dump` work on files; `loads/dumps` on strings ("s" for string).

Exercises. (1) Read a CSV of sales, compute totals per region, write a summary CSV. (2) Write a script that appends a timestamped line to a log file each run. (3) Handle a missing input file gracefully with a helpful message. (4) Read a JSON config with defaults for any missing key.

Chapter 11 — Modules and the Standard Library

Importing

```
import math                # math.sqrt(9)
import math as m           # m.sqrt(9)
from math import sqrt, pi  # sqrt(9)
from math import sqrt as square_root
```

⚠ Avoid `from module import *`. It hides where names came from and silently overwrites your own.

Your own files are modules too. `helpers.py` next to your script is `import helpers`.

`math`

```

math.sqrt(16)      math.pow(2, 10)      math.floor(3.7)    math.ceil(3.2)
math.pi            math.e            math.inf           math.nan
math.isclose(a, b) math.log(x, base)  math.factorial(5)
math.gcd(12, 18)   math.dist(p, q)    math.hypot(3, 4)

```

Use `round(x, 2)` for rounding to decimals (note it rounds ties to even: `round(2.5) == 2`).

random

```

import random
random.random()           # float in [0.0, 1.0)
random.randint(1, 6)      # inclusive on BOTH ends
random.randrange(0, 10, 2) # exclusive stop, like range
random.choice(["a", "b"])
random.choices(pop, weights=w, k=3) # with replacement
random.sample(pop, k=3)             # without replacement
random.shuffle(xs)                  # in place, returns None
random.seed(42)                     # reproducible results

```

⚠ `random` is not secure. For passwords, tokens, or anything an attacker should not predict, use `secrets` (`secrets.token_urlsafe(32)`, `secrets.choice(...)`).

datetime

```

from datetime import datetime, date, timedelta, timezone

now = datetime.now(timezone.utc)      # aware – always prefer this
today = date.today()
d = datetime(2026, 8, 25, 14, 30)

now + timedelta(days=7, hours=3)
(later - earlier).days
(later - earlier).total_seconds()

now.strftime("%Y-%m-%d %H:%M")        # datetime -> string
datetime.strptime("2026-08-25", "%Y-%m-%d") # string -> datetime
datetime.fromisoformat("2026-08-25T14:30:00+00:00")

```

Format codes: `%Y` 4-digit year, `%m` month, `%d` day, `%H` hour (24h), `%M` minute, `%S` second, `%A` weekday name, `%B` month name.

Two rules to adopt now and never break: **work in UTC**, **convert only for display**, and **never use `datetime.utcnow()`** — it returns a naive datetime that pretends to be local and causes bugs that surface months later. Volume IV, Part XI has the full treatment.

os, sys and pathlib

```
import os, sys

os.environ.get("API_KEY", "")      # environment variables
os.getcwd()                       # current directory
os.listdir(".")                   # prefer Path.iterdir()
os.makedirs("a/b", exist_ok=True) # prefer Path.mkdir(parents=True, exist_ok=True)
os.remove(path) / os.rename(a, b)
os.cpu_count()

sys.argv                          # command-line arguments; argv[0] is the script
sys.exit(1)                       # exit with a status code
sys.platform                      # 'linux', 'darwin', 'win32'
```

Prefer `pathlib` over `os.path` for anything path-shaped. Keep `os` for environment variables and process-level concerns.

Secrets belong in environment variables, never in your source code:

```
api_key = os.environ.get("API_KEY")
if not api_key:
    sys.exit("API_KEY is not set")
```

Other modules to know exist

`json` (Chapter 10), `csv` (Chapter 10), `re` (regular expressions), `collections` (Chapter 20), `statistics` (`mean`, `median`, `stdev`), `time` (`time.sleep`, `time.perf_counter`), `argparse` (command-line interfaces), `urllib.request` (basic HTTP — but install `requests` or `httpx` for anything real), `sqlite3` (a real database, no server required), `subprocess` (run other programs), `logging` (better than `print` for anything long-lived), `unittest`/`pytest` (Chapter 21).

Browse the standard library index once. Not to memorise it — just so that later, when you need to shuffle a deck or parse a date or spawn a process, you remember that Python already has it.

Exercises. (1) Write a dice-rolling simulator that reports the distribution over 10,000 rolls. (2) Write a script that prints how many days until a date given on the command line. (3) Read a configuration value from the environment with a sensible default and a clear error if it is required. (4) Use `pathlib` to find every `.log` file under a directory tree larger than 1 MB.

Where you are now

You can write real programs. What you cannot yet do — and what Volume II is for — is structure a program that grows beyond one file, express ideas that Python has better tools for than loops and `ifs`, and prove your code works. That is the difference between writing Python and building software.

Volume II — Intermediate

You can write scripts. This volume is about writing *software*: code with structure, tests, types, and a version history — code that someone else can change without fear.

Chapter 12 — Comprehensions and Generator Expressions

A comprehension builds a collection from an iterable in one expression.

```
squares = [x * x for x in range(10)]
evens   = [x for x in nums if x % 2 == 0]
labels  = [f"#{i}" for i in ids]
matrix  = [[0] * 3 for _ in range(3)]          # 3 INDEPENDENT rows
pairs   = [(x, y) for x in "ab" for y in (1, 2)]
```

The general shape: `[expression for item in iterable if condition]`. The `if` filters; a conditional *expression* transforms:

```
[x if x > 0 else 0 for x in nums]          # note: the conditional goes BEFORE `for`
```

⚠ `[[0] * 3] * 3` creates three references to **one** row — changing `m[0][0]` changes all three. The comprehension above does not. This is the aliasing lesson from Chapter 7 applied.

Dict and set comprehensions

```
{k: v for k, v in pairs}
{word: len(word) for word in words}
{v: k for k, v in mapping.items()}      # invert
{w.lower() for w in words}              # set – deduplicates
```

Generator expressions

Swap the brackets for parentheses and nothing is built in memory:

```
total = sum(x * x for x in range(1_000_000))    # constant memory
any(line.startswith("ERROR") for line in f)     # stops at the first hit
```

Use a list comprehension when you need the result more than once or need indexing; a generator expression when you are feeding a consumer (`sum`, `max`, `any`, `all`, `join`, a `for` loop) — especially over large or infinite sources.

When *not* to use one

Comprehensions are for building a collection. If the body has side effects, or you need more than one condition and a nested loop, write the loop:

```
# unreadable
result = [transform(x) for sub in data for x in sub if x and check(x) and x.ok]

# fine
result = []
for sub in data:
    for x in sub:
        if x and check(x) and x.ok:
            result.append(transform(x))
```

A good rule: if it does not fit on one line at a comfortable width, or it needs a comment, expand it.

Exercises. (1) Rewrite five loops from your Volume I exercises as comprehensions; keep whichever version reads better. (2) Build a word-length index (`{length: [words]}`) with a dict comprehension plus `defaultdict`, then without. (3) Show memory usage of a list comprehension versus a generator expression over 10 million items.

Chapter 13 — `*args`, `**kwargs`, Keyword-Only Arguments, Unpacking

Variadic parameters

```
def total(*numbers):          # numbers is a TUPLE
    return sum(numbers)

total(1, 2, 3)

def configure(**options):    # options is a DICT
    for key, value in options.items():
        print(key, value)

configure(debug=True, retries=3)

def call(func, *args, **kwargs): # the universal forwarding signature
    return func(*args, **kwargs)
```

The names `args` and `kwargs` are convention, not syntax; `*` and `**` are the syntax.

Unpacking at the call site

The same operators go the other way:

```
nums = [1, 2, 3]
total(*nums)          # same as total(1, 2, 3)
```

```

opts = {"debug": True, "retries": 3}
configure(**opts)

merged = [*a, *b] # combine lists
merged = {**defaults, **overrides} # combine dicts, later wins

```

Unpacking in assignment

```

first, *rest = [1, 2, 3, 4] # first=1, rest=[2,3,4]
*init, last = [1, 2, 3, 4] # init=[1,2,3], last=4
a, (b, c) = 1, (2, 3) # nested
head, *_ , tail = items # _ conventionally means "ignored"

```

Positional-only and keyword-only

```

def render(doc, /, *, strict=False, encoding="utf-8"):
    ...

```

- Everything before `/` is **positional-only**: callers cannot use its name, so you are free to rename it later.
- Everything after `*` is **keyword-only**: callers must write `strict=True`.

Make every boolean and every optional flag keyword-only. `render(doc, True, False)` is unreadable and impossible to extend safely; `render(doc, strict=True)` is neither.

Exercises. (1) Write a `log(*parts, sep=" ", level="INFO")` function. (2) Write a decorator-free wrapper that logs any function's arguments before calling it. (3) Convert a function with five positional parameters into one with two positional and three keyword-only, and explain what that buys you.

Chapter 14 — Lambdas, `map/filter`, and `sorted` with Keys

lambda

An anonymous, single-expression function:

```

square = lambda x: x * x # ⚠ if you name it, use def instead
sorted(words, key=lambda w: len(w))

```

Lambdas are for passing a tiny function *as an argument*. Assigning one to a name gains you nothing over `def` and loses a useful `__name__` in tracebacks.

sorted with keys — the workhorse

```
sorted(words, key=len)
sorted(words, key=str.lower)           # case-insensitive
sorted(people, key=lambda p: p["age"], reverse=True)
sorted(people, key=lambda p: (p["last"], p["first"])) # multi-level
sorted(items, key=lambda x: (-x.score, x.name))      # score desc, name asc
```

The key function is called **once per element** and the results are sorted — so it can be expensive without being called $O(n \log n)$ times. Python's sort is **stable**: equal keys keep their original order, which lets you sort by one field, then another, to get a compound ordering.

`operator.itemgetter` and `attrgetter` are faster and clearer than lambdas for field access:

```
from operator import itemgetter, attrgetter
sorted(rows, key=itemgetter("age"))
sorted(objs, key=attrgetter("created_at"))
max(rows, key=itemgetter("score"))
```

`min`, `max`, and `sorted` all take `key`. So does `itertools.groupby` — and it needs the data sorted by the same key first.

map and filter

```
list(map(str.upper, words))
list(filter(None, items)) # drops falsy values
```

Both return lazy iterators. In modern Python, a comprehension is usually clearer:

```
[w.upper() for w in words] # preferred over map
[x for x in items if x]    # preferred over filter
```

`map` earns its place when you already have a named function and no transformation to express — `map(int, line.split())` is idiomatic.

`functools.reduce` exists but is rarely the clearest option; `sum`, `math.prod`, `any`, `all`, and `itertools.accumulate` cover most real uses.

Exercises. (1) Sort a list of dicts by three fields with mixed ascending/descending order. (2) Rewrite three `map/filter` chains as comprehensions and compare readability. (3) Explain why sorting stability lets you achieve multi-key sorting with repeated single-key sorts, and when that is worse than a tuple key.

Chapter 15 — Object-Oriented Programming

Classes and instances

```

class Account:
    interest_rate = 0.02                # class attribute – shared

    def __init__(self, owner, balance=0): # runs on construction
        self.owner = owner              # instance attributes – per object
        self.balance = balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("amount must be positive")
        self.balance += amount
        return self.balance

    def __repr__(self):
        return f"Account(owner={self.owner!r}, balance={self.balance})"

a = Account("Ada", 100)
a.deposit(50)

```

`self` is the instance, passed automatically. It is a parameter name by convention, not a keyword — but never call it anything else.

⚠ Mutable class attributes are shared by every instance. `class C: items = []` gives every object the same list. Initialise mutable state in `__init__`.

When to use a class

When data and behaviour belong together and there is state to keep. A class with one method and no state should be a function. A class that is only data should be a dataclass (Chapter 33). Do not create a `Manager/Helper/Utils` class to hold functions — a module already does that.

Inheritance

```

class SavingsAccount(Account):
    def __init__(self, owner, balance=0, rate=0.05):
        super().__init__(owner, balance)    # always call super().__init__
        self.rate = rate

    def add_interest(self):
        self.deposit(self.balance * self.rate)

```

Inheritance means *is-a*. A `SavingsAccount` is an `Account`. If the relationship is *has-a* — a `Car` has an `Engine` — use **composition**: store the other object as an attribute. Composition is more flexible, easier to test, and should be your default. Deep inheritance hierarchies are one of the most reliable sources of unmaintainable code.

`isinstance(obj, Account)` checks type including subclasses. Avoid `type(obj) == Account`.

Dunder methods

Special methods let your objects work with built-in syntax:

```
class Money:
    def __init__(self, cents): self.cents = cents
    def __repr__(self): return f"Money({self.cents})"           # for developers
    def __str__(self): return f"${self.cents / 100:.2f}"         # for users
    def __eq__(self, other):
        if not isinstance(other, Money): return NotImplemented
        return self.cents == other.cents
    def __hash__(self): return hash(self.cents)                 # needed if
__eq__
    def __lt__(self, other): return self.cents < other.cents     # enables sorting
    def __add__(self, other): return Money(self.cents + other.cents)
    def __len__(self): return ...                                # only if "length" makes sense
    def __bool__(self): return self.cents != 0
    def __iter__(self): ...                                       # makes it usable in a for loop
    def __getitem__(self, key): ...                               # enables obj[key]
    def __contains__(self, x): ...                                # enables `x in obj`
    def __call__(self, ...): ...                                  # makes instances callable
```

Two you should write for nearly every class: `__repr__` (so debugging output is useful — make it look like the constructor call) and `__eq__` (so tests can compare objects). Remember that defining `__eq__` makes the class unhashable unless you also define `__hash__`.

`functools.total_ordering` fills in the other comparisons from `__eq__` and `__lt__`.

Properties

Start with plain attributes. When you later need validation or a computed value, `@property` lets you add it **without changing any calling code**:

```
class Temperature:
    def __init__(self, celsius=0.0):
        self.celsius = celsius                                # goes through the setter

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("below absolute zero")
        self._celsius = value

    @property
    def fahrenheit(self):                                     # read-only, computed
        return self._celsius * 9 / 5 + 32
```

This is why Python does not need getters and setters everywhere: you can add them later. Writing `get_x()/set_x()` from the start is a Java habit that Python does not reward.

Class and static methods

```
class Date:
    def __init__(self, y, m, d): ...

    @classmethod
    def from_string(cls, s):          # alternative constructor
        y, m, d = map(int, s.split("-"))
        return cls(y, m, d)          # cls, so subclasses work

    @staticmethod
    def is_leap(year):                # related, but needs no state
        return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)
```

`@classmethod` for alternative constructors and anything that needs the class; `@staticmethod` for a function that lives in the class only for organisation.

Naming conventions

- `name` — public.
- `_name` — internal. Not enforced; a signal to other developers that it may change.
- `__name` — name-mangled to `__ClassName__name`; use only to avoid collisions in inheritance, not as "private".

Exercises. (1) Build an `Inventory` class with `add/remove/total`, `__len__`, `__contains__`, `__iter__`, and a good `__repr__`. (2) Convert a public attribute to a validated property without changing calling code. (3) Model a shape hierarchy with an abstract base, then re-model it with composition and compare. (4) Write `__eq__` and `__hash__` for a value object and put instances in a set.

Chapter 16 — Iterators and Generators

The protocol

Anything you can loop over is an **iterable**; `iter()` turns it into an **iterator**, which yields values via `__next__` until it raises `StopIteration`.

```
it = iter([1, 2, 3])
next(it)          # 1
next(it)          # 2
next(it, None)    # 3, then None instead of StopIteration
```

An iterator is consumed once. This explains a common confusion: after `list(gen)`, a second `list(gen)` is empty.

Generators

A function with `yield` returns a generator. Execution pauses at each `yield` and resumes on the next request:

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
    return "done"          # becomes StopIteration.value

for x in countdown(3):
    print(x)               # 3, 2, 1
```

Why this matters: **memory**. Reading a 10 GB file with `readlines()` needs 10 GB of RAM; a generator needs one line at a time.

```
def read_errors(path):
    with open(path, encoding="utf-8") as f:
        for line in f:
            if "ERROR" in line:
                yield line.rstrip()

for line in read_errors("app.log"):    # constant memory, any file size
    print(line)
```

Generators can be infinite, because nothing is materialised:

```
def naturals():
    n = 0
    while True:
        yield n
        n += 1

from itertools import islice
first_ten = list(islice(naturals(), 10))
```

yield from

Delegate to another iterable:

```
def flatten(nested):
    for item in nested:
        if isinstance(item, list):
```

```

        yield from flatten(item)    # recursion made readable
    else:
        yield item

```

Pipelines

Generators compose into pipelines that stream:

```

lines    = (l for l in open("data.log", encoding="utf-8"))
records  = (parse(l) for l in lines if l.strip())
recent   = (r for r in records if r.date >= cutoff)

for record in recent:
    save(record)

```

Nothing is read until the final loop pulls, and memory stays flat regardless of file size.

Custom iterables

```

class Deck:
    def __init__(self, cards): self._cards = cards
    def __iter__(self):
        return iter(self._cards)    # fresh iterator each time – reusable
    def __len__(self):
        return len(self._cards)

```

⚠ If `__iter__` returns `self` and mutates state, your object works in exactly one `for` loop and then silently appears empty.

Exercises. (1) Write a generator producing Fibonacci numbers indefinitely; take the first 20. (2) Build a three-stage pipeline over a large CSV using only generators and prove memory stays constant. (3) Write `chunked(iterable, n)` yielding lists of size `n`. (4) Show a bug caused by consuming a generator twice, and fix it two ways.

Chapter 17 — Closures and Decorators

Closures

A function defined inside another function remembers the enclosing variables:

```

def make_counter():
    count = 0
    def increment():
        nonlocal count    # without this, assignment creates a local
        count += 1

```

```

        return count
    return increment

c = make_counter()
c(); c()          # 1, 2

```

nonlocal writes to the enclosing scope; **global** writes to module scope. The captured variable is looked up **when the inner function runs**, not when it is defined — which causes the classic loop bug:

```

fs = [lambda: i for i in range(3)]
[f() for f in fs]          # [2, 2, 2] - all share one `i`
fs = [lambda i=i: i for i in range(3)]
[f() for f in fs]          # [0, 1, 2] - default binds at definition

```

Decorators

A decorator is a function that takes a function and returns a replacement. `@deco` above a `def` is exactly `f = deco(f)`.

```

import functools, time

def timed(func):
    @functools.wraps(func)          # preserve name, docstring,
    signature
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            print(f"{func.__name__} took {time.perf_counter() - start:.3f}s")
    return wrapper

@timed
def slow():
    time.sleep(1)

```

⚠ **Always use `@functools.wraps`**. Without it the wrapped function loses its `__name__` and `__doc__`, `help()` becomes useless, and pytest may fail to collect your tests.

Decorators with arguments

Three layers: a function that returns a decorator that returns a wrapper.

```

def retry(times=3, delay=1.0):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):

```

```

        for attempt in range(times):
            try:
                return func(*args, **kwargs)
            except Exception:
                if attempt == times - 1:
                    raise
                time.sleep(delay * 2 ** attempt)
        return wrapper
    return decorator

@retry(times=5, delay=0.5)
def fetch(url): ...

```

Ready-made decorators

```

from functools import cache, lru_cache, cached_property

@cache                                # [3.9] unbounded memoisation
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

@lru_cache(maxsize=1024)              # bounded
def lookup(key): ...

```

⚠️ Cached arguments must be hashable (no lists or dicts), and `@cache` on a *method* keeps every instance alive forever — a memory leak. Use `cached_property` for per-instance caching.

Stacking applies bottom-up: with `@a` above `@b`, `b` wraps the function first, then `a` wraps that.

Exercises. (1) Write a `@log_calls` decorator printing arguments and return values. (2) Write `@count_calls` that exposes a `.calls` attribute on the decorated function. (3) Demonstrate what breaks without `@wraps`. (4) Write a decorator that caches results to a JSON file on disk.

Chapter 18 — Exceptions in Depth

Custom exception classes

Define your own so callers can catch precisely what they can handle:

```

class AppError(Exception):
    """Base for everything this application raises."""

class ValidationError(AppError):
    def __init__(self, field, value):
        super().__init__(f"invalid {field}: {value!r}")
        self.field, self.value = field, value

```

```
class NotFoundError(AppError):
    pass
```

One base class per project or library means callers can write `except AppError:` and catch everything you raise without catching everything in the world. Carry structured attributes (`field`, `value`) so handlers do not have to parse message strings.

The full statement

```
try:
    result = risky()
except ValidationError as exc:
    log.warning("bad input on %s", exc.field)
    raise                # bare `raise` re-raises, preserving the traceback
except (IOError, OSError) as exc:
    raise AppError("storage unavailable") from exc    # keep the cause
else:
    commit(result)                # only if no exception
finally:
    release()                    # always, even on return or raise
```

- `raise X from exc` records the cause, so the traceback shows both. Losing the original exception is the most common debugging crime in wrapper code.
- Bare `raise` inside an `except` re-raises the current exception unchanged — use it after logging.
- `finally` runs even if the `try` block returns.

Context managers

`with` guarantees cleanup:

```
with open("f.txt") as f:        # closed even if an exception occurs
    data = f.read()
```

Write your own with `contextlib`:

```
from contextlib import contextmanager

@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield
    finally:
        print(f"{label}: {time.perf_counter() - start:.3f}s")
```

```
with timer("import"):
    load_everything()
```

Or as a class:

```
class Transaction:
    def __init__(self, conn): self.conn = conn
    def __enter__(self):
        self.conn.execute("BEGIN")
        return self.conn
    def __exit__(self, exc_type, exc, tb):
        if exc_type is None:
            self.conn.execute("COMMIT")
        else:
            self.conn.execute("ROLLBACK")
        return False # False = do not suppress the exception
```

Returning `True` from `__exit__` swallows the exception. Almost always wrong; be deliberate.

Useful members of `contextlib`: `suppress(FileNotFoundError)` (an honest scoped `try/except/pass`), `closing(obj)`, `nullcontext()` for optional context managers, and `ExitStack` for a variable number of them.

Exercises. (1) Design an exception hierarchy for a file-processing tool. (2) Write a context manager that changes the working directory and restores it. (3) Show what a traceback looks like with and without `raise ... from exc`. (4) Write a context manager that retries its body on failure — and explain why that is harder than it looks.

Chapter 19 — Modules, Packages and Project Structure

`__name__`

Every module has a `__name__`. It is `"__main__"` when the file is run directly, and the module's name when imported:

```
def main():
    ...

if __name__ == "__main__":
    main()
```

Without this guard, importing your module runs its script code — which breaks tests, tooling, and `multiprocessing`.

Packages

A package is a directory of modules with an `__init__.py`:

```
myproject/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/
│   └── myapp/
│       ├── __init__.py
│       ├── __main__.py      # enables `python -m myapp`
│       ├── config.py
│       ├── models.py
│       └── services/
│           ├── __init__.py
│           └── email.py
└── tests/
    ├── test_models.py
    └── test_services.py
```

`__init__.py` may be empty, or it may re-export the package's public API:

```
# src/myapp/__init__.py
from .models import User, Order
__all__ = ["User", "Order"]
__version__ = "0.1.0"
```

Imports inside a package

```
from myapp.models import User      # absolute – preferred
from .models import User           # relative – fine within a package
from ..config import settings      # up one level
```

Use absolute imports for clarity; relative imports are acceptable inside a package and make it renameable.

⚠ **Circular imports:** A imports B, B imports A. Symptoms are `ImportError: cannot import name X (most likely due to a circular import)` or an inexplicable `AttributeError`. Fixes: move the shared piece into a third module; import inside the function that needs it; or restructure so dependencies flow one way.

The `src/` layout

Putting your package under `src/` means it can only be imported when installed. That sounds annoying and it is the point: it makes "works on my machine because the file happened to be in the current directory" impossible.

```
python -m pip install -e .          # editable install of your own project
```

Exercises. (1) Split a 400-line script into a package with at least three modules and a `__main__.py`. (2) Create a circular import deliberately, then fix it three ways. (3) Make your package runnable both as `python -m myapp` and via a console script entry point.

Chapter 20 — The Working Library

pathlib

```
from pathlib import Path

Path.home() / ".config" / "app.toml"
p.resolve()                # absolute, symlinks followed
p.with_suffix(".bak")
p.with_name("other.txt")
p.relative_to(base)
list(Path("src").rglob("*.py")) # recursive glob
p.stat().st_size
p.unlink(missing_ok=True)
```

json

```
json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False, default=str)
json.loads(text)
```

`default=str` is the quick fix for "Object of type datetime is not JSON serializable"; the correct fix is an explicit encoder or a serialisation layer (Chapter 33). $\triangle$ JSON has no integer/float distinction beyond IEEE doubles, no bytes, no dates, and by default Python's `json` will happily emit `NaN`, which is invalid JSON — pass `allow_nan=False` when talking to other systems.

collections

```
from collections import Counter, defaultdict, deque, namedtuple, ChainMap

Counter(words).most_common(5)
Counter(a) - Counter(b)                # multiset arithmetic

groups = defaultdict(list)
for item in items:
    groups[item.kind].append(item)      # no key check needed

q = deque(maxlen=100)                  # O(1) at both ends; ring buffer
q.appendleft(x); q.popleft()
```

```
Point = namedtuple("Point", "x y")
p = Point(1, 2); p.x; p._asdict()

config = ChainMap(cli_args, env_vars, file_config, defaults)  # precedence
```

⚠ `defaultdict` inserts on read: `d[missing]` creates an entry. That silently grows the dict and breaks later membership tests.

itertools

```
from itertools import (chain, islice, groupby, product, combinations,
                      permutations, cycle, count, accumulate, pairwise, batched)

chain(a, b)                # concatenate iterables
islice(gen, 10)            # slice any iterator
batched(items, 100)        # [3.12] fixed-size chunks
pairwise([1,2,3])          # [3.10] (1,2), (2,3) – for deltas
accumulate(values)         # running totals
product(a, b)              # nested loops, flattened
combinations(items, 2)     # unordered pairs
groupby(sorted(rows, key=k), key=k)  # ⚠ requires SORTED input
```

⚠ `groupby` groups **consecutive** equal keys only. Sort first, or you get fragmented groups and a very confusing afternoon.

functools

```
from functools import wraps, cache, lru_cache, cached_property, partial, reduce

connect_local = partial(connect, host="localhost", port=5432)

class Report:
    @cached_property
    def rows(self):          # computed once per instance, then stored
        return expensive_query()
```

Exercises. (1) Replace every `os.path` call in an old script with `pathlib`. (2) Use `Counter` and `groupby` to produce a report from a CSV. (3) Build a config system with `ChainMap` layering CLI > env > file > defaults. (4) Write `chunked` yourself, then compare with `itertools.batched`.

Chapter 21 — Testing and Debugging

pytest

```
python -m pip install pytest
```

```
# tests/test_money.py
import pytest
from myapp.money import Money, parse_amount

def test_addition():
    assert Money(100) + Money(50) == Money(150)

def test_rejects_negative():
    with pytest.raises(ValueError, match="positive"):
        Money(-1)

@pytest.mark.parametrize(
    ("text", "expected"),
    [("1.00", 100), ("0.99", 99), ("1,000.00", 100_000)],
)
def test_parse(text, expected):
    assert parse_amount(text) == expected
```

Run with `pytest`, or `pytest -v` for detail, `pytest -k addition` to filter, `pytest -x` to stop at the first failure, `pytest --lf` to rerun only last failures.

Rules: test files are `test_*.py`, test functions are `test_*`, and you use plain `assert` — `pytest` rewrites it to show you both sides of the comparison on failure.

Fixtures

Fixtures provide test dependencies and clean up afterwards:

```
@pytest.fixture
def sample_file(tmp_path):
    # tmp_path is built in
    p = tmp_path / "data.csv"
    p.write_text("name,age\nAda,36\n", encoding="utf-8")
    return p

def test_load(sample_file):
    rows = load_csv(sample_file)
    assert rows[0]["name"] == "Ada"
```

Anything after `yield` in a fixture is teardown. Shared fixtures live in `conftest.py`, which `pytest` discovers automatically for every test below it.

What to test

Test **behaviour at the boundary**, not implementation details. Good candidates: edge cases (empty, one, many), error paths, the specific bug you just fixed (write the test *first*, watch it fail, then fix). Do not chase a

coverage number — a suite with no assertions can hit 100%.

Structure each test as **Arrange, Act, Assert**, and give it a name that describes the expectation:

`test_withdrawal_over_balance_raises`.

Debugging

```
breakpoint()           # [3.7] drops into pdb at this line
```

Inside pdb: `n` next line, `s` step into, `c` continue, `l` list source, `p` `expr` print, `pp` `expr` pretty-print, `w` where (the stack), `u/d` move up/down the stack, `b 42` set a breakpoint, `q` quit. `python -m pdb script.py` starts under the debugger; `pytest --pdb` drops in on failure.

Before reaching for the debugger, the cheapest tools are `print(f"{value=}")` and reading the traceback properly (Chapter 9). Once past those, `breakpoint()` beats scattering prints, because you can inspect *everything* at that moment rather than guessing what to print.

For anything long-lived, replace prints with `logging`:

```
import logging
log = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(name)s %(message)s")
log.info("processed %d rows", n)           # lazy %s formatting, not an f-string
```

Exercises. (1) Write tests for a function *before* implementing it. (2) Reproduce a bug with a failing test, then fix it. (3) Use `breakpoint()` to inspect a wrong intermediate value in a loop. (4) Parametrise one test over ten inputs including two edge cases.

Chapter 22 — Type Hints and mypy

The syntax

```
def greet(name: str, times: int = 1) -> str:
    return f"Hello {name}! " * times

count: int = 0
names: list[str] = []
scores: dict[str, float] = {}
maybe: str | None = None           # [3.10]
pair: tuple[int, str] = (1, "a")

from collections.abc import Iterable, Callable, Sequence
def process(items: Iterable[str], key: Callable[[str], int]) -> list[str]: ...
```

What they are for

Annotations do **nothing at runtime**. Their value is: your editor autocompletes and catches mistakes as you type; a checker finds bugs before you run the code; and they document intent more reliably than a comment, because a checker verifies them.

The most valuable single hint is `X | None`, because it forces you to handle the `None` case — which is where a large share of production crashes come from.

mypy

```
python -m pip install mypy
mypy src/
```

```
# pyproject.toml
[tool.mypy]
python_version = "3.12"
strict = true
```

`strict` is a lot on an existing codebase. Adopt it gradually: start with `warn_return_any` and `disallow_untyped_defs` on new modules, add `# type: ignore[error-code]` (always with the code) where you must, and delete those over time.

Practical rules

- Annotate **function signatures**; local variables only where inference fails.
- Accept general types, return specific ones: take `Iterable[str]`, return `list[str]`.
- Avoid `Any` — it disables checking for everything it touches.
- Use `TYPE_CHECKING` for imports needed only for annotations:

```
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from myapp.models import User      # not imported at runtime

def save(user: "User") -> None: ...
```

Volume III, Chapter 25 covers generics, protocols, `TypedDict` and overloads; Volume IV, Part II covers variance and the full system.

Exercises. (1) Annotate one module fully and run mypy until clean. (2) Find a real bug using `X | None` annotations. (3) Explain why annotations do not validate input, and what you would use instead at a trust boundary.

Chapter 23 — Git, requirements.txt and pyproject.toml

Git, the parts you need daily

```
git init
git status # run this constantly
git add file.py # stage
git add -p # stage selectively, hunk by hunk
git commit -m "Add CSV export"
git log --oneline --graph
git diff # unstaged changes
git diff --staged # staged changes

git switch -c feature/export # new branch
git switch main
git merge feature/export
git restore file.py # discard local changes to a file
git restore --staged file.py # unstage

git remote add origin <url>
git push -u origin main
git pull
```

Commit messages: a short imperative summary line ("Add CSV export", not "added stuff"), and a body explaining *why* if it is not obvious. Commit small and often; a commit should be one coherent change.

.gitignore for Python:

```
__pycache__/
*.py[cod]
.venv/
venv/
.env
.pytest_cache/
.mypy_cache/
.ruff_cache/
dist/
build/
*.egg-info/
.coverage
htmlcov/
.DS_Store
```

⚠ **Never commit secrets.** A **.env** file, an API key, a database password — once pushed, it is in the history forever even if you delete it in the next commit. Rotate the credential; do not just remove the file.

requirements.txt

The simple approach for applications:

```
python -m pip install requests
python -m pip freeze > requirements.txt      # snapshot everything
python -m pip install -r requirements.txt    # recreate elsewhere
```

⚠ **pip freeze** captures your entire environment including transitive dependencies and anything you installed by accident. Better practice: maintain a hand-written **requirements.in** of direct dependencies and compile it (**pip-compile**, or **uv pip compile**) into a fully pinned **requirements.txt**.

pyproject.toml

The modern standard, and the only file you need for a real project:

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "myapp"
version = "0.1.0"
description = "What this does"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "requests>=2.31",
    "pydantic>=2.7",
]

[project.optional-dependencies]
dev = ["pytest>=8", "mypy>=1.10", "ruff>=0.5"]

[project.scripts]
myapp = "myapp.__main__:main"

[tool.ruff]
line-length = 100
```

Then:

```
python -m pip install -e ".[dev]"    # your package plus dev tools, editable
```

Version constraints, the rule that saves you later: **applications pin exact versions** (via a lock file), **libraries specify minimum versions and avoid upper bounds** unless they have evidence of breakage. A library that pins **requests==2.31.0** is unusable alongside anything else.

Volume III, Chapter 31 covers publishing; Volume IV, Part IX covers the whole packaging standard.

Exercises. (1) Put an existing project under Git with a sensible `.gitignore` and ten meaningful commits. (2) Convert `requirements.txt` to `pyproject.toml` with a dev extra. (3) Make a branch, break something, and recover with `git restore` and `git switch`. (4) Set up a fresh machine (or container) from your repo alone and confirm it runs.

Where you are now

You can build, structure, test and version a real project. Volume III is about the decisions that only matter once other people depend on your code: how memory behaves, how to use more than one core, how to make it fast, how to design it so it survives change, and how to ship it.

Volume III — Advanced

Everything here is a decision you make on behalf of other people: the users who hit your memory ceiling, the colleague who has to change your class hierarchy, the on-call engineer reading your logs at 3am. Volume IV explains the mechanisms in full; this volume tells you what to do about them.

Chapter 24 — The Memory Model

Names, objects, references

A Python variable is a **name bound to an object**, not a box holding a value. Assignment rebinds the name; it never copies.

```
a = [1, 2, 3]
b = a           # two names, one list
b.append(4)
a              # [1, 2, 3, 4]

id(a) == id(b)  # True — same object
a is b         # True
```

Everything else follows from this. Function arguments are passed the same way — "pass by object reference." A function can mutate a list you passed it; it cannot rebind your name:

```
def mutate(xs): xs.append(4)    # caller sees this
def rebind(xs):  xs = [9]      # caller does not
```

Mutable vs immutable

Immutable: `int`, `float`, `str`, `bytes`, `tuple`, `frozenset`, `bool`, `None`. Operations return new objects. **Mutable:** `list`, `dict`, `set`, `bytearray`, most of your own classes.

Consequences that bite:

- Only immutable (hashable) objects can be dict keys or set members.
- Mutable default arguments persist across calls (Volume I, Chapter 8).
- A tuple is immutable but can *contain* a mutable list — so it may be unhashable, and its contents can change.
- `x += 1` on an int rebinds; `xs += [1]` on a list mutates in place. Same syntax, different semantics.

Copying

```
import copy
shallow = original[:]          # or list(original), or copy.copy(original)
deep    = copy.deepcopy(original)
```

Shallow copies duplicate the container, not the contents. For a list of dicts, the dicts are still shared. `deepcopy` recurses (and handles cycles), at real cost. Better than either: prefer immutable data and construct new values rather than copying and mutating.

Reference counting and garbage collection

CPython frees an object the instant its reference count hits zero — which is why `with open(...)` feels immediate and why `del big_thing` can actually free memory. A cyclic **garbage collector** exists only for reference cycles (`a.b = b; b.a = a`), which refcounting alone cannot reclaim.

Practical implications:

- **Memory is not returned to the OS promptly.** CPython's allocator holds freed small objects in arenas; RSS stays high after a spike. The industrial answer is not to create the spike, and to recycle worker processes.
- **Cycles are collected late**, so an object holding a large buffer may linger. Break cycles with `weakref` for back-pointers and caches.
- Long-lived caches, module-level lists, and unbounded queues are the usual causes of "memory grows forever." So is `@cache` on a method, and holding exception objects (which hold tracebacks, which hold every frame's locals).

```
import tracemalloc
tracemalloc.start()
snap1 = tracemalloc.take_snapshot()
run_workload()
for stat in tracemalloc.take_snapshot().compare_to(snap1, "lineno")[:10]:
    print(stat)
```

Cutting memory

In order of impact: use generators instead of lists; use `__slots__` on classes you create by the million; use `array`/NumPy for homogeneous numbers; stream instead of loading; and `sys.intern()` repeated strings from parsing.

Volume IV, Part IV covers the allocator, GC generations, and immortal objects.

Exercises. (1) Demonstrate a shallow-copy bug with nested data and fix it. (2) Build a reference cycle holding 100 MB, show RSS not dropping, then fix it with `weakref`. (3) Measure the memory of a million objects with and without `__slots__`.

Chapter 25 — Advanced Typing

Generics

```
from typing import TypeVar, Generic
T = TypeVar("T")

class Stack(Generic[T]):
    def __init__(self) -> None:
        self._items: list[T] = []
    def push(self, item: T) -> None: self._items.append(item)
    def pop(self) -> T: return self._items.pop()
```

[3.12] the same thing, with no imports and correct scoping:

```
class Stack[T]:
    def __init__(self) -> None:
        self._items: list[T] = []

def first[T](xs: list[T]) -> T:
    return xs[0]

type Pair[T] = tuple[T, T]
```

Bounds constrain what `T` may be: `def total[T: (int, float)](xs: list[T]) -> T.`

Protocols

Structural typing — shape is enough, no inheritance required. This is usually the right tool at a library boundary, because callers need not import your base class.

```
from typing import Protocol

class Writer(Protocol):
    def write(self, data: str) -> int: ...

def emit(target: Writer, text: str) -> None:
    target.write(text)

emit(open("f.txt", "w"), "hi")      # files match structurally
emit(io.StringIO(), "hi")          # so does StringIO
```

`@runtime_checkable` allows `isinstance`, but it only checks that the method *names* exist — not signatures.

TypedDict

For dict-shaped data (JSON, config) where you want key-level checking:

```
from typing import TypedDict, NotRequired

class User(TypedDict):
    id: int
    name: str
    email: NotRequired[str]          # [3.11]

def send(user: User) -> None:
    print(user["name"])              # checker verifies the key exists
```

⚠ `TypedDict` is checked statically only. Data arriving from outside still needs real validation (Chapter 33).

Overloads

When the return type depends on the arguments:

```
from typing import overload, Literal

@overload
def load(path: str, *, as_bytes: Literal[True]) -> bytes: ...
@overload
def load(path: str, *, as_bytes: Literal[False] = False) -> str: ...
def load(path: str, *, as_bytes: bool = False) -> str | bytes:
    return Path(path).read_bytes() if as_bytes else Path(path).read_text()
```

Callers now get `bytes` or `str` precisely, instead of a union they must narrow.

Other tools worth adopting

- `Self` [3.11] for fluent APIs and alternative constructors — annotating `-> "QueryBuilder"` breaks every subclass.
- `Final` for constants, `ClassVar` for class attributes.
- `NewType("UserId", int)` to stop yourself passing an order ID where a user ID belongs.
- `Literal["read", "write"]` instead of a bare `str` for mode flags.
- `assert_never` for exhaustiveness — add an enum member and the checker finds every missing branch.

Volume IV, Part II covers variance, `ParamSpec`, narrowing with `TypeIs`, and deferred annotations.

Exercises. (1) Make a generic `Repository[T]` with typed `get/save`. (2) Replace an ABC with a Protocol and note what the caller no longer needs to import. (3) Type a function whose return type depends on a boolean flag, using overloads.

Chapter 26 — Metaclasses, Descriptors, `__slots__`, ABCs

These four are the machinery behind features you already use. You need them rarely; when you need them, nothing else will do.

Descriptors

A class attribute defining `__get__`/`__set__` controls attribute access on every instance. This is what `property`, methods, `classmethod`, and `__slots__` are made of.

```
class Positive:
    def __set_name__(self, owner, name):
        self._name = f"_{name}"
    def __get__(self, obj, owner=None):
        return self if obj is None else getattr(obj, self._name)
    def __set__(self, obj, value):
        if value <= 0:
            raise ValueError(f"{self._name[1:]} must be positive")
        setattr(obj, self._name, value)

class Product:
    price = Positive()
    quantity = Positive()
```

Use a descriptor when the *same* validation applies to many attributes across many classes; use `@property` for one-off cases.

⚠ Never store per-instance state on the descriptor itself — it is shared by every instance.

`__slots__`

```
class Point:
    __slots__ = ("x", "y")
```

Replaces the per-instance `__dict__` with fixed offsets: much smaller instances, slightly faster access, and typos become `AttributeError`. Costs: no arbitrary attributes, no weak references unless you add `"__weakref__"`, and the saving disappears if any base class lacks slots. Worth it above roughly 100,000 instances; irrelevant below that. `@dataclass(slots=True)` is the easy path.

ABCs

```

from abc import ABC, abstractmethod

class Storage(ABC):
    @abstractmethod
    def save(self, key: str, data: bytes) -> None: ...
    @abstractmethod
    def load(self, key: str) -> bytes: ...

    def save_text(self, key: str, text: str) -> None:  # shared default
        self.save(key, text.encode())

```

Instantiating a subclass that has not implemented every abstract method raises `TypeError` at construction — an early, clear failure. Use an ABC when you want shared default implementations and a nominal contract; use a Protocol when you only want the shape and do not want callers to inherit.

Metaclasses

A metaclass customises class *creation*. Nearly every historical use is now better served by lighter hooks:

```

class Plugin:
    registry: dict[str, type] = {}
    def __init_subclass__(cls, /, name: str, **kwargs):
        super().__init_subclass__(**kwargs)
        Plugin.registry[name] = cls

class CsvPlugin(Plugin, name="csv"): ...

```

`__init_subclass__` for registration and validation, `__set_name__` for descriptor naming. Reach for a real metaclass only to control the class namespace during body execution or to intercept instantiation. Metaclasses conflict under multiple inheritance and confuse type checkers.

Volume IV, Part I is the full mechanism: attribute lookup order, the MRO, `type.__call__`, and the complete dunder surface.

Exercises. (1) Implement `property` from scratch using a descriptor. (2) Convert a validated class to use one reusable descriptor for five fields. (3) Build a plugin registry with `__init_subclass__` and again with a metaclass; compare.

Chapter 27 — Concurrency: Threading, Multiprocessing, the GIL

The decision

Workload	Use
Waiting on network/disk, many at once	<code>asyncio</code> (Chapter 28)
Waiting, but the library is blocking	<code>ThreadPoolExecutor</code>

Workload	Use
Pure-Python CPU work	<code>ProcessPoolExecutor</code>
NumPy/pandas/compiled CPU work	threads (those libraries release the GIL)
Isolation from crashes or untrusted code	processes

The GIL

Only one thread executes Python bytecode at a time. The lock is released during I/O and inside C extensions that opt out, which is why threads help I/O-bound work and not CPU-bound Python.

⚠ **The GIL does not make your code thread-safe.** `counter += 1` is three bytecodes and can be interrupted between any of them. Any check-then-act sequence is a race.

```
import threading
lock = threading.Lock()
with lock:
    counter += 1
```

[3.13/3.14] a free-threaded build exists in which the GIL is disabled; it is now officially supported but opt-in and a separate binary. Volume IV, Part III covers it and what changes.

concurrent.futures

The right default API for both threads and processes:

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor,
as_completed

with ThreadPoolExecutor(max_workers=16) as pool:
    futures = {pool.submit(fetch, url): url for url in urls}
    for fut in as_completed(futures):
        try:
            data = fut.result()           # exceptions surface HERE, only here
        except Exception:
            log.exception("failed: %s", futures[fut])
```

⚠ If you never call `.result()`, worker exceptions vanish silently.

Multiprocessing

Processes give you real parallelism and real isolation, at the cost of **serialisation**: every argument and result is pickled and piped.

```
if __name__ == "__main__":
    # required on Windows and with 'spawn'
    with ProcessPoolExecutor() as pool:
```

```
results = list(pool.map(score, documents, chunksize=100))
```

Practical rules: pass file paths or small values, not large data structures; use `chunksize` for many small tasks; share large arrays via `multiprocessing.shared_memory` rather than copying; and prefer the `spawn` start method — `fork` is unsafe in a process that has threads, which today means almost every process.

Thread-safe patterns

- Use `queue.Queue` to hand work between threads; it is thread-safe and gives you backpressure with `maxsize`.
- Prefer immutable messages over shared mutable state.
- Never share a database connection or an HTTP session across threads unless the library documents it as safe.
- Set timeouts on everything. A hung worker without a timeout is an outage.

Exercises. (1) Fetch 100 URLs serially, with threads, and with `asyncio`; compare. (2) Show a lost-update race and fix it with a lock. (3) Measure the pickling overhead of passing a 100 MB array to a process pool.

Chapter 28 — `asyncio`

The model

One thread, one event loop, many tasks. Each task runs until it hits an `await` on something that is not ready, then yields control. This gets you tens of thousands of concurrent connections on one core — as long as **nothing blocks**.

```
import asyncio, httpx

async def fetch(client, url):
    resp = await client.get(url)
    return resp.text

async def main(urls):
    async with httpx.AsyncClient(timeout=10) as client:
        async with asyncio.TaskGroup() as tg:
            # [3.11]
            tasks = [tg.create_task(fetch(client, u)) for u in urls]
        return [t.result() for t in tasks]

asyncio.run(main(urls))
```

- `async def` defines a coroutine; calling it returns a coroutine object that does nothing until awaited or scheduled.
- `await` suspends the current task and lets others run.
- `asyncio.run(main())` starts the loop once, at the top of your program.

TaskGroup over gather

`TaskGroup` [3.11] is structured concurrency: nothing outlives the block, a failure cancels the siblings, and errors arrive as an `ExceptionGroup` you handle with `except*`. `asyncio.gather` either leaks running siblings on failure or returns exceptions mixed into a results list. Use `TaskGroup`.

Async context managers and iterators

```
async with session.get(url) as resp:      # __aenter__ / __aexit__
    async for chunk in resp.aiter_bytes(): # __aiter__ / __anext__
        await sink.write(chunk)
```

The rules that matter

⚠ **One blocking call freezes every task on the loop.** `time.sleep`, `requests.get`, a synchronous database driver, a large `json.loads`, or heavy CPU work all stall everything. Fixes: use the async library (`httpx`, `asyncpg`, `aiofiles`), or push it off-loop:

```
result = await asyncio.to_thread(blocking_call, arg)    # blocking I/O
result = await loop.run_in_executor(process_pool, cpu_heavy, arg) # CPU
```

⚠ **Fire-and-forget tasks get garbage collected.** `asyncio.create_task(f())` without keeping a reference can vanish mid-flight. Use a `TaskGroup`, or hold the task in a set until done.

⚠ **Cancellation is an exception.** `asyncio.CancelledError` is raised inside your coroutine at its next `await`. Never swallow it — catch, clean up, re-raise. Cleanup that awaits for a long time stalls shutdown.

Other essentials: `async with asyncio.timeout(5)` [3.11] for deadlines; `asyncio.Queue(maxsize=...)` for bounded producer/consumer pipelines; `asyncio.Semaphore(20)` to cap concurrency against an API; and `asyncio.run(main(), debug=True)` in development to catch un-awaited coroutines and slow callbacks.

Volume IV, Parts III and XIV go into the loop internals, cancellation semantics, backpressure and custom loops.

Exercises. (1) Write a crawler with a concurrency cap, per-request timeout, and clean Ctrl-C shutdown. (2) Deliberately put `time.sleep(5)` in a task and observe every other task freeze; fix it. (3) Convert a `gather`-based function to `TaskGroup` and handle partial failure with `except*`.

Chapter 29 — Performance

The order of operations

1. **Measure.** Never guess. The bottleneck is somewhere you did not expect — usually logging, serialisation, an N+1 query, or a DNS lookup.
2. **Fix the algorithm.** Big-O beats every micro-optimisation.
3. **Cache.**
4. **Reduce work:** batch, stream, avoid intermediates.
5. **Only then** reach for compiled code.

Measuring

```
import timeit
timeit.timeit("f(x)", globals=globals(), number=10_000)
```

```
import cProfile, pstats
with cProfile.Profile() as pr:
    main()
pstats.Stats(pr).sort_stats("cumulative").print_stats(25)
```

Read **totttime** (time in the function itself) to find hot leaves; **cumtime** (including callees) to find expensive subtrees. For production, use a sampling profiler that needs no code changes: **py-spy top --pid <pid>**, **py-spy dump --pid <pid>** on a hung process. **memray** for memory, **scalene** to see how much time is Python versus native.

Algorithmic wins

The biggest speed-ups in real code are usually these:

```
if x in big_list:      # O(n)
if x in big_set:      # O(1)    - convert once, query many times

for user in users:    # N+1 queries
    orders = db.query(user.id)
orders_by_user = db.query_all(user_ids)    # one query, group in memory

sorted(items)[:10]    # O(n log n)
heapq.nlargest(10, items)    # O(n log k)
```

Also: **deque** instead of **list.pop(0)**; **str.join** instead of **+=** in a loop; **bisect** for sorted lookup; precompiled regexes; and doing work once outside a loop instead of every iteration.

Caching

```
from functools import cache, lru_cache, cached_property

@lru_cache(maxsize=4096)
def geocode(address: str) -> tuple[float, float]: ...
```

Rules: cache pure functions only; arguments must be hashable; bound the size unless the key space is small; and remember **@cache** on a method pins every instance in memory. For anything shared across processes or restarts, use Redis or a disk cache with explicit invalidation. Cache invalidation is a design problem, not a decorator problem — decide up front how entries expire.

I/O and databases

For most applications, Python is not the bottleneck at all. Batch inserts instead of row-by-row; use `executemany`; add the missing index; reuse HTTP connections (`httpx.Client`, not a new client per call); compress payloads; page large result sets. A 10× improvement usually lives here, not in your loops.

When to compile

If the profile shows a genuine pure-Python hot loop: vectorise with NumPy/Polars first, then consider Numba (`@njit` for numeric loops), Cython, mypyc, or Rust via PyO3. Volume IV, Part IV covers the tradeoffs.

Exercises. (1) Profile a slow script and make one 10× improvement without compiling anything. (2) Find and fix an N+1 query. (3) Add caching to an expensive pure function and measure the hit rate. (4) Show a case where caching made things worse.

Chapter 30 — Design Patterns and SOLID in Python

Most "Gang of Four" patterns exist to work around limitations Python does not have. Functions are objects, classes are objects, and duck typing removes the need for most interface ceremony. Learn the intent; use the Pythonic form.

SOLID, translated

S — Single Responsibility. A module, class, or function should have one reason to change. In practice: if you cannot name it without "and", split it. A class that fetches data, transforms it, and writes a report is three things.

O — Open/Closed. Open for extension, closed for modification. In Python this rarely means abstract base classes; it means passing behaviour in.

```
# closed to extension: every new format edits this function
def export(data, fmt):
    if fmt == "csv": ...
    elif fmt == "json": ...

# open: register new formats without touching the core
EXPORTERS: dict[str, Callable[[Data], str]] = {}
def register(name):
    def deco(fn): EXPORTERS[name] = fn; return fn
    return deco

@register("csv")
def to_csv(data: Data) -> str: ...
```

L — Liskov Substitution. A subclass must be usable wherever the parent is. If your subclass raises `NotImplementedError` for an inherited method, narrows what inputs it accepts, or strengthens preconditions, the hierarchy is wrong — use composition.

- I — Interface Segregation.** Small, focused interfaces. A **Protocol** with one method that ten classes can satisfy beats a twelve-method ABC that three classes half-implement.
- D — Dependency Inversion.** Depend on abstractions, not concretions. In Python: **pass dependencies in as arguments** instead of importing and constructing them inside.

```
# hard to test: creates its own dependencies
class OrderService:
    def __init__(self):
        self.db = PostgresClient(os.environ["DSN"])
        self.mailer = SmtplibMailer()

# testable: dependencies injected, typed by Protocol
class OrderService:
    def __init__(self, repo: OrderRepository, mailer: Mailer):
        self.repo, self.mailer = repo, mailer
```

The second version needs no mocking library — tests pass a dict-backed fake repository. If a test needs six `mock.patch` calls, that is the design telling you it has six hidden dependencies.

Patterns, the Pythonic forms

Pattern	Pythonic form
Strategy	pass a function, or a Protocol-typed object
Factory	a function, or a <code>@classmethod</code> alternative constructor
Singleton	a module — modules are already single instances
Observer	a list of callbacks, or an event bus
Decorator	<code>@decorator</code>
Iterator	generators
Template Method	a function taking hook callables, or an ABC with defaults
Command	a callable or a <code>functools.partial</code>
Adapter	a thin wrapper class or function
Builder	keyword arguments, or a frozen dataclass plus <code>replace()</code>
Dependency Injection	function parameters and constructor arguments
Repository	a Protocol + a real implementation + an in-memory fake
Null Object	<code>None</code> with an early return, or a no-op implementation

```
# Strategy — no classes required
def process(items, key=lambda x: x.name, transform=str.upper): ...
```

```
# Observer
class EventBus:
    def __init__(self): self._subs: dict[str, list[Callable]] = defaultdict(list)
    def on(self, event, fn): self._subs[event].append(fn)
    def emit(self, event, **data):
        for fn in self._subs[event]:
            fn(**data)
```

Architectural guidance

- **Layer by dependency direction.** Domain logic must not import your web framework or your ORM. Adapters at the edges, pure logic at the centre; dependencies point inward.
- **Keep I/O at the boundary.** A function that both computes and writes to a database is untestable. Compute in a pure function; do I/O in a thin caller.
- **Prefer composition over inheritance.** Inheritance couples you to the parent's internals forever.
- **Make illegal states unrepresentable:** enums instead of strings, frozen dataclasses, separate types instead of a `mode` flag.
- **Errors are part of the design.** Decide what callers can handle and give them exception types that map onto those decisions (Volume II, Chapter 18).

⚠ The most common over-engineering in Python is inventing an abstraction for one implementation. Write the concrete thing first; extract the interface when the second implementation actually arrives.

Exercises. (1) Refactor a class that constructs its own dependencies to accept them, and delete the mocks from its tests. (2) Replace an `if fmt == ...` chain with a registry. (3) Find a subclass in your code that violates Liskov and re-model it with composition. (4) Take one function that computes *and* writes, and split it.

Chapter 31 — Packaging and Publishing

Building

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "myapp"
version = "1.2.0"
description = "..."
readme = "README.md"
requires-python = ">=3.11"
license = "MIT"
dependencies = ["httpx>=0.27"]
classifiers = ["Programming Language :: Python :: 3 :: Only", "Typing :: Typed"]

[project.scripts]
myapp = "myapp.cli:main"

[project.urls]
```

```
Homepage = "https://example.com"
Changelog = "https://example.com/changelog"
```

```
python -m pip install build twine
python -m build                # produces dist/*.whl and dist/*.tar.gz
twine check dist/*
twine upload --repository testpypi dist/*    # rehearse on TestPyPI first
twine upload dist/*
```

Ship a `py.typed` marker file inside your package if it is annotated, or consumers get `Any` for everything.

Trusted publishing (OIDC from CI) is now the recommended way to upload: PyPI verifies a short-lived token tied to your repository and workflow, so there is no long-lived API token to leak.

Semantic versioning

MAJOR.MINOR.PATCH:

- **PATCH** — bug fixes, no API change.
- **MINOR** — new functionality, backwards compatible.
- **MAJOR** — breaking changes.

What counts as breaking is broader than people expect: removing or renaming anything public, changing a parameter's name (callers may use keywords), narrowing accepted input, widening returned types, changing an exception type, or changing default behaviour. Raising a minimum Python version is breaking. Tightening a dependency constraint can be.

Deprecate before removing:

```
import warnings

def old_api(x):
    warnings.warn(
        "old_api() is deprecated since 1.2 and will be removed in 2.0; use new_api()",
        DeprecationWarning,
        stacklevel=2,           # points at the CALLER, not this line
    )
    return new_api(x)
```

Announce in a minor release, warn for at least one full cycle, remove in a major. Keep a `CHANGELOG.md` with a section per release. Run your own tests with `-W error::DeprecationWarning` so you see your dependencies' warnings early.

Constraints, again

Libraries: lower bounds always, upper bounds only with evidence. Applications: pin everything with a lock file (`uv.lock`, `poetry.lock`, or `pip-compile` output with hashes) and install with `--require-hashes` in production.

Volume IV, Part IX covers wheels, ABI tags, `manylinux`, entry points, and supply-chain hardening.

Exercises. (1) Publish a small package to TestPyPI and install it in a clean venv. (2) Add a console script entry point and verify it works after install. (3) Write a deprecation with a warning, a changelog entry, and a test asserting the warning is raised.

Chapter 32 — CI/CD, Linting, Formatting, Pre-commit

The tool set

- **ruff** — linter and formatter in one, fast enough to run on every keystroke. It replaces flake8, isort, pyupgrade, pydocstyle, autoflake, bandit rules, and (with `ruff format`) black.
- **black** — the original opinionated formatter. Still widely used; `ruff format` is compatible with its style.
- **mypy** or **pyright** — type checking.
- **pytest** — tests.
- **pip-audit** — dependency vulnerabilities.

Formatting is not a matter of taste once a team exists. Pick one formatter, run it automatically, and never discuss it again — the value is entirely in ending the discussion and making diffs meaningful.

```
[tool.ruff]
line-length = 100
target-version = "py311"

[tool.ruff.lint]
select = [
    "E", "F",      # pycodestyle, pyflakes
    "I",           # import sorting
    "UP",         # pyupgrade – modernise syntax
    "B",          # bugbear – real bug patterns (mutable defaults, etc.)
    "SIM",        # simplifications
    "C4",         # comprehension improvements
    "RUF",        # ruff-specific
    "S",          # bandit security rules
    "ASYNC",      # async pitfalls
]
ignore = ["E501"]      # line length is the formatter's job

[tool.ruff.lint.per-file-ignores]
"tests/*" = ["S101"]  # assert is fine in tests
```

Start with `E`, `F`, `I`, `UP`, `B` on an existing codebase; add rule families as you clear them. `ruff check --fix` fixes most violations automatically.

Pre-commit hooks

Run the fast checks before a commit is created, so broken code never reaches CI:

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.5.7
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-added-large-files
      - id: check-merge-conflict
      - id: detect-private-key
  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.11.1
    hooks:
      - id: mypy
        additional_dependencies: [types-requests]
```

```
pip install pre-commit
pre-commit install           # once per clone
pre-commit run --all-files  # first run, to fix the backlog
```

Keep hooks **fast** — under a few seconds. Slow hooks get bypassed with `--no-verify`, and then they protect nothing. Put the slow things (full test suite, integration tests) in CI.

Continuous integration

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ${ matrix.os }
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        python: ["3.11", "3.12", "3.13"]
    steps:
      - uses: actions/checkout@v4
```

```

- uses: actions/setup-python@v5
  with:
    python-version: ${{ matrix.python }}
- run: pip install -e ".[dev]"
- run: ruff check .
- run: ruff format --check .
- run: mypy src
- run: pytest --cov=src --cov-report=term-missing
- run: pip-audit

```

A sound pipeline, in order: **lint** → **format check** → **type check** → **test** → **build** → **publish**. Fail fast on the cheap checks. Cache the dependency install. Run the full OS/version matrix only on the main branch and on pull requests, not on every push to a scratch branch.

Deployment

- Build once, deploy the same artifact everywhere. Never rebuild per environment.
- Pin dependencies with hashes for applications.
- Tag releases; automate publishing from the tag, using trusted publishing rather than stored tokens.
- Health checks, structured logs to stdout, and a rollback path you have actually tested.
- Keep secrets in the platform's secret store, never in the image or the repo.

Exercises. (1) Add ruff, mypy, and pre-commit to an existing project and clear the backlog. (2) Write a CI workflow that runs the matrix above. (3) Deliberately push a lint failure and confirm CI blocks the merge. (4) Add `pip-audit` and resolve whatever it finds.

Chapter 33 — Dataclasses, Pydantic, Serialization

dataclasses

```

from dataclasses import dataclass, field, replace, asdict

@dataclass(frozen=True, slots=True, kw_only=True)
class Order:
    id: str
    customer: str
    items: list[str] = field(default_factory=list)
    total_cents: int = 0
    notes: str = field(default="", repr=False)

    def __post_init__(self):
        if self.total_cents < 0:
            raise ValueError("total cannot be negative")

```

You get `__init__`, `__repr__`, and `__eq__` for free. Options worth using by default:

- `frozen=True` — immutable, hashable, safe to share. Update with `replace(order, total_cents=500)`.

- `slots=True` [3.10] — smaller and faster.
- `kw_only=True` [3.10] — no positional-argument ambiguity, and it removes the "non-default follows default" inheritance headache.
- `field(default_factory=list)` — mandatory for mutable defaults.
- `field(compare=False, repr=False)` — exclude noisy or derived fields.

`__post_init__` is where light validation goes. On a frozen class use `object.__setattr__(self, "x", v)` to assign.

Pydantic — for data that came from outside

Dataclasses do not validate. Anything crossing a trust boundary — HTTP bodies, queue messages, config files, third-party API responses — needs real validation:

```
from pydantic import BaseModel, Field, field_validator, ConfigDict

class WebhookEvent(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)

    event_id: str = Field(min_length=1, max_length=64)
    amount_cents: int = Field(gt=0)
    currency: str = Field(pattern=r"^[A-Z]{3}$")
    occurred_at: datetime

    @field_validator("currency")
    @classmethod
    def supported(cls, v: str) -> str:
        if v not in SUPPORTED:
            raise ValueError(f"unsupported currency {v}")
        return v

event = WebhookEvent.model_validate(request.json()) # raises ValidationError
payload = event.model_dump(mode="json")
```

`extra="forbid"` turns a typo in a config file into an immediate error instead of a silently ignored key — worth adopting everywhere.

The rule: **dataclasses for data you created, Pydantic for data you received**. Validating internal objects on every construction is pure cost.

`pydantic-settings` reads and validates configuration from environment variables with the same model syntax — the clean answer to config sprawl.

Serialization

Format	Good for	Watch out for
JSON	interoperability, config, APIs	no dates/bytes/decimals; <code>NaN</code> is emitted by default but invalid JSON

Format	Good for	Watch out for
orjson / msgspec	fast JSON, native dataclass & datetime support	third-party
CSV	tabular exchange, spreadsheets	everything is a string; needs <code>newline=""</code>
YAML/TOML	human-edited config	⚠ <code>yaml.load</code> executes code — use <code>safe_load</code> ; TOML via <code>tomllib</code> [3.11]
MessagePack / CBOR	compact binary	not human-readable
Protobuf / Avro	cross-language, schema evolution	build step, schema registry
Parquet / Arrow	analytics, columnar data	not for records-at-a-time
pickle	internal, trusted, same-version	⚠ arbitrary code execution on load

⚠ **Never unpickle untrusted data.** It is not a data format; it is a program. It also breaks across Python versions and across your own class renames, so it is unfit for storage or wire use. Same warning for `yaml.load` without `SafeLoader`.

Custom JSON encoding:

```
class Encoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, datetime):
            return obj.isoformat()
        if isinstance(obj, Decimal):
            return str(obj)
        if is_dataclass(obj):
            return asdict(obj)
        return super().default(obj)
```

Design your wire format for change: version your payloads, ignore unknown fields on the read side where you must tolerate newer producers (but forbid them for config), and never make a field's meaning depend on another field's value.

Volume IV, Part VIII covers `enum`, `attrs`, `collections.abc` and the modelling tradeoffs in full; Part XII covers deserialization security.

Exercises. (1) Convert three plain classes to frozen dataclasses with slots. (2) Model an external API response with Pydantic, rejecting unknown fields, and write tests for three malformed payloads. (3) Round-trip a nested structure containing dates and decimals through JSON and back with no loss. (4) Load a config from environment variables with `pydantic-settings`, with required fields and typed defaults.

Where you are now

You can design, build, test, ship, and operate Python that other people rely on. Volume IV is about the layer underneath — what the interpreter actually does, and what becomes possible once you know.

Volume IV — Expert

What the interpreter actually does, and what becomes possible once you know.

This volume changes register. Volumes I–III told you what to do; this one tells you what is true, and lets you work out what to do from there. Each part answers three questions:

1. **What is the mechanism?** — the actual rule the interpreter follows, not an analogy.
2. **Why does it exist?** — the constraint that forced the design.
3. **Where does it bite?** — the failure mode you will hit in production.

Read the parts in any order. Parts I, IV and V are the interpreter itself; Parts II, VII, VIII and IX are the craft of building libraries; Parts III, X, XI, XII and XIV–XVI are the disciplines that keep systems alive.

Environment. Keep three interpreters available while reading: a current CPython, a free-threaded build, and a debug build (`--with-pydebug`) for the internals chapters. `uv python install 3.14 3.14t` gets you the first two in one command.

A note on version facts. The interpreter is moving quickly — free-threading, the JIT, and annotation semantics have all changed in consecutive releases. Where this volume states version-specific behaviour, verify against the release notes for the version you actually run before depending on it.

Part I — The Object Model

Everything in Python is an object; every object has a type; every type is itself an object whose type is `type`. The consequences of those three sentences fill this part.

1.1 What an object actually is

At the C level a Python object is a struct beginning with a refcount and a type pointer. The type pointer determines behaviour: when you write `a + b`, the interpreter does not look at `a` for an `__add__` attribute the way `getattr` would. It looks at `type(a)`'s C-level slot table.

This is the single most important asymmetry in the language:

```
class Broken:
    def __init__(self):
        self.__len__ = lambda: 42

b = Broken()
b.__len__()    # 42
len(b)         # TypeError: object of type 'Broken' has no len()
```

Implicit special-method lookup skips the instance. `len()`, `+`, `with`, `in`, iteration, and every other protocol resolve the dunder on the `type`, bypassing `__getattr__` on the instance entirely. This is done for speed

(the type's slots are cached in a C struct) and for sanity (an instance attribute named `__init__` should not break construction).

Consequence: to make instances of a class behave differently per-instance, you must vary the *class*, not the instance. Per-instance dunders require per-instance classes — which is exactly what `mock.MagicMock` and `unittest.mock` do internally.

1.2 Attribute lookup, in exact order

When you evaluate `obj.x`, `object.__getattr__` runs roughly this algorithm:

1. Walk `type(obj).__mro__` looking for `x`. Call the result *meta-attr*.
2. If meta-attr is found **and** it defines both `__get__` and `__set__` (or `__delete__`), it is a **data descriptor**. Call `meta_attr.__get__(obj, type(obj))` and return. *Data descriptors win over instance dicts.*
3. Otherwise look in `obj.__dict__`. If found, return it raw.
4. Otherwise, if meta-attr was found and defines only `__get__`, it is a **non-data descriptor** (e.g. a plain function). Call `__get__` and return.
5. Otherwise, if meta-attr was found, return it as-is.
6. Otherwise raise `AttributeError` — which the interpreter catches and converts into a call to `type(obj).__getattr__(obj, 'x')` if that exists.

Memorise the precedence: **data descriptor > instance dict > non-data descriptor > class attribute > `__getattr__`**.

```
class Data:
    def __get__(self, obj, owner): return "descriptor"
    def __set__(self, obj, value): raise AttributeError("read-only")

class NonData:
    def __get__(self, obj, owner): return "descriptor"

class C:
    d = Data()
    n = NonData()

c = C()
c.__dict__['d'] = "instance"  # bypass __set__ deliberately
c.__dict__['n'] = "instance"
c.d  # 'descriptor' — data descriptor wins
c.n  # 'instance' — instance dict wins
```

`__getattr__` vs `__getattribute__`

- `__getattribute__` intercepts **every** attribute access. Override it and you own the whole algorithm above — including the risk of infinite recursion if you write `self.x` inside it. Always delegate with `object.__getattribute__(self, name)` or `super().__getattribute__(name)`.
- `__getattr__` is a **fallback**, called only after normal lookup raised `AttributeError`. It is cheap, safe, and what you want 95% of the time (lazy attributes, proxies, dynamic APIs).

⚠ A `__getattr__` that never raises `AttributeError` breaks `hasattr`, `copy`, `pickle`, and every duck-typing check in the stdlib, because those probe for `__deepcopy__`, `__getstate__`, `__reduce_ex__` and expect failure. Always re-raise for dunder:

```
def __getattr__(self, name):
    if name.startswith("__") and name.endswith("__"):
        raise AttributeError(name)
    return self._backing[name]
```

[3.7+] Modules support `__getattr__` too (PEP 562), which is how you deprecate a module-level name or make an import lazy:

```
# mypkg/__init__.py
def __getattr__(name):
    if name == "HeavyThing":
        from .heavy import HeavyThing
        return HeavyThing
    raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
```

1.3 Descriptors: the mechanism behind everything

A descriptor is any object defining `__get__`, `__set__`, or `__delete__`, stored as a *class* attribute. Descriptors are not a niche feature — they are the substrate of the object system:

Feature	Implemented as
Methods	functions are non-data descriptors; <code>__get__</code> returns a bound method
<code>property</code>	data descriptor
<code>classmethod</code> / <code>staticmethod</code>	descriptors returning differently-bound callables
<code>__slots__</code> members	<code>member_descriptor</code> , a data descriptor over a C struct offset
<code>functools.cached_property</code>	non-data descriptor that writes into the instance dict

The bound-method trick in one line:

```
class Function:
    def __get__(self, obj, owner=None):
        return self if obj is None else MethodType(self, obj)
```

That is why `C.f` gives you a plain function but `c.f` gives you a bound method: same object, different `obj` argument.

`__set_name__` [3.6+]

Descriptors used to need their attribute name passed in redundantly. Now the metaclass machinery calls `__set_name__(owner, name)` on every class-body attribute at class creation:

```
class Typed:
    def __init__(self, expected):
        self.expected = expected
    def __set_name__(self, owner, name):
        self.private = f"_{name}"
        self.public = name
    def __get__(self, obj, owner=None):
        if obj is None:
            return self
        try:
            return getattr(obj, self.private)
        except AttributeError:
            raise AttributeError(f"{self.public!r} not set") from None
    def __set__(self, obj, value):
        if not isinstance(value, self.expected):
            raise TypeError(f"{self.public} must be {self.expected.__name__}")
        setattr(obj, self.private, value)

class Order:
    quantity = Typed(int)
    customer = Typed(str)
```

⚠ **Never store per-instance state on the descriptor itself.** The descriptor is shared by every instance of the class. `self.value = value` inside `__set__` is the classic bug: every object sees the last write. Store in the instance (`obj.__dict__`, or a `WeakKeyDictionary` if instances are unhashable/slotted).

`__get__(None, owner)`

When accessed on the class rather than an instance, `obj` is `None`. Returning `self` (as above) is the convention — it lets `Order.quantity` give you the descriptor for introspection instead of exploding.

🔗 Implement `property` from scratch in ten lines, then implement a `validated_property` that caches. You will understand descriptors permanently.

1.4 `__slots__` and object layout

By default, instances carry a `__dict__` (a hash table) and a `__weakref__` slot. `__slots__` replaces the dict with fixed C-struct offsets exposed as data descriptors.

```
class Point:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x, self.y = x, y
```

What you gain: substantially smaller instances (no dict, no dict pointer indirection) and slightly faster attribute access. On a million-object workload this is often the difference between fitting in RAM and not.

What you lose and must plan for:

- No arbitrary attributes. Typos become `AttributeError` — often a feature.
- No `__weakref__` unless you list it: `__slots__ = ("x", "y", "__weakref__")`.
- Inheritance is subtle: if *any* base lacks `__slots__`, instances get a `__dict__` anyway and the saving evaporates. Repeating a slot name in a subclass wastes space and shadows the parent's descriptor.
- Multiple inheritance from two classes with non-empty `__slots__` raises `TypeError` (conflicting instance lay-out).
- Default pickling changes: without `__dict__`, `__reduce_ex__` uses the `(dict, slots_dict)` two-tuple state form. Custom `__getstate__` must account for it. [3.11+] `object.__getstate__` handles slots correctly by default, which removed a long-standing class of bugs.

[3.10+] `@dataclass(slots=True)` generates a *new class* with slots — meaning `super()` zero-arg form and any closure referencing the original class can misbehave, and decorators that captured the old class see a stale object.

1.5 The MRO and cooperative `super()`

Python linearizes multiple inheritance with **C3**, which guarantees: a class precedes its parents, parents appear in declaration order, and the result is monotonic (a subclass's MRO is consistent with every ancestor's). If no such order exists, class creation raises `TypeError`.

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
D.__mro__    # D, B, C, A, object
```

`super()` does not mean "my parent." It means "the next class after *this* one in the MRO of `type(self)`" — which at runtime may be a sibling the author never heard of. That is the point: it enables cooperative multiple inheritance.

```
class Base:
    def process(self, data): return data

class Trim(Base):
    def process(self, data): return super().process(data.strip())

class Upper(Base):
    def process(self, data): return super().process(data.upper())

class Pipeline(Trim, Upper, Base): pass
Pipeline().process(" hi ")    # 'HI'
```

Rules for cooperative classes: every participant must accept and forward `**kwargs`, every participant must call `super()`, and there must be a common base that stops the chain (typically `object`, which is why `object.__init__` accepting no args matters).

The zero-argument `super()` is compiler magic: it inserts a `__class__` cell into the method's closure. It therefore only works lexically inside a class body method. In a nested function, comprehension, or a re-assigned method, use the explicit two-argument form `super(Pipeline, self)`.

1.6 Metaclasses and the class creation protocol

`class C(Base, metaclass=M, **kws)` executes roughly:

1. Resolve the metaclass: the most derived of `M` and the types of all bases (conflicts → `TypeError`).
2. `ns = M.__prepare__(name, bases, **kws)` — returns the mapping used as the class body namespace. Returning an `OrderedDict`-like or a custom mapping lets you detect duplicate definitions or record order.
3. Execute the class body with `ns` as its locals.
4. `cls = M(name, bases, ns, **kws) → M.__new__` then `M.__init__`.
5. `__set_name__` is called on every value in the namespace that defines it.
6. `__init_subclass__` is called on the *parent* class.

```
class Registry(type):
    registry = {}
    def __new__(mcls, name, bases, ns, *, key=None, **kw):
        cls = super().__new__(mcls, name, bases, ns, **kw)
        if key:
            Registry.registry[key] = cls
        return cls

class Handler(metaclass=Registry): pass
class JSONHandler(Handler, key="json"): pass
```

`type.__call__` is where instantiation lives. `C(...)` invokes `type(C).__call__(C, ...)`, which calls `C.__new__` and then, *only if the result is an instance of C*, `C.__init__`. Override the metaclass `__call__` for singletons, interning, or caching — that is the only place you can suppress `__init__` entirely.

`__new__` vs `__init__`: `__new__` is a static method that *creates*; `__init__` *configures*. You need `__new__` for immutable types (`str`, `tuple`, `int` subclasses) because there is nothing to configure after the fact:

```
class Currency(str):
    def __new__(cls, code):
        if len(code) != 3:
            raise ValueError("ISO 4217 codes are 3 letters")
        return super().__new__(cls, code.upper())
```

Prefer the lighter hooks

Metaclasses are viral (they conflict on multiple inheritance) and hostile to type checkers. Since 3.6 most uses are better served by:

- `__init_subclass__(cls, **kwargs)` — an implicit classmethod on the parent, called whenever a subclass is created. Registration, validation, and default injection all belong here.
- `__set_name__` — attribute-name binding.
- `__mro_entries__` (PEP 560) — lets a non-class object appear in a bases list and substitute real bases. This is how `Generic[T]` stopped needing a metaclass, which is why generics no longer conflict with ABCs.

```
class Plugin:
    registry = {}
    def __init_subclass__(cls, /, name, **kwargs):
        super().__init_subclass__(**kwargs)
        cls.registry[name] = cls

class Csv(Plugin, name="csv"): pass
```

Reach for a metaclass only when you must control the namespace *during* body execution or intercept instantiation itself.

1.7 The dunder surface

Equality and hashing — the contract

- `a == b` implies `hash(a) == hash(b)`. The converse is not required.
- Hash must be stable for the object's lifetime *within a dict*.
- Defining `__eq__` sets `__hash__ = None`, making instances unhashable. Restore it explicitly with `__hash__ = SomeBase.__hash__` or by defining your own.

⚠ Mutable objects with value equality are a trap. If you hash on a field and then mutate that field, the object is lost inside its own dict — `d[k]` raises `KeyError` for a key that is visibly in `d.keys()`. Either freeze the object or hash on identity.

```
@dataclass(frozen=True)    # eq + hash generated, mutation blocked
class Key:
    tenant: str
    shard: int
```

Binary operator dispatch

For `a + b` the interpreter tries `type(a).__add__(a, b)`; if it returns the `NotImplemented` singleton, it tries `type(b).__radd__(b, a)`; if that also declines, it raises `TypeError` with a good message. Special case: if `type(b)` is a *proper subclass* of `type(a)` and overrides the reflected method, `b`'s reflected method goes first.

```
class Vec:
    def __init__(self, *xs): self.xs = xs
    def __add__(self, other):
        if isinstance(other, Vec):
            return Vec(*(a + b for a, b in zip(self.xs, other.xs)))
        return NotImplemented # never raise TypeError here
    __radd__ = __add__
```

⚠ Returning `NotImplemented` from a `__bool__`-adjacent context or *raising* instead of returning it both break interoperability with third-party numeric types. Return the singleton; do not raise, and never return the string `"NotImplemented"`.

In-place operators (`__iadd__`) should mutate and `return self`; if absent, `a += b` silently falls back to `a = a + b`, which is why `+=` on a tuple attribute of an immutable container produces the famous "TypeError and the mutation happened anyway" puzzle with lists inside tuples.

The rest of the protocol, by group

- **Containers:** `__len__`, `__getitem__`, `__setitem__`, `__delitem__`, `__contains__`, `__iter__`, `__reversed__`, `__missing__` (dict subclasses only). Defining only `__getitem__` still gives you legacy iteration via integer indices — usually not what you want; define `__iter__`.
- **Numeric coercion:** `__index__` (lossless integer conversion — required for slicing and `hex()`), `__int__`, `__float__`, `__complex__`, `__round__`, `__trunc__`, `__floor__`, `__ceil__`.
- **Strings:** `__repr__` (unambiguous, ideally eval-able, for developers), `__str__` (readable, defaults to `__repr__`), `__format__` (drives f-strings and `format()`; parse the spec yourself or delegate).
- **Context:** `__enter__` / `__exit__`; returning a truthy value from `__exit__` suppresses the exception — a rare and dangerous power.
- **Callable:** `__call__`.
- **Class-check hooks:** `__instancecheck__`, `__subclasscheck__` (on the metaclass), `__subclasshook__` (on ABCs) — how `isinstance(x, Iterable)` works structurally.
- **Copying and pickling:** `__copy__`, `__deepcopy__`, `__reduce__`, `__reduce_ex__`, `__getstate__`, `__setstate__`, `__getnewargs__` / `__getnewargs_ex__`.
- **Lifecycle:** `__del__` — not a destructor. It runs at an unpredictable time, may never run at interpreter shutdown, and exceptions inside it are swallowed to stderr. Use it only as a last-resort warning ("you forgot to close me"), never as the primary cleanup path. Prefer context managers and `weakref.finalize`.

`__init_subclass__` + `__set_name__` + `__mro_entries__` recap

These three hooks are the "modern metaclass." A library that uses them instead of a custom metaclass composes cleanly with ABCs, Protocols, dataclasses, and Pydantic. A library with a bespoke metaclass fights all four.

1.8 Weak references and lifetime

`weakref.ref(obj)` gives you a callable returning the object or `None` once collected. It is the standard answer to "I need a cache/registry/back-pointer that must not keep the object alive."

```
import weakref

class Node:
    __slots__ = ("value", "children", "__weakref__", "_parent")
    def __init__(self, value):
        self.value, self.children, self._parent = value, [], None
    @property
    def parent(self):
        return self._parent() if self._parent is not None else None
    @parent.setter
    def parent(self, node):
        self._parent = weakref.ref(node) if node is not None else None
```

Key tools: `WeakValueDictionary` (cache that evicts when the value dies), `WeakKeyDictionary` (per-object side data without touching the object), `WeakSet`, and `weakref.finalize(obj, callback, *args)` — the correct replacement for `__del__` because it runs deterministically at collection and holds no reference to `obj` itself.

Not everything is weak-referenceable: `int`, `str`, `tuple`, and slotted classes without `__weakref__` are not. Test early; the failure is a `TypeError` at registration time.

1.9 Exercises for Part I

1. Write `LazyProperty` that computes once, stores in `obj.__dict__`, and works on slotted classes (hint: you cannot — explain precisely why, then write the `WeakKeyDictionary` variant).
2. Build a class whose instances compare equal but hash differently. Insert both into a dict. Explain the observed behaviour in terms of the hash table's probe sequence.
3. Implement a `Final` metaclass forbidding subclassing, then implement the same thing with `__init_subclass__`. Compare how each behaves under multiple inheritance.
4. Given `class D(B, C)` where `B` and `C` both inherit `A` and both define `save()`, make `D().save()` run `B.save`, then `C.save`, then `A.save`, exactly once each.
5. Explain why `len(b)` failed in §1.1 by reading `dis.dis(lambda b: len(b))` and `dis.dis(lambda b: b.__len__())`.

Part II — The Type System

Python's type system is *gradual*: unannotated code is implicitly `Any` and checks are erased at runtime. That design choice explains most of its oddities. A type checker is a separate program with its own semantics; the interpreter mostly does not care what you wrote.

2.1 The mental model

Three separate things travel under the word "type":

1. **Runtime classes** — what `isinstance` sees.
2. **Static types** — what mypy/pyright reason about. These include things with no runtime class at all: `Literal[3]`, `None` as a type, `Callable[..., int]`, unions, `Never`.

3. **Annotations** — the syntax that carries (2) into the source, stored in `__annotations__` and usable for anything (Pydantic, FastAPI, attrs).

Assignability is not subclassing. `int` is assignable to `float` by special-case fiat in the spec (the "numeric tower promotion"), not because `int` subclasses `float`. `Sequence[str]` accepts a `tuple` and a `list` but not `str` in most designs' intent — yet `str` is a `Sequence[str]`, which is a permanent source of bugs the checker will happily allow.

2.2 Generics and type variables

```
from typing import TypeVar, Generic
T = TypeVar("T")

class Box(Generic[T]):
    def __init__(self, item: T) -> None:
        self._item = item
    def get(self) -> T:
        return self._item
```

[3.12, PEP 695] The same thing with the new syntax — no imports, no explicit `Generic`, and the type parameter is properly scoped:

```
class Box[T]:
    def __init__(self, item: T) -> None:
        self._item = item
    def get(self) -> T:
        return self._item

def first[T](xs: list[T]) -> T:
    return xs[0]

type Pair[T] = tuple[T, T]           # lazily-evaluated type alias
```

PEP 695 aliases are *lazy*: the right-hand side is not evaluated until accessed via `.__value__`, so forward references work without strings.

Bounds vs constraints — different tools:

```
from typing import TypeVar
TNum = TypeVar("TNum", bound=float)           # any subtype of float
TStrBytes = TypeVar("TStrBytes", str, bytes) # exactly str OR exactly bytes
```

A *bound* means "at least this"; the return type can be the narrower actual type. *Constraints* mean "one of these, solved independently per call" — the classic use is a function that must not mix `str` and `bytes`. New syntax: `def f[T: float](...)` for a bound, `def f[T: (str, bytes)](...)` for constraints.

[3.13, PEP 696] `TypeVar` defaults let generic parameters be omitted:

```
class Response[T = dict[str, object]]:
    body: T
    Response          # implicitly Response[dict[str, object]]
```

2.3 Variance — the part everyone gets wrong

Given `Dog <: Animal`, is `Container[Dog] <: Container[Animal]`?

- **Covariant** (`Container[Dog] <: Container[Animal]`): safe when the container only *produces* `T`. `Sequence`, `Iterable`, `Mapping`'s value type, function *return* types, `frozenset`.
- **Contravariant** (`Container[Animal] <: Container[Dog]`): safe when it only *consumes* `T`. Function *parameter* types, `Callable`'s arguments.
- **Invariant**: required when it both produces and consumes. `list`, `dict`, `set`, and every mutable container.

The proof that `list` must be invariant:

```
def feed_all(animals: list[Animal]) -> None:
    animals.append(Cat())          # legal for list[Animal]

dogs: list[Dog] = [Dog()]
feed_all(dogs)                    # if this were allowed...
dogs[1].bark()                    # ...we'd call bark() on a Cat
```

Old syntax declared variance on the `TypeVar` (`TypeVar("T_co", covariant=True)`, conventionally suffixed `_co` / `_contra`). [3.12+] variance is **inferred** from usage for PEP 695 classes — a genuine improvement, because it is inferred correctly and consistently.

The practical rule for API design: **accept the most general** (`Iterable`, `Mapping`, `Sequence`), **return the most specific** (`list`, `dict`). Invariance on `list` parameters is the single most common false positive people complain about, and the fix is nearly always to annotate the parameter as `Sequence` or `Iterable` instead.

2.4 Protocols: structural typing

ABCs are *nominal* — you must inherit or `register()`. Protocols are *structural* — shape is enough. For a library boundary, protocols are almost always the better tool, because your callers do not have to import your base class.

```
from typing import Protocol, runtime_checkable

class Closeable(Protocol):
    def close(self) -> None: ...

def shutdown(r: Closeable) -> None:
```

```
r.close()

shutdown(open("f"))          # files match structurally; no registration
```

Details that matter:

- Protocols can declare attributes, properties, class variables, and other protocols.
- A protocol with `__init__` or non-method members cannot be used with `issubclass()`.
- `@runtime_checkable` enables `isinstance` — but it checks **only the presence of member names**, not signatures or types. It is a duck-typing sanity check, not proof.
- Explicitly inheriting a Protocol is allowed and gives you the checker's "did I implement it" verification plus default implementations. Structural matching still works for everyone else.
- Protocols compose: `class ReadableCloseable(Readable, Closeable, Protocol): ...`. Forgetting to re-list `Protocol` silently turns it into a concrete ABC — a classic bug.

Callback protocols express what `Callable` cannot — keyword arguments, overloads, and named parameters:

```
class Handler(Protocol):
    def __call__(self, event: Event, *, retries: int = 3) -> None: ...
```

2.5 ParamSpec, Concatenate, TypeVarTuple

`ParamSpec` [PEP 612] captures an entire parameter list so decorators stop erasing signatures:

```
from typing import ParamSpec, TypeVar, Callable
from functools import wraps
P = ParamSpec("P"); R = TypeVar("R")

def retry(times: int) -> Callable[[Callable[P, R]], Callable[P, R]]:
    def deco(fn: Callable[P, R]) -> Callable[P, R]:
        @wraps(fn)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            for attempt in range(times):
                try:
                    return fn(*args, **kwargs)
                except TransientError:
                    if attempt == times - 1:
                        raise
            raise AssertionError("unreachable")
        return wrapper
    return deco
```

`P.args` and `P.kwargs` must be used together and only in that position. [3.12+]: `def deco[**P, R](fn: Callable[P, R]) -> Callable[P, R]:`

`Concatenate` adds or removes leading parameters — the type of a decorator that injects a resource:

```
def with_conn[**P, R](
    fn: Callable[Concatenate[Connection, P], R]
) -> Callable[P, R]: ...
```

`TypeVarTuple` + `Unpack` [PEP 646] gives variadic generics, mainly for shape-typed arrays:

```
class Array[*Shape]: ...
def add[*S](a: Array[*S], b: Array[*S]) -> Array[*S]: ...
```

2.6 The annotation vocabulary

Construct	Meaning / use
<code>Optional[X]</code> / <code>X None</code>	may be None. Prefer the <code> </code> form [3.10+, PEP 604]
<code>Union[A, B]</code> / <code>A B</code>	either
<code>Literal["r", "w"]</code>	exact values; drives narrowing and overloads
<code>LiteralString</code> [3.11]	built only from literals — SQL/shell injection defence
<code>Final</code> / <code>Final[int]</code>	no reassignment; enables literal-type inference
<code>ClassVar[X]</code>	class attribute, not an instance field (dataclasses enforce this at runtime)
<code>Annotated[int, Ge(0)]</code>	attach arbitrary metadata; how Pydantic/FastAPI carry validators
<code>NewType("UserId", int)</code>	zero-cost distinct type; prevents mixing ID kinds
<code>Self</code> [3.11]	the actual subclass — correct for fluent APIs and alt constructors
<code>Never</code> / <code>NoReturn</code>	function never returns; enables exhaustiveness
<code>TypeAlias</code> / <code>type X = ...</code>	explicit alias declaration
<code>TypedDict</code>	structural dict shape; <code>total=False</code> , <code>Required</code> , <code>NotRequired</code> , <code>ReadOnly</code> [3.13]
<code>NamedTuple</code>	typed tuple with fields
<code>@override</code> [3.12]	assert this really overrides a base method
<code>@deprecated</code> [3.13]	checker- and runtime-visible deprecation

`Self` is the fix for the oldest annotation bug in Python:

```
from typing import Self
class QueryBuilder:
    def where(self, cond: str) -> Self:        # not "QueryBuilder"
```

```
...
return self
```

Annotating -> `"QueryBuilder"` breaks every subclass's chaining.

`TypedDict` with `Unpack` [PEP 692] types `**kwargs` precisely:

```
class Opts(TypedDict, total=False):
    timeout: float
    retries: int
    def fetch(url: str, **opts: Unpack[Opts]) -> bytes: ...
```

2.7 Overloads, narrowing, exhaustiveness

`@overload` describes input/output relationships a single signature cannot:

```
from typing import overload, Literal

@overload
def read(path: str, *, binary: Literal[True]) -> bytes: ...
@overload
def read(path: str, *, binary: Literal[False] = False) -> str: ...
def read(path: str, *, binary: bool = False) -> str | bytes:
    mode = "rb" if binary else "r"
    with open(path, mode) as f:
        return f.read()
```

Rules: overload stubs have `...` bodies, the implementation is not itself an overload and must be compatible with all of them, and order matters — checkers pick the first match.

Narrowing is what makes unions usable. Checkers narrow on `isinstance`, `is None`, `type(x) is C`, truthiness, `in` against a `Literal` union, `assert`, and equality against literals. `TypeGuard` and `TypeIs` let you write your own narrowing function:

- `TypeGuard[X]` — one-way. If the function returns `True`, the value is `X`. The negative branch is *not* narrowed.
- `TypeIs[X]` [3.13, PEP 742] — two-way, and `X` must be consistent with the parameter type. Narrows both branches. **Prefer `TypeIs` unless you genuinely need the asymmetry** (e.g. `is_list_of_str(x: list[object]) -> TypeGuard[list[str]]`, where the negative branch tells you nothing).

```
def is_str_list(v: list[object]) -> TypeGuard[list[str]]:
    return all(isinstance(x, str) for x in v)

def is_bird(a: Animal) -> TypeIs[Bird]:
    return isinstance(a, Bird)
```

Exhaustiveness turns "I added an enum member and forgot a branch" from a production incident into a type error:

```
from typing import assert_never

def area(s: Shape) -> float:
    match s:
        case Circle(r=r):      return math.pi * r * r
        case Square(side=a):   return a * a
        case _ as unreachable: assert_never(unreachable)
```

Add a `Triangle` variant and the checker fails on `assert_never` because the residual type is no longer `Never`. This pattern plus `Literal` unions or a sealed base class is the closest Python gets to algebraic data types.

2.8 Annotations at runtime

Annotations are just objects in `__annotations__`, but *when* they are evaluated has changed three times.

- Originally: evaluated eagerly at definition time. Forward references required string quoting.
- [3.7–3.13] `from __future__ import annotations` (PEP 563): all annotations become strings. Cheap and forward-reference-friendly, but breaks any library that needs the real object — hence `typing.get_type_hints()` and its namespace-resolution headaches.
- [3.14, PEP 649/749] **Deferred evaluation by default**: annotations are compiled into a lazily-invoked function and materialised on demand. You get forward references *and* real objects. The `annotationlib` module exposes formats (`VALUE`, `FORWARDREF`, `STRING`) so tooling can ask for what it can handle.

Always read annotations through the official API, never `__annotations__` directly:

```
from typing import get_type_hints, get_origin, get_args
hints = get_type_hints(func, include_extras=True) # keeps Annotated metadata
get_origin(list[int])      # list
get_args(dict[str, int])   # (str, int)
get_origin(int | None)     # types.UnionType (or typing.Union for Optional[])
```

⚠ `get_type_hints` needs the right globals/locals to resolve names. For methods on classes defined inside functions, or under `TYPE_CHECKING` imports, resolution fails at runtime. Pattern: put runtime-needed imports outside `TYPE_CHECKING`, or pass explicit namespaces.

`if TYPE_CHECKING:` blocks import types that are needed only statically — essential for breaking import cycles and keeping startup fast. They pair with deferred annotations; without them you need string annotations.

2.9 Tooling and strictness

mypy — the reference implementation. Practical `pyproject.toml` for a serious codebase:

```
[tool.mypy]
python_version = "3.12"
strict = true
warn_unreachable = true
warn_return_any = true
disallow_any_explicit = false
enable_error_code = ["ignore-without-code", "redundant-expr", "truthy-bool"]

[[tool.mypy.overrides]]
module = ["legacy.*"]
ignore_errors = true
```

`strict = true` bundles a dozen flags including `disallow_untyped_defs`, `disallow_any_generics`, and `no_implicit_optional`. Adopting it on a legacy codebase: enable strict globally, then add per-module `ignore_errors` overrides and delete them one module at a time. Never use a blanket `# type: ignore` — always `# type: ignore[arg-type]` with the code, which `ignore-without-code` enforces.

pyright/pylance — faster, better inference and narrowing, different (often stricter) opinions on variance and protocol matching. Many libraries CI against both. `basedpyright` adds further strictness.

Stubs. `.pyi` files describe untyped code. Sources of stubs: inline (best — ship `py.typed` per PEP 561), bundled stub-only packages (`types-requests`), or `typeshed`. If you ship a typed library, the `py.typed` marker file in the package is mandatory or your users get `Any` for everything.

Where annotations stop helping. They do not validate at runtime. If input crosses a trust boundary — HTTP, a queue, a config file — you need actual validation (Part VIII), and the annotation is only the schema declaration.

2.10 Exercises for Part II

1. Write a `Result[T, E]` type with `Ok/Err` variants and make `assert_never` catch a missing branch.
2. Convert a decorator that currently returns `Callable[..., Any]` to a `ParamSpec` version; verify the IDE now completes the wrapped function's arguments.
3. Explain why `def f(x: list[float])` rejects `list[int]` but `def f(x: Sequence[float])` accepts it. Draw the variance argument.
4. Take a `runtime_checkable` protocol and construct an object that passes `isinstance` but crashes at call time. This is the limit of structural runtime checks.
5. Type a plugin registry so that `registry.get("json")` returns the specific handler class, using `Literal` overloads.

Part III — Concurrency and Parallelism

Three orthogonal questions decide your architecture: is the work **I/O-bound or CPU-bound**? does it need **shared mutable state**? can it tolerate **process boundaries**? Everything below follows from the answers.

3.1 The GIL, precisely

The Global Interpreter Lock is a mutex protecting interpreter state — most importantly reference counts. Only one thread executes bytecode at a time in the default build.

Mechanics worth knowing:

- A thread holds the GIL until it blocks on I/O, calls into a C extension that releases it, or exceeds the **switch interval** (default 5 ms, `sys.setswitchinterval()`).
- Since 3.2 the drop request is explicit: a waiting thread sets a flag and the holder yields at the next check. This fixed the old convoy problem but did not eliminate it for CPU-bound threads competing with I/O threads.
- The GIL does **not** make your code thread-safe. It guarantees no torn refcounts; it guarantees nothing about your invariants. `counter += 1` is **LOAD, ADD, STORE** — three bytecodes, interruptible between any of them.

```
# Not atomic. Loses increments under contention.
counter += 1

# Atomic-enough operations (single bytecode + C-level lock):
some_list.append(x)
some_dict.setdefault(k, v)
# But "check then act" is never atomic:
if k not in d: d[k] = v      # race
```

Consequences for design:

- **I/O-bound → threads or async help**, because the GIL is released during blocking syscalls.
- **CPU-bound in pure Python → threads do not help** in the default build; use processes, a native extension that releases the GIL, or a free-threaded build.
- **CPU-bound in NumPy/pandas/compiled code → threads often do help**, because those release the GIL around their inner loops.

3.2 Free-threading [3.13 experimental, 3.14 supported]

PEP 703 removes the GIL; PEP 779 declared the free-threaded build officially supported in Python 3.14, with the single-threaded performance penalty reduced to roughly 5–10% and the specializing interpreter re-enabled in that build. It ships as a separate build in which some objects are made immortal to avoid refcount contention.

Operational facts:

- The binary is separate and suffixed: `python3.14t.uv python install 3.14t` is the easy path.
- *If you import a C extension that has not declared free-threaded safety, the interpreter re-enables the GIL for the whole process* — your threads keep running but stop running in parallel. Watch for the `RuntimeWarning`; override at your own risk with `PYTHON_GIL=0`.
- `sys._is_gil_enabled()` tells you the truth at runtime. Assert it in benchmarks.
- Phase 3 (free-threading as the default build) has not been committed to a version.

What changes for *your* code: the data races that the GIL papered over become real. `counter += 1` was already broken; now it is broken faster and in more interesting ways. Under free-threading you must actually

lock shared mutable state, and `list/dict` internal consistency is preserved by per-object locks but your compound operations still are not atomic.

 Run the same CPU-bound thread pool benchmark on `python3.14` and `python3.14t`, asserting `sys._is_gil_enabled()` in each, and confirm every dependency is compatible before believing any speedup.

3.3 Threads

```
import threading

lock = threading.Lock()           # non-reentrant, fastest
rlock = threading.RLock()         # same thread may re-acquire; needed for
                                   # recursive/reentrant class APIs
sem = threading.Semaphore(10)     # bound concurrency
event = threading.Event()         # one-shot broadcast flag
barrier = threading.Barrier(4)    # rendezvous
cond = threading.Condition(lock)  # wait/notify on a predicate
```

`Condition` is the one people misuse. The contract is: hold the lock, **loop** on the predicate (never `if`), `wait()` releases the lock and re-acquires on wake:

```
with cond:
    while not queue:               # while, not if – spurious/stale wakeups
        cond.wait()
    item = queue.popleft()
```

Deadlock avoidance is mostly one rule: **establish a global lock ordering and never acquire out of order**. If you cannot, use `acquire(timeout=...)` and back off. `threading.Lock` has no owner and no deadlock detection; a lock acquired twice by the same thread simply hangs forever.

`threading.local()` gives per-thread storage — but in async code it is *wrong*, because many tasks share one thread. Use `contextvars` instead:

```
from contextvars import ContextVar
request_id: ContextVar[str] = ContextVar("request_id", default="-")

token = request_id.set("abc123")
try:
    ...
finally:
    request_id.reset(token)
```

`ContextVar` values propagate into tasks created *after* the set (each task copies the context at creation), and mutations inside a task do not leak back out. That containment is exactly what you want for request-scoped logging metadata.

Daemon threads (`daemon=True`) are killed abruptly at interpreter exit with no cleanup — no `finally`, no context manager exit. Never hold a file, socket, or database transaction in one.

3.4 asyncio internals

An `async def` function returns a **coroutine object** when called; nothing runs. The event loop drives it. `await x` requires `x` to implement `__await__` returning an iterator; awaiting yields control all the way up to the loop.

The layers:

- **Future** — a placeholder for a result with callbacks. Low level.
- **Task** — a Future that wraps a coroutine and drives it, scheduling each resumption on the loop.
- **Event loop** — a selector (`epoll/kqueue/IOCP`) plus a callback queue plus a timer heap. `loop.call_soon`, `call_later`, and I/O readiness callbacks feed a single-threaded run loop.

```
async def fetch(session, url):
    async with session.get(url) as resp:      # async context manager
        return await resp.text()

async def main():
    async with asyncio.TaskGroup() as tg:     # [3.11] structured concurrency
        tasks = [tg.create_task(fetch(s, u)) for u in urls]
    return [t.result() for t in tasks]
```

Cancellation is the hard part

`task.cancel()` schedules a `CancelledError` to be *raised inside* the coroutine at its next suspension point. Since 3.8 `CancelledError` inherits from `BaseException`, so `except Exception` does not swallow it — deliberately.

Rules:

- Never catch `CancelledError` without re-raising, unless you are implementing a cancellation boundary and you know it.
- Cleanup in `finally` must not `await` anything long, or the cancellation stalls. If you must await during cleanup, wrap in `asyncio.shield()` or a fresh `asyncio.timeout()`.
- `asyncio.shield(coro)` protects the inner operation, but the *awaiting* code still gets cancelled — the shielded task keeps running detached. That is usually a bug unless you keep a reference.
- [3.11] `uncancel()` and the cancelling-count model exist so nested `timeout()` blocks can tell "my timeout fired" from "my caller cancelled me."

```
async def with_cleanup():
    try:
        await long_operation()
    except asyncio.CancelledError:
        with contextlib.suppress(asyncio.TimeoutError):
            async with asyncio.timeout(2):
```

```

        await graceful_close()
    raise      # always re-raise

```

⚠ **Fire-and-forget tasks get garbage collected.** The loop keeps only a weak reference. `asyncio.create_task(f())` without holding the result can vanish mid-flight. Keep a set:

```

_background: set[asyncio.Task] = set()
def spawn(coro):
    t = asyncio.create_task(coro)
    _background.add(t)
    t.add_done_callback(_background.discard)
    return t

```

Better still: use a `TaskGroup` so lifetime is scoped and errors propagate.

Structured concurrency

`TaskGroup` [3.11] is the modern default. On exit it awaits all children; if any child raises, the rest are cancelled and errors surface as an `ExceptionGroup` (Part VII). Compare with `asyncio.gather`:

- `gather(..., return_exceptions=False)` — first exception propagates but siblings keep running. Leaks.
- `gather(..., return_exceptions=True)` — you get results and exceptions mixed in a list and must sort them out yourself.
- `TaskGroup` — nothing outlives the block. Prefer it.

`anyio` and `trio` take this further with nurseries and a uniform API across `asyncio`/`trio` backends; `anyio` is the pragmatic choice for library authors who must support both.

Other essentials

- `asyncio.timeout(delay)` [3.11] as an async context manager is the modern replacement for `wait_for`, and it composes.
- `asyncio.to_thread(fn, *args)` moves blocking work off the loop. Use it for any library call that is not async — a single blocking call freezes *every* task on that loop.
- `loop.run_in_executor(pool, fn)` for a controlled pool, including a `ProcessPoolExecutor` for CPU work.
- `asyncio.Queue` for backpressure between producers and consumers; give it a `maxsize` or you have built an unbounded memory leak.
- Debug mode (`PYTHONASYNCIODEBUG=1` or `asyncio.run(main(), debug=True)`) warns about slow callbacks and un-awaited coroutines. Run your test suite with it on.
- [3.12+] the eager task factory (`loop.set_task_factory(asyncio.eager_task_factory)`) starts coroutines synchronously until first suspension, cutting scheduling overhead for tasks that often complete without blocking.

Async generators and finalization

Async generators need the loop alive to run their `finally` blocks, which is why `asyncio.run()` calls `shutdown_asyncgens()`. If you drive the loop manually, do it yourself, or `aclose()` explicitly. `contextlib.aclosing()` makes this local and obvious:

```
async with contextlib.aclosing(stream()) as agen:
    async for chunk in agen:
        ...
```

3.5 Multiprocessing

Start methods, and why they matter more than anything else here:

- **fork** — fast, copies memory lazily. ⚠ Unsafe with threads: only the forking thread survives, and any lock held by another thread at fork time stays locked forever. This is the classic "hangs after logging" bug. Being phased out as the default on Linux.
- **spawn** — fresh interpreter, re-imports `__main__`. Safe, slow, requires everything passed to be picklable and the entry point guarded by `if __name__ == "__main__":`.
- **forkserver** — fork from a small clean server process. Good middle ground on POSIX.

```
import multiprocessing as mp
if __name__ == "__main__":
    ctx = mp.get_context("spawn")
    with ctx.Pool(4) as pool:
        results = pool.map(work, items)
```

The real cost is **serialisation**. Every argument and return value is pickled, piped, and unpickled. Passing a 500 MB DataFrame to four workers costs 2 GB of copying before any work happens. Mitigations: pass file paths or offsets instead of data; use `multiprocessing.shared_memory` with NumPy views; use Arrow/Plasma; or chunk with `imap_unordered` and a sensible `chunksize`.

```
from multiprocessing import shared_memory
import numpy as np
shm = shared_memory.SharedMemory(create=True, size=arr.nbytes)
view = np.ndarray(arr.shape, dtype=arr.dtype, buffer=shm.buf)
view[:] = arr[:]
# children attach by name; then shm.close() everywhere and shm.unlink() once
```

⚠ Shared memory leaks are real: if a child dies without `close()`, the resource tracker warns at exit and the segment can survive in `/dev/shm`. Always `unlink()` exactly once, from the owner.

`Manager()` gives proxied shared objects (dict, list, Namespace). Convenient and *slow* — every access is an IPC round trip.

`concurrent.futures` unifies threads and processes behind one API and is the right default:

```
from concurrent.futures import ProcessPoolExecutor, as_completed
with ProcessPoolExecutor(max_workers=8) as ex:
    futs = {ex.submit(score, doc): doc for doc in docs}
    for fut in as_completed(futs):
        try:
            result = fut.result()
        except Exception:
            logging.exception("failed on %s", futs[fut])
```

⚠ Exceptions in a worker are re-raised in the parent when you call `.result()` — and *only* then. Never iterate futures without touching `.result()`; you will silently discard failures. A worker that segfaults or is OOM-killed gives you a `BrokenProcessPool` and takes the whole pool with it.

3.6 Subinterpreters [3.12 C API, 3.13/3.14 stdlib]

PEP 684 gave each interpreter its own GIL; PEP 734 exposes multiple interpreters to Python code (`concurrent.interpreters`, with an `InterpreterPoolExecutor`). The model: isolated interpreters in one process, no shared objects, cheap-ish creation, message passing through channels/queues of simple picklable data.

Where it fits between the alternatives:

	Threads (GIL)	Free-threading	Subinterpreters	Processes
Parallel CPU	no	yes	yes	yes
Shared objects	yes	yes	no	no
Start cost	µs	µs	ms	tens of ms
Crash isolation	none	none	partial	full
C-ext support	universal	improving	limited	universal

Subinterpreters are still the least mature option; C extensions must be multi-phase-init aware to load in them.

3.7 Choosing, in one page

- Many sockets, low CPU → **asyncio**, one loop, `to_thread` for the odd blocking call.
- Blocking libraries you cannot replace, moderate concurrency → **ThreadPoolExecutor**.
- Pure-Python CPU work → **ProcessPoolExecutor** today; measure a **free-threaded** build before committing to it.
- NumPy/Polars/compiled CPU work → **threads** work; check the library releases the GIL.
- Isolation or untrusted plugins → **processes**, always.
- Mixed → an async front end submitting to a process pool executor is a good, boring architecture.

3.8 Exercises for Part III

1. Demonstrate a lost update with two threads incrementing a shared counter one million times. Fix it three ways (Lock, `itertools.count`, per-thread accumulation) and benchmark each.

2. Write a `TaskGroup`-based crawler with bounded concurrency (`Semaphore`), a timeout per request, and correct cancellation cleanup. Kill it with Ctrl-C and prove no connection leaks.
3. Reproduce the fork-with-threads deadlock, then fix it by switching to `forkserver`.
4. Measure pickling overhead: pass a 100 MB array to a process pool by value, then by `shared_memory`. Plot the crossover point.
5. Take a CPU-bound function and compare: single thread, 4 threads (GIL build), 4 threads (free-threaded build), 4 processes. Explain each number.

Part IV — Performance and Internals

The order of operations for performance work never changes: **measure, find the dominant cost, change the algorithm, then change the implementation, then leave Python**. Skipping straight to step four is how teams end up with a Cython module that is 3% of runtime.

4.1 Bytecode and the evaluation loop

CPython compiles source to bytecode stored in a code object, then executes it in a loop.

```
import dis
def f(a, b):
    return a * 2 + b
dis.dis(f)
```

What to look for when reading `dis` output:

- `LOAD_FAST` (locals, array index — fast) vs `LOAD_GLOBAL` (dict lookups — slower) vs `LOAD_ATTR` (full lookup protocol). Hoisting `self.method` or a global into a local inside a hot loop is a real, measurable win and the reason you see `_len = len` in old `stdlib` code.
- `CALL` counts. Function call overhead in Python is significant; inlining a tiny helper in a million-iteration loop matters.
- `BUILD_*` and `LIST_APPEND` in comprehensions vs repeated `.append` through the attribute protocol.
- [3.11+] `RESUME`, `PRECALL`/`CALL` restructuring, and specialised opcode families.

Code objects (`f.__code__`) expose `co_varnames`, `co_consts`, `co_names`, `co_freevars`, `co_cellvars`, `co_flags`, and `co_positions()` — the last is what powers [3.11]'s fine-grained error locations that underline the exact failing subexpression.

The specializing adaptive interpreter [3.11+, PEP 659]

At runtime CPython rewrites hot bytecode into specialised forms with inline caches. `BINARY_OP` on two ints becomes `BINARY_OP_ADD_INT` after it observes consistent types; `LOAD_ATTR` on a stable class layout becomes a version-tag check plus a direct offset read. If the assumption breaks, it de-optimises back.

Practical implications:

- **Monomorphic code is faster.** A function called with `int` every time specialises; one called with `int`, then `str`, then `Decimal` thrashes.

- **Stable class shapes matter.** Adding attributes after `__init__` or mutating a class at runtime invalidates version tags and kills attribute caches.
- Use `dis.dis(f, adaptive=True)` after warming a function to see what it actually specialised into.

Zero-cost exceptions [3.11]

`try` blocks no longer push a block-stack entry at runtime; the compiler emits an exception table mapping instruction ranges to handlers. Entering a `try` is now free; *raising* is somewhat more expensive.

Design consequence: wrapping hot code in `try/except` costs nothing when nothing raises, so EAFP is now unambiguously the right idiom — **unless the exception path is common**, in which case the check is cheaper.

The JIT and the interpreter loop [3.13+]

3.13 added an experimental copy-and-patch JIT (opt-in at build time) which compiles hot micro-op traces to machine code. 3.14 added an opt-in tail-call interpreter, < cite index="7-1">which the release described as improving performance by a few percent. Neither is a magic multiplier today, and neither is enabled in most distributed builds. Treat them as "check the release notes and benchmark your build," not as a plan.

4.2 Memory

Reference counting plus a cycle collector

Every object has a refcount; at zero it is freed immediately (deterministic, cache-friendly, and why `with open(...)` cleanup feels instant). The generational **garbage collector** exists only to break reference cycles.

- Three generations. Objects surviving a collection are promoted. Gen 0 runs often and cheaply; gen 2 is a full traversal of every container object and can pause for hundreds of milliseconds on large heaps.
- Only container types are tracked. `int`, `str`, and other atomic types cannot participate in cycles and are untracked.
- `gc.freeze()` moves everything currently alive into a permanent generation — call it after imports and before forking. It both shortens gen-2 collections and prevents copy-on-write page faults in forked children (refcount writes on shared pages are the reason "fork is cheap" is a lie for Python servers).

```
import gc
# preload everything, then:
gc.freeze()
# now fork workers
```

Tuning: `gc.set_threshold(700, 10, 10)` are the defaults; raising the gen-0 threshold helps allocation-heavy workloads. Disabling the GC entirely (`gc.disable()`) is viable only if you are certain you create no cycles, or you run a manual `gc.collect()` at a safe point. `gc.set_debug(gc.DEBUG_SAVEALL)` plus `gc.garbage` finds uncollectable cycles (historically those with `__del__`; much rarer since 3.4).

Allocation layers

Small objects (≤ 512 bytes) come from **pymalloc**: 4 KB pools inside 1 MB arenas, segregated by size class. Larger allocations go to the system allocator. Arenas are returned to the OS only when completely empty —

so **fragmentation means RSS stays high after you free a lot of small objects**. A long-running process that peaked at 8 GB may never give it back. The industrial answer is worker recycling (`--max-requests` in gunicorn) and not building the peak in the first place.

`sys.getsizeof` measures one object, shallowly. A `dict` of 1M small strings is mostly *not* in the dict. Use `tracemalloc` for attribution and `memray` for a full picture.

Immortal objects [3.12, PEP 683] give `None`, `True`, small ints and interned strings a sentinel refcount that is never modified — necessary groundwork for free-threading and a copy-on-write win.

Interning: identical string literals and small ints are shared. `sys.intern()` on repeated keys (parsing large CSVs, JSON with repeated field names) can cut memory dramatically and speed up dict lookups because equality short-circuits on identity. Never rely on `is` for value comparison — that is an implementation detail, and [3.8+] the compiler warns you about it.

Making objects cheaper

In rough order of impact: `__slots__` (Part I) → dataclasses with slots → `array.array` or NumPy for homogeneous numbers → `struct/memoryview` over raw buffers → generators instead of materialised lists.

4.3 The buffer protocol and zero copy

`memoryview` exposes an object's underlying buffer without copying:

```
data = bytearray(10_000_000)
mv = memoryview(data)
chunk = mv[1000:2000]           # no copy
chunk[:] = b"x" * 1000         # writes through to data
sock.send_into(mv[offset:])    # no intermediate bytes object
```

`bytes(mv)` copies; slicing does not. This turns quadratic parsing loops (`buf = buf[n:]`) into linear ones. `mmap` maps a file into memory and gives you the same interface over gigabytes without reading them. `struct` packs and unpacks binary records; `array.array` gives compact homogeneous storage. [3.12, PEP 688] `__buffer__`/`__release_buffer__` expose the protocol to pure Python classes and `collections.abc.Buffer` types it.

NumPy is the buffer protocol taken seriously:

- An array is a data pointer plus **shape**, **strides**, and dtype. Slicing produces a *view* sharing memory; fancy indexing (boolean masks, index arrays) produces a *copy*. Confusing them is the #1 NumPy performance and correctness bug.
- Broadcasting avoids materialising intermediates in *shape* but not in *time*: `(a + b) * c` on large arrays allocates two temporaries. `numexpr`, in-place ops (`np.add(a, b, out=a)`), or chunking fix it.
- C-contiguous vs Fortran-contiguous layout determines cache behaviour; a transpose is free but iterating the wrong axis afterwards is not.
- `np.ascontiguousarray`, `arr.flags`, and `arr.base` (is this a view?) are your diagnostic tools.

4.4 Profiling: the actual workflow

Never optimise without a profile. Tools, and when each is right:

Tool	Kind	Use for
<code>timeit</code>	microbenchmark	comparing two expressions
<code>cProfile</code> + <code>pstats</code>	deterministic, function-level	first look; ~2× overhead, distorts small calls
<code>py-spy</code>	sampling, no code changes	production ; attaches to a running PID, <code>py-spy dump</code> for hung processes
<code>scalene</code>	sampling, splits CPU/GPU/memory and Python vs native	finding "the time is in C"
<code>memray</code>	allocation tracing with flamegraphs	memory growth and peak attribution
<code>tracemalloc</code>	stdlib allocation snapshots	leak diffing inside tests
<code>pyinstrument</code>	sampling, call-tree output	readable async-aware profiles
<code>pytest-benchmark</code> / <code>pyperf</code>	statistical benchmarking	regression gates in CI
<code>perf</code> + [3.12] <code>perf trampolines</code>	native profiler with Python frames	mixed C/Python stacks
[3.14] <code>remote attach</code> / PEP 768	zero-overhead external debugger	inspecting a live process, <code>pdb</code> remote attach

```
import cProfile, pstats
with cProfile.Profile() as pr:
    main()
pstats.Stats(pr).sort_stats("cumulative").print_stats(30)
```

Read `totttime` (self) to find hot leaves, `cumtime` to find expensive subtrees. For async code, `cProfile` attributes time badly across awaits — use `pyinstrument` or `py-spy`.

Benchmark hygiene: warm up (specialisation needs iterations), pin frequency/turbo where possible, run enough repetitions to report a distribution not a number, compare medians with an interval, and never benchmark in CI on shared runners without treating the noise floor as ±20%.

`tracemalloc` leak-hunting pattern:

```
tracemalloc.start(25)
snap1 = tracemalloc.take_snapshot()
run_workload()
snap2 = tracemalloc.take_snapshot()
for stat in snap2.compare_to(snap1, "lineno")[:20]:
    print(stat)
```

4.5 Leaving Python

When the profile says the bottleneck is a genuine Python-level hot loop, options in ascending order of commitment:

- **Algorithmic/stdlib fix** — the right data structure, `set` membership instead of `list`, `functools.lru_cache`, `bisect`, `heapq`, `collections.deque`. Free, and usually bigger than any of the below.
- **NumPy/Polars vectorisation** — move the loop into C without writing C.
- **Cython** — annotate Python with C types; excellent for incremental optimisation of existing code and for wrapping C libraries. `cython -a` produces an annotated HTML showing which lines still touch the Python API.
- **mypyc** — compiles type-annotated Python to C extensions with no new syntax. Used to compile mypy itself. Great when you already have strict annotations.
- **Numba** — JIT for numeric loops via `@njit`; near-C for array math, restrictive about what it supports, `nopython=True` or nothing.
- **Rust with PyO3/maturin** — the modern default for new native code: memory safety, good tooling, wheels via maturin. Pydantic v2, Polars, uv, and ruff took this path.
- **C/C++ with the C API, `ctypes`, or `cffi`** — `ctypes` for calling an existing shared library with no build step; `cffi` for a saner, ABI- or API-mode binding.

Rules for native extensions

- **Release the GIL** around any long computation that does not touch Python objects (`Py_BEGIN_ALLOW_THREADS` / `py: nogil` in Cython / `py.allow_threads()` in PyO3). This is what makes threads useful for compiled work.
- **Stable ABI / limited API** (`Py_LIMITED_API`, `abi3` wheels) lets one wheel serve many Python versions. You give up some API surface and some speed; you gain not rebuilding every October. Free-threaded builds need their own tag (`abi3t` work is ongoing).
- **Multi-phase init** (PEP 489) and per-module state are required for subinterpreter support and are good hygiene regardless.
- Reference counting errors in C are the classic crash source: know which APIs return new vs borrowed references, and use `Py_XDECREF` on possibly-null.

4.6 A performance checklist

1. Is there a reproducible benchmark? If not, stop and write one.
2. What fraction of total time is this? (Amdahl: a 10× win on 5% of runtime is 4.5%.)
3. Is it algorithmic? Big-O beats constants.
4. Is it I/O? Then it is batching, caching, connection reuse, or concurrency — not CPU work.
5. Is it allocation? Reuse buffers, avoid intermediates, consider slots.
6. Is it a Python-level loop over data? Vectorise.
7. Only now: compile it.
8. Re-measure. Keep the benchmark in CI so the win does not silently regress.

4.7 Exercises for Part IV

1. `dis` a function before and after 10,000 calls with `adaptive=True`. Identify three specialised opcodes and explain the guard each one checks.
2. Build a deliberate reference cycle holding a 100 MB buffer. Show RSS not dropping, then fix it with `weakref` and with an explicit `gc.collect()`.
3. Parse a 1 GB log file three ways: line iteration, `mmap` + `memoryview`, and chunked reads. Measure time and peak RSS.
4. Take a nested-loop numeric function; port it to NumPy, then Numba, then Cython. Record the effort/speedup ratio for each.
5. Profile a web handler with `py-spy` in production-like load and find the surprise. There is always a surprise, and it is usually logging, JSON serialisation, or a DNS lookup.

Part V — Metaprogramming, Introspection and the Import System

5.1 Decorators, done properly

A decorator is a function taking a callable and returning a replacement. The complexity lives in preserving identity, signature, and behaviour.

```
from functools import wraps
import time

def timed(fn):
    @wraps(fn)
    def inner(*args, **kwargs):
        # copies __name__, __doc__, __qualname__,
        # __module__, __dict__, __wrapped__
        t0 = time.perf_counter()
        try:
            return fn(*args, **kwargs)
        finally:
            log.info("%s took %.3fs", fn.__qualname__, time.perf_counter() - t0)
    return inner
```

Without `@wraps`, you break `help()`, `inspect.signature`, pickling by name, Sphinx docs, and pytest's test collection. `__wrapped__` lets `inspect.signature` see through the wrapper — which is also why `inspect.signature(f, follow_wrapped=False)` exists.

Decorators with arguments are three nested functions. Support both `@deco` and `@deco(...)` with a `None` sentinel:

```
def retry(fn=None, *, times=3, exceptions=(Exception,)):
    if fn is None:
        return lambda f: retry(f, times=times, exceptions=exceptions)
    @wraps(fn)
    def inner(*a, **kw):
        for attempt in range(times):
            try:
```

```

        return fn(*a, **kw)
    except exceptions:
        if attempt == times - 1:
            raise
        time.sleep(2 ** attempt)
    return inner

```

Decorating methods works because the decorator returns a plain function, which is still a descriptor. But order matters: `@staticmethod` and `@classmethod` must be outermost (they produce non-callable descriptors in older versions), and a decorator that assumes `args[0]` is `self` breaks on static methods.

Class decorators are simpler than metaclasses and compose better — `@dataclass`, `@functools.total_ordering`, and `@runtime_checkable` are all class decorators.

The `functools` toolkit

- `lru_cache(maxsize)` / `cache` — memoisation keyed by a tuple of args (and a marker separating positional from keyword). ⚠ **Arguments must be hashable**, `f(1)` and `f(x=1)` are different keys, and a cache on a *method* keeps `self` alive forever — a textbook memory leak. Use `cached_property`, a per-instance cache, or a `WeakKeyDictionary`.
- `cached_property` — non-data descriptor writing into the instance dict; the second access never touches the descriptor. Does not work with `__slots__` (no dict to write to). Not thread-safe for expensive idempotent computation (the lock was removed in 3.12 after deadlocks; concurrent first-calls may compute twice).
- `singledispatch` / `singledispatchmethod` — type-based overloading on the first argument, registered by annotation. The Pythonic alternative to `isinstance` chains, and it lets third parties extend your function for their types.
- `partial` / `partialmethod` — pre-binding. Picklable, unlike a lambda.
- `reduce`, `total_ordering`, `wraps`, `cmp_to_key`.

```

from functools import singledispatch
@singledispatch
def render(node) -> str:
    raise TypeError(f"no renderer for {type(node).__name__}")

@render.register
def _(node: Text) -> str: return node.value

@render.register
def _(node: list) -> str: return "".join(map(render, node))

```

5.2 The AST

`ast` parses source into a tree you can inspect and rewrite. It is the foundation of linters, formatters, codemods, and safe-ish expression evaluation.

```
import ast
tree = ast.parse(source, filename="x.py")
print(ast.dump(tree, indent=2))

class RemoveAsserts(ast.NodeTransformer):
    def visit_Assert(self, node):
        return None # delete the node

new = ast.fix_missing_locations(RemoveAsserts().visit(tree))
code = compile(new, "x.py", "exec")
exec(code, {})
```

NodeVisitor for read-only analysis, **NodeTransformer** for rewriting (return the node, a new node, a list of nodes, or **None** to delete). Always **fix_missing_locations** before compiling generated nodes, or you get a cryptic **ValueError**. **ast.unparse()** [3.9+] round-trips a tree back to source (without comments — for comment-preserving work use LibCST).

Realistic uses: enforcing an in-house rule ruff cannot express; auto-generating stubs; extracting a dependency graph; a codemod across 400 files; a safe expression evaluator that whitelists node types.

5.3 Code objects, frames, and closures

```
def outer():
    x = 1
    def inner():
        nonlocal x
        x += 1
        return x
    return inner

f = outer()
f.__closure__          # (<cell at 0x...: int object at 0x...>,)
f.__code__.co_freevars # ('x',)
```

A closure is a tuple of **cells**. Cells are mutable boxes shared by the defining and nested scopes; **nonlocal** writes through the cell. This is why the late-binding loop bug happens:

```
fs = [lambda: i for i in range(3)]
[f() for f in fs] # [2, 2, 2] — one cell, read at call time
fs = [lambda i=i: i for i in range(3)] # [0, 1, 2] — default binds at def time
```

Frames: **sys._getframe(1)** gets the caller's frame; **inspect.currentframe()** is the polite version. Frames expose **f_locals**, **f_globals**, **f_back**, **f_lineno**, **f_code**. Legitimate uses are logging, debugging, and error reporting; frame-based magic in library APIs (guessing the caller's variable names) is a maintenance disaster and breaks under other implementations. **△** Holding a frame reference keeps every local alive — including in tracebacks, which is why storing exceptions in a long-lived structure leaks. **traceback.clear_frames(tb)** or

`del` the exception variable (Python does this automatically at the end of an `except` block, which is why the name is unbound afterwards).

5.4 exec, eval, compile

`compile(source, filename, mode)` with mode `"exec"`, `"eval"`, or `"single"`. `exec` takes globals and locals dicts; if you pass only globals, locals is the same dict.

```
ns = {}
exec(compile("def f(x): return x * 2", "<gen>", "exec"), ns)
ns["f"](21)
```

This is how `dataclasses`, `attrs`, and `namedtuple` generate `__init__` — building source text and `exec`ing it is dramatically faster at *call* time than a generic loop, at the cost of construction time.

⚠ `eval` on untrusted input is unfixable (Part XII). No amount of `__builtins__` stripping is a sandbox.

5.5 The import system

`import x.y` executes, in order: check `sys.modules` (the cache — note it holds partially-initialised modules during circular imports, which is why the failure is `AttributeError`, not `ImportError`); then walk `sys.meta_path` finders; a finder returns a **spec** naming a **loader**; the loader creates and executes the module; the module is bound into `sys.modules` *before* execution completes.

Key pieces:

- `sys.meta_path` — `BuiltinImporter`, `FrozenImporter`, `PathFinder`. Insert your own to implement import-from-a-database, import-from-a-zip, or lazy loading.
- `sys.path_hooks` and `sys.path_importer_cache` — how `PathFinder` turns a `sys.path` entry into a finder.
- **Namespace packages** (PEP 420) — a directory with no `__init__.py` becomes a portion of a namespace package that can span multiple `sys.path` entries. Powerful for plugin trees (`mycompany.plugins.*` from separate distributions), and a silent trap when you forget an `__init__.py` and get a package with no contents instead of an error.
- `importlib.import_module`, `importlib.reload` (dangerous: existing references keep the old classes, so `isinstance` fails mysteriously), `importlib.util.find_spec`.
- `importlib.resources` — the correct way to read package data. **Never** use `__file__`-relative paths; they break in zip imports, wheels-as-zipapps, and frozen builds.

```
from importlib.resources import files
schema = files("mypkg.data").joinpath("schema.json").read_text()
```

Lazy imports for expensive optional dependencies:

```
import importlib.util, sys
def lazy(name):
```

```
spec = importlib.util.find_spec(name)
module = importlib.util.module_from_spec(spec)
loader = importlib.util.LazyLoader(spec.loader)
sys.modules[name] = module
loader.exec_module(module)
return module
```

Circular imports: fix by moving the import inside the function, by extracting the shared piece into a third module, or by `if TYPE_CHECKING`: when it is only needed for annotations.

5.6 Tracing, profiling hooks and monitoring

- `sys.settrace(fn)` — per-line/call/return/exception callbacks. This is how `pdb` and `coverage` worked. Enormous overhead (often 10–100×) because it fires on every line.
- `sys.setprofile(fn)` — call/return only. Cheaper; `cProfile`'s mechanism.
- [3.12, PEP 669] `sys.monitoring` — the modern API. You register a tool id and callbacks for specific events (`PY_START`, `LINE`, `BRANCH`, `RAISE`, ...), and the interpreter instruments only what you asked for, with per-code-object granularity and near-zero cost for unmonitored code. Coverage tools and debuggers moved onto it and got dramatically faster.

```
import sys.monitoring as mon          # conceptually
TOOL = mon.COVERAGE_ID
mon.use_tool_id(TOOL, "mycov")
mon.set_events(TOOL, mon.events.LINE)
mon.register_callback(TOOL, mon.events.LINE, lambda code, lineno: record(code,
lineno))
```

[3.14] adds an external-process debugging interface (PEP 768) so tools can attach to a running interpreter safely — the foundation for `pdb` remote attach and low-overhead production inspection.

audit hooks (PEP 578): `sys.addaudithook` fires on security-relevant events (`exec`, `open`, `socket.connect`, `subprocess.Popen`). The right place to build a runtime allow-list or a forensic log; note hooks cannot be removed once installed.

5.7 Exercises for Part V

1. Write a decorator that works on functions, methods, classmethods and async functions, preserving signatures for all four. Verify with `inspect.signature`.
2. Build an AST checker that fails CI when a mutable default argument is used.
3. Implement a meta-path finder that imports modules from a dict of source strings.
4. Instrument a function with `sys.settrace` and then `sys.monitoring`; measure the overhead difference.
5. Explain, using cells, why `functools.lru_cache` on a method leaks, and write a version that does not.

Part VI — Iteration, Generators and Laziness

6.1 The iterator protocol

`iter(obj)` returns an iterator: an object with `__next__` raising `StopIteration` when exhausted, and `__iter__` returning itself. Iterables are not iterators; conflating them is the source of the "my object can only be looped over once" bug.

```
class Countdown:
    # iterable, reusable
    def __init__(self, n):
        self.n = n
    def __iter__(self):
        return iter(range(self.n, 0, -1))  # fresh iterator each time
```

If `__iter__` returns `self` and mutates state, a second `for` loop silently sees an empty sequence — the classic bug with file objects and generators.

Two-argument `iter(callable, sentinel)` is the underused form:

```
for chunk in iter(lambda: f.read(8192), b''):
    process(chunk)
```

6.2 Generators

A function containing `yield` returns a generator when called. Execution is suspended and resumed, with locals preserved on a heap-allocated frame.

The full protocol is bidirectional:

- `gen.send(value)` — resume, making `yield` evaluate to `value`. The first `send` must be `None` (or use `next`).
- `gen.throw(exc)` — raise inside the generator at the suspension point.
- `gen.close()` — raise `GeneratorExit` at the suspension point. Catching it and yielding again is a `RuntimeError`.

```
def averager():
    total = count = 0
    while True:
        value = yield (total / count if count else None)
        total += value; count += 1

a = averager(); next(a)
a.send(10); a.send(20)  # 15.0
```

`yield from sub` delegates completely: it forwards `send`, `throw`, and `close` to the sub-generator and evaluates to the sub-generator's `return` value. It is not sugar for a `for` loop.

```
def chain(*iterables):
    for it in iterables:
        yield from it
```

⚠ **finally in generators is not guaranteed to run promptly.** An abandoned generator's cleanup runs when it is collected. Under exceptions or in long-lived programs that can be much later — or never at shutdown. If a generator holds a lock, file, or transaction, wrap it in `contextlib.closing`, or restructure so the resource is managed outside.

`return value` inside a generator sets `StopIteration.value`, which is what `yield from` reads.

6.3 Generators vs coroutines

Historically `asyncio` used generator-based coroutines (`@asyncio.coroutine` + `yield from`). Since 3.5 they are distinct types: `async def` produces a coroutine with `__await__`, and `await` is not `yield from` — you cannot `yield` inside an ordinary coroutine (only inside `async def` generators, which are their own type with `__anext__`/`asend`/`athrow`/`aclose`).

6.4 Pipelines and backpressure

Generators compose into pipelines with $O(1)$ memory:

```
def read(path):
    with open(path) as f:
        yield from f

def parse(lines):
    for line in lines:
        if line.strip():
            yield json.loads(line)

def enrich(records, lookup):
    for r in records:
        yield {**r, "region": lookup.get(r["id"], "unknown")}

for record in enrich(parse(read(path)), regions):
    sink(record)
```

Nothing is materialised; memory is constant regardless of file size. **Backpressure is automatic** in a pull pipeline — the slowest consumer sets the pace. That property disappears the moment you introduce a queue or a thread pool, at which point you must bound the queue explicitly.

⚠ Laziness and resources interact badly: the `with open` above stays open until the generator is exhausted or closed. Returning a generator from a function that opened a file is an anti-pattern in library APIs; return a context manager instead.

6.5 itertools, in anger

The ones expert code actually uses:

- `islice(it, start, stop, step)` — slice any iterator (no negative indices).
- `chain / chain.from_iterable` — flatten one level.
- `groupby(it, key)` — [⚠](#) groups **consecutive** runs only; sort first, and the group iterator is invalidated when you advance to the next group.
- `tee(it, n)` — `n` independent iterators, but it buffers everything consumed by the fastest; unbounded memory if they diverge.
- `zip_longest`, and [3.10] `zip(..., strict=True)` to catch length mismatches instead of silently truncating.
- `accumulate(it, func, initial=)` — running totals, running max, state machines.
- `pairwise(it)` [3.10] — sliding window of 2; deltas without index arithmetic.
- `batched(it, n)` [3.12] — the chunking function everyone re-implemented.
- `count, cycle, repeat` — infinite; always bound them with `islice` or a condition.
- `product, permutations, combinations, combinations_with_replacement`.
- `filterfalse, dropwhile, takewhile, starmap, compress`.

`more-itertools` fills the gaps: `chunked, windowed, unique_everseen, first, peekable, spy, bucket, ilen, flatten, always_iterable`.

Two idioms worth memorising:

```
# stable dedupe preserving order (dict is ordered [3.7+])
list(dict.fromkeys(items))

# sliding window of arbitrary size
from collections import deque
def windowed(it, n):
    win = deque(maxlen=n)
    for x in it:
        win.append(x)
        if len(win) == n:
            yield tuple(win)
```

6.6 Comprehension semantics

Comprehensions have their own scope (the loop variable does not leak; the walrus `:=` deliberately does assign outward). Generator expressions evaluate the **outermost iterable eagerly** at creation and everything else lazily — which is why a `gen = (f(x) for x in source)` created before `source` is populated still sees an empty source, while later filters see live values.

Prefer a generator expression when feeding a consumer that reduces (`sum, any, min, join`); prefer a list comprehension when you will index or iterate more than once. `sum(x for x in ...)` avoids building a list; `"".join(...)` requires an iterable and is fine either way, though a list is marginally faster because `join` pre-sizes.

6.7 Exercises for Part VI

1. Write a `pushback` iterator supporting `peek()` without consuming, then use it to write a recursive-descent parser.
2. Demonstrate `groupby` producing wrong results on unsorted input, then fix it.
3. Build a generator pipeline that reads a 5 GB CSV, filters, aggregates, and writes output using less than 100 MB of RSS. Prove it with `tracemalloc`.
4. Implement `tee` yourself with a shared deque, and explain its memory characteristics.
5. Write a generator that acquires a lock, and demonstrate the lock being held after abandonment. Fix it two ways.

Part VII — Errors, Context Managers and API Design

7.1 Designing an exception hierarchy

Rules that hold up over years of maintenance:

1. **One base exception per library.** `class RedisError(Exception)`. Callers can then catch everything you raise without catching everything.
2. **Subclass the `stdlib` type callers would otherwise expect** when the semantics match: your `ConfigKeyError(YourError, KeyError)` lets `d["missing"]`-style code keep working. Multiple inheritance here is legitimate and useful.
3. **Carry structured data, not just a message.** Attributes are greppable and testable; parsing message strings is not.
4. **Never raise bare `Exception`**, and never catch it without re-raising or logging with a traceback.
5. **Don't leak internals:** wrap `sqlite3.OperationalError` in your own type at the boundary, `raise ... from` the original.

```
class StorageError(Exception):
    """Base for everything this package raises."""

class ObjectNotFound(StorageError, KeyError):
    def __init__(self, bucket: str, key: str):
        super().__init__(f"{bucket}/{key} not found")
        self.bucket, self.key = bucket, key
```

7.2 Chaining: `__cause__`, `__context__`, `__suppress_context__`

- Raising inside an `except` block sets `__context__` automatically → "During handling of the above exception, another exception occurred."
- `raise New() from old` sets `__cause__` → "The above exception was the direct cause..." — use this when the new exception is a *translation* of the old.
- `raise New() from None` sets `__suppress_context__` and hides the chain — use it when the inner exception is an implementation detail that would only confuse (e.g. a failed `dict` lookup inside your `__getattr__`).

```
try:
    conn.execute(sql)
except sqlite3.OperationalError as exc:
    raise StorageUnavailable(dsn) from exc    # keep the cause
```

⚠ Losing the cause is the most common debuggability crime in wrapper code. `from exc` costs nothing.

7.3 Exception groups and `except*` [3.11, PEP 654]

Structured concurrency needs to report *several* simultaneous failures. `ExceptionGroup` is a container; `except*` matches by type *inside* the group and can match several clauses.

```
try:
    async with asyncio.TaskGroup() as tg:
        tg.create_task(a()); tg.create_task(b())
except* ConnectionError as eg:
    for exc in eg.exceptions:
        log.warning("connection issue: %s", exc)
except* ValueError as eg:
    log.error("bad data in %d tasks", len(eg.exceptions))
```

Semantics to internalise:

- Every `except*` clause that matches runs — unlike `except`, where the first wins.
- The variable is always an `ExceptionGroup`, even for one exception.
- Groups nest; use `eg.subgroup(pred)` / `eg.split(pred)` to partition, both of which preserve tracebacks.
- `BaseExceptionGroup` is used automatically if any member is a `BaseException` (e.g. `CancelledError`).
- Do not put `ExceptionGroup` in a public API "just in case" — it forces every caller to handle groups. Raise it only where genuine concurrent failure exists.

7.4 Context managers and `contextlib`

The protocol: `__enter__` returns the bound value; `__exit__(exc_type, exc, tb)` returns truthy to **suppress**. Suppressing silently is almost always wrong; be explicit.

```
from contextlib import contextmanager
@contextmanager
def transaction(conn):
    conn.execute("BEGIN")
    try:
        yield conn
    except BaseException:    # BaseException, so Ctrl-C rolls back too
        conn.execute("ROLLBACK")
        raise
```

```
else:
    conn.execute("COMMIT")
```

⚠ A `@contextmanager` generator that yields more than once raises `RuntimeError`. And a generator-based CM is **not reentrant or reusable** — entering the same object twice breaks. For a reusable one, write a class or return a fresh instance from a factory.

The toolkit:

- `ExitStack` — dynamic, variable numbers of context managers; `enter_context`, `callback`, `pop_all` (transfer ownership so cleanup does *not* run — the correct way to hand a half-built resource to a caller).
- `AsyncExitStack`, `aclosing`, `asyncontextmanager` for the async equivalents.
- `suppress(FileNotFoundError)` — an honest, scoped replacement for `try/except/pass`.
- `redirect_stdout` / `redirect_stderr`, `chdir` [3.11] (⚠ process-global, never in threaded code), `nullcontext` (for optional context managers).

```
with ExitStack() as stack:
    files = [stack.enter_context(open(p)) for p in paths]
    stack.callback(cleanup_temp_dir)
    process(files)
```

7.5 API design principles

- **Keyword-only arguments** for anything boolean or optional: `def render(doc, *, strict=False, encoding="utf-8")`. Booleans at call sites are unreadable and you can never reorder positionals later.
- **Positional-only** (/) for arguments whose names are not part of the contract — it frees you to rename them and matches `stdlib` behaviour.
- **Return values over out-parameters; exceptions over sentinel returns**, unless failure is routine and expected (`dict.get`).
- **Narrow inputs, wide outputs**: accept `Iterable`, return `list`. Accept `os.PathLike` | `str`, return `Path`.
- **No mutable default arguments**. `def f(items=[])` shares one list across all calls forever. Use `None` and construct inside.
- **Make illegal states unrepresentable**: frozen dataclasses, enums instead of strings, separate types instead of a `mode` flag.
- `__all__` defines the public surface; prefix everything else with `_`. Names without a leading underscore and not in `__all__` are still de-facto public to users of `from x import *` — be deliberate.
- **Errors should say what to do**, not just what happened: include the offending value, the expected values, and the parameter name.

Deprecation

Semantic versioning with a real policy:

```
import warnings
def old_api(x):
```

```

warnings.warn(
    "old_api() is deprecated since 2.3 and will be removed in 3.0; use
new_api()",
    DeprecationWarning,
    stacklevel=2,                # point at the CALLER, not this line
)
return new_api(x)

```

`stacklevel=2` (or `skip_file_prefixes` [3.12]) is what makes the warning actionable. `DeprecationWarning` is hidden by default outside `__main__`; run your tests with `-W error::DeprecationWarning` so *your* CI sees your dependencies' warnings, and so consumers' CI sees yours. [3.13] `warnings.deprecated` / `typing.deprecated` (PEP 702) makes deprecation visible to type checkers as well.

Timeline discipline: announce in a minor release, warn for at least one full minor cycle, remove in a major. Document the replacement in the message itself.

Part VIII — Data Modeling

8.1 dataclasses

```

from dataclasses import dataclass, field, asdict, replace

@dataclass(frozen=True, slots=True, kw_only=True, order=True)
class Trade:
    symbol: str
    quantity: int
    price: float = 0.0
    tags: list[str] = field(default_factory=list, compare=False, repr=False)
    _internal: int = field(default=0, init=False)

    def __post_init__(self):
        if self.quantity <= 0:
            raise ValueError("quantity must be positive")

```

Parameter by parameter:

- `frozen=True` — generates `__setattr__`/`__delattr__` that raise. Note it is *shallow*: the `tags` list is still mutable. Frozen + `eq` gives you `__hash__` automatically.
- `slots=True` [3.10] — creates a **new class object**. ⚠ Breaks zero-arg `super()` in methods and any decorator that captured the pre-transform class.
- `kw_only=True` [3.10] — sidesteps the "non-default argument follows default argument" inheritance problem entirely. Excellent default for anything with more than three fields.
- `order=True` — generates comparisons from the field tuple, in declaration order.
- `field(default_factory=...)` — mandatory for mutable defaults.
- `field(compare=False, repr=False, hash=None)` — exclude noisy or derived fields.
- `field(init=False)` — computed in `__post_init__`.

- `ClassVar[...]` annotations are excluded from fields; `InitVar[...]` are init-only and passed to `__post_init__`.

`__post_init__` is your validation hook. On a frozen class you cannot assign, so use `object.__setattr__(self, "name", value)` — ugly but correct and idiomatic.

`replace(obj, price=1.5)` is the functional-update idiom for frozen objects. `asdict` recurses (and deep-copies — expensive on big trees; `dataclasses.fields()` plus your own loop is faster).

Inheritance: fields are ordered base-first, so a base with defaults blocks a subclass without them unless you use `kw_only`.

8.2 attrs and Pydantic v2 — choosing

	dataclasses	attrs	Pydantic v2
Dependency	stdlib	third-party	third-party (Rust core)
Validation	manual	validators/converters	automatic, from annotations
Coercion	none	opt-in converters	configurable strict/lax
(De)serialisation	<code>asdict</code> only	<code>cattrs</code>	first-class, fast
Speed of construction	fast	fast	fast (Rust), validation costs
Best for	internal structs	rich internal models	trust boundaries

The decision rule: **use dataclasses for data you created, Pydantic for data that came from outside**. HTTP bodies, queue messages, config files, and third-party APIs all need real validation; internal value objects do not, and paying validation cost on every internal construction is waste.

```
from pydantic import BaseModel, Field, field_validator, ConfigDict

class WebhookPayload(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True, strict=False)
    event_id: str = Field(min_length=1, max_length=64)
    amount_cents: int = Field(gt=0)
    currency: str = Field(pattern=r"^[A-Z]{3}$")

    @field_validator("currency")
    @classmethod
    def known_currency(cls, v: str) -> str:
        if v not in SUPPORTED:
            raise ValueError(f"unsupported currency {v}")
        return v
```

Pydantic v2 notes: `extra="forbid"` is what turns a typo in a config file into an error instead of a silently-ignored key. `model_validate` / `model_dump` (v2) replace `parse_obj` / `dict` (v1). `TypeAdapter` validates arbitrary annotated types without defining a model. Discriminated unions (`Field(discriminator="type")`)

give fast, precise polymorphic parsing. `Annotated[int, Field(ge=0)]` keeps constraints in the type, so they compose and are reusable.

`attrs` remains the most flexible: `@define`, converters, validators, `__attrs_post_init__`, and `cattrs` for structuring/unstructuring with none of Pydantic's coercion opinions.

8.3 enum

```
from enum import Enum, StrEnum, IntEnum, Flag, auto, unique

@unique
class State(StrEnum):          # [3.11] members ARE strings
    PENDING = "pending"
    RUNNING = "running"

class Perm(Flag):
    READ = auto(); WRITE = auto(); EXEC = auto()
    ALL = READ | WRITE | EXEC

p = Perm.READ | Perm.WRITE
Perm.WRITE in p                # True
```

Points that catch people out:

- Duplicate values create **aliases**, not new members; `@unique` forbids them. Aliases do not appear in iteration but are reachable by value.
- `_missing_(cls, value)` is the hook for lenient parsing (case-insensitive lookup, legacy value mapping).
- `IntEnum/StrEnum` are for interoperability with code that expects the primitive — they compare equal to raw values, which is convenient and occasionally hides bugs. Prefer plain `Enum` unless you need the compatibility.
- `Flag` supports set-like operations; [3.11] `boundary=` controls what happens with out-of-range bits.
- Members are singletons: compare with `is`.
- `auto()` values, `_generate_next_value_`, and `@member/@nonmember` [3.11] for controlling what becomes a member.

Enums beat string constants because they are exhaustively checkable (Part II), self-documenting in tracebacks, and impossible to typo silently.

8.4 collections and the ABC hierarchy

`collections.abc` is both a type vocabulary and a set of mixins. Inherit from `Sequence` and implement `__len__` + `__getitem__`, and you get `__contains__`, `__iter__`, `__reversed__`, `index`, and `count` free.

```
from collections.abc import Sequence
class Page(Sequence):
    def __init__(self, rows): self._rows = rows
```

```
def __len__(self): return len(self._rows)
def __getitem__(self, i): return self._rows[i]
```

The hierarchy worth memorising: `Container` / `Iterable` / `Sized` → `Collection` → `Sequence` / `Set` / `Mapping`, plus `MutableX` variants, `Iterator`, `Generator`, `Callable`, `Hashable`, `Awaitable`, `Coroutine`, `AsyncIterable`, `AsyncIterator`, `Buffer` [3.12].

`ABCMeta.register(cls)` declares a virtual subclass — `isinstance` passes with no inheritance and no method verification. `__subclasshook__` implements structural checks (how `isinstance(x, Iterable)` works for anything with `__iter__`).

For annotations, prefer the ABCs over concrete types (Part II §2.3). For implementation, the concrete `collections` types matter:

- `deque` — $O(1)$ at both ends, `maxlen` for ring buffers. `list.pop(0)` is $O(n)$; this is the fix.
- `defaultdict` —  reading a missing key *inserts* it; that silently grows dicts and breaks membership tests afterwards.
- `Counter` — *most common*, arithmetic on multisets.
- `ChainMap` — layered lookup for config precedence without merging.
- `OrderedDict` — still meaningful for `move_to_end`, `popitem(last=False)`, and order-sensitive equality (plain dicts compare order-insensitively).
- `namedtuple` / `NamedTuple` — tuple-compatible records; good for interop, but a frozen dataclass with slots is usually clearer.

8.5 Exercises for Parts VII–VIII

1. Design the exception hierarchy for an HTTP client: transport errors, timeouts, 4xx, 5xx, decoding failures. Make retry logic expressible as a single `except` clause.
2. Convert a `gather`-based fan-out to `TaskGroup` and handle partial failures with `except*`.
3. Write an `ExitStack`-based function that opens N resources and, on success, transfers ownership to the caller via `pop_all()`.
4. Model a config file with Pydantic such that unknown keys are errors, env vars override file values, and secrets never appear in `repr`.
5. Build a `Sequence` subclass over a memory-mapped file with no full materialisation.

Part IX — Packaging, Distribution and Environments

9.1 The standards, and what each one solved

PEP	Gives you
427 / 491	the wheel — a built, installable zip; no arbitrary code at install time
517	build backends : <code>pip</code> calls a documented interface, not <code>setup.py</code> directly
518	<code>[build-system]</code> — declare build dependencies in <code>pyproject.toml</code>
621	<code>[project]</code> — standard metadata (name, version, deps, entry points)

PEP	Gives you
660	standardised editable installs
420	namespace packages
561	py.typed — distributing type information
508	dependency specifier syntax and environment markers
440	version specifiers and ordering
751	a standard lock file format

The historical arc: `setup.py` executed arbitrary code at install time, so nobody could resolve metadata without running it. PEP 517/518/621 killed that. Today `pyproject.toml` is the only file you need.

9.2 A complete pyproject.toml

```
[build-system]
requires = ["hatchling>=1.24"]
build-backend = "hatchling.build"

[project]
name = "acme-widgets"
version = "2.3.0"
description = "Widget manipulation for the ACME platform"
readme = "README.md"
requires-python = ">=3.11"
license = "Apache-2.0"
authors = [{name = "A. Engineer", email = "eng@acme.example"}]
keywords = ["widgets"]
classifiers = [
    "Programming Language :: Python :: 3 :: Only",
    "Typing :: Typed",
]
dependencies = [
    "httpx>=0.27,<1.0",
    "pydantic>=2.7,<3",
    "typing-extensions>=4.10; python_version < '3.12'",
]

[project.optional-dependencies]
cli = ["typer>=0.12"]

[dependency-groups]
# dev deps, not shipped to users
dev = ["pytest>=8", "mypy>=1.10", "ruff>=0.5", "hypothesis>=6"]

[project.scripts]
acme = "acme.cli:main"

[project.entry-points."acme.plugins"]
csv = "acme.plugins.csv:Plugin"
```

```
[project.urls]
Homepage = "https://example.com"
Changelog = "https://example.com/changelog"

[tool.ruff]
line-length = 100
[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B", "SIM", "RUF", "ASYNC", "S"]
```

Version constraints. Libraries: lower bounds always, upper bounds only where you have evidence of breakage (a blanket `<2` on every dependency makes your package uninstallable alongside others). Applications: pin everything via a lock file. This asymmetry is the single most important packaging rule.

Backends: `hatchling` (fast, good defaults), `setuptools` (universal, C extensions), `poetry-core`, `pdm-backend`, `flit-core` (pure-Python simplicity), `maturin` (Rust), `scikit-build-core` (CMake/C++).

9.3 Wheels, tags and ABI

A wheel filename encodes compatibility: `acme-2.3.0-cp312-cp312-manylinux_2_28_x86_64.whl` is `{distribution}-{version}-{python tag}-{abi tag}-{platform tag}`.

- `py3-none-any` — pure Python, installs anywhere. Aim for this if you can.
- `cp312-cp312-...` — built for one CPython version and ABI.
- `cp39-abi3-...` — built against the **stable ABI**; one wheel serves 3.9 and everything after.
- `manylinux_2_28` — glibc-based tag guaranteeing the wheel runs on any distro with glibc ≥ 2.28 ; built in the official container images. `musllinux` for Alpine.
- Free-threaded builds use a `t` suffix (`cp314t`); a stable ABI for them (`abi3t`) is being worked on so extensions can ship one wheel across free-threaded releases.

`cibuildwheel` in CI builds the matrix (OS $\times$ Python $\times$ arch) and runs `auditwheel/delocate` to vendor shared libraries. The sdist remains important: it is the fallback for platforms without a wheel and the input for distro packagers, so it must contain everything needed to build.

9.4 Environments, resolution and locking

Virtual environments (PEP 405) are a `pyvenv.cfg` plus a `bin/` and a `site-packages`. Activation only manipulates `PATH`; running `.venv/bin/python` directly is equivalent and better in scripts. Always call `python -m pip`, never bare `pip`, so you know which interpreter you are modifying.

Resolution is a SAT-like backtracking problem: find one version of each package satisfying every constraint. Pip's resolver backtracks and can spend minutes downloading old sdists to read their metadata when constraints conflict. Symptoms of an over-constrained tree: long resolution, or `ResolutionImpossible`.

Lock files pin the entire transitive graph with hashes so builds are reproducible.

- `uv` — extremely fast Rust resolver/installer with `uv.lock`, `uv sync`, `uv run`, and Python version management. The current default recommendation for new projects.
- `poetry`, `pdm` — full project managers with their own lock formats.

- `pip-tools` — `requirements.in` → `requirements.txt` with `--generate-hashes`; minimal and pip-native.
- PEP 751 defines a standard interoperable lock format so this fragmentation ends.

For applications, install with hash checking so a compromised index cannot substitute a package:

```
pip install --require-hashes -r requirements.txt
```

Editable installs (`pip install -e .`) put a path hook or `.pth` into site-packages so your source directory is importable. The `src/` layout (`src/acme/__init__.py`) is strongly recommended: it makes it impossible to accidentally import your package from the working directory instead of the installed one, which is how "it works locally, breaks in CI" bugs are born.

9.5 Entry points and plugins

Entry points let a *third-party* distribution register itself with your application, discovered without importing anything:

```
from importlib.metadata import entry_points
for ep in entry_points(group="acme.plugins"):
    plugin_cls = ep.load()          # imports only when you ask
    register(ep.name, plugin_cls)
```

Alternatives: namespace packages (scan `acme.plugins.*`), or explicit config. Entry points are the standard and integrate with everything; namespace packages are better when plugins are internal and you control the tree.

`importlib.metadata` also gives you `version("acme-widgets")`, `metadata()`, `files()`, and `requires()` — use it instead of hard-coding `__version__` in two places. (`version(__package__)` at import time is the idiom, though it costs a metadata read.)

9.6 Publishing and supply chain

```
python -m build          # builds sdist + wheel into dist/
twine check dist/*
twine upload dist/*      # or better: trusted publishing from CI
```

Trusted publishing (OIDC) replaces long-lived API tokens: PyPI verifies a short-lived token issued by your CI provider for a specific repository and workflow. There is no secret to leak. Use it.

Supply-chain hygiene, in priority order:

1. Trusted publishing + 2FA on the account.
2. Hash-pinned lock files for applications; `--require-hashes` in deploys.
3. Automated dependency updates (Dependabot/Renovate) with CI gates — stale is also a vulnerability.

4. Vulnerability scanning (`pip-audit`, `osv-scanner`) in CI.
5. Attestations/provenance and an SBOM (CycloneDX or SPDX) for anything you ship to customers.
6. Beware **typosquatting** and **dependency confusion**: pin your index, and if you use a private index, ensure it cannot be shadowed by a public package of the same name.

⚠️ `pip install` from an sdist executes the build backend — that is code execution. Prefer wheels, and never install from an untrusted index.

Part X — Testing and Quality

10.1 pytest, properly

Fixtures are dependency injection with lifecycle management. Scopes: `function` (default), `class`, `module`, `package`, `session`. Anything after the `yield` is teardown and runs even if the test fails.

```
import pytest

@pytest.fixture(scope="session")
def db_url():
    with PostgresContainer() as pg:
        yield pg.url  # torn down after the whole session

@pytest.fixture
def conn(db_url):
    c = connect(db_url)
    tx = c.begin()
    yield c
    tx.rollback()  # every test isolated, no truncation needed
```

The transaction-rollback pattern above is the single highest-value database testing trick: session-scoped container, function-scoped transaction.

- `conftest.py` files layer by directory — fixtures are available to everything below them. This is the mechanism for per-package test configuration.
- `autouse=True` applies a fixture everywhere in scope. Use sparingly; invisible dependencies are hard to debug.
- `request` gives access to the test node: `request.param`, `request.node.name`, `request.addfinalizer`.
- Factory fixtures (returning a function) are how you parameterise object creation inside a test.
- ⚠️ Mutable session-scoped fixtures leak state between tests and produce order-dependent failures. Run with `pytest-randomly` to expose them.

Parametrisation:

```
@pytest.mark.parametrize(
    ("value", "expected"),
```

```
[(0, "zero"), (1, "one"), pytest.param(-1, "?", marks=pytest.mark.xfail)],
ids=["zero", "one", "negative"],
)
def test_describe(value, expected):
    assert describe(value) == expected
```

Stack multiple `parametrize` decorators for a cartesian product. Parametrise *fixtures* (`@pytest.fixture(params=[...])`) to run an entire suite against several backends.

Assertion rewriting is why bare `assert` gives rich output: pytest rewrites the AST of test modules at import. It only applies to test modules and to plugins registered via `pytest.register_assert_rewrite` — put shared assertion helpers in a plugin or register them, or you lose the diffs.

Hooks and plugins — `pytest_collection_modifyitems` (reorder/skip), `pytest_runtest_makereport` (custom reporting), `pytest_addoption` (CLI flags). A `conftest.py` is already a plugin; graduate it to a package when several repos need it.

Essential plugins: `pytest-cov`, `pytest-xdist` (parallel; `-n auto`), `pytest-randomly`, `pytest-timeout`, `pytest-asyncio` or `anyio`, `pytest-benchmark`, `syrupy` (snapshots), `pytest-mock`.

Configuration worth having:

```
[tool.pytest.ini_options]
addopts = "-ra --strict-markers --strict-config"
testpaths = ["tests"]
filterwarnings = ["error", "ignore::DeprecationWarning:somedep.*"]
```

`filterwarnings = ["error"]` turns warnings into failures — the cheapest way to stay ahead of deprecations.

10.2 Property-based testing with Hypothesis

Example-based tests check the cases you thought of. Property-based tests search for the ones you did not, then **shrink** the failure to a minimal reproducer.

```
from hypothesis import given, strategies as st, settings, assume

@given(st.lists(st.integers()))
def test_sort_idempotent(xs):
    assert sorted(sorted(xs)) == sorted(xs)

@given(st.text())
def test_roundtrip(s):
    assert decode(encode(s)) == s
```

Properties that generalise across domains: **round-trip** (encode/decode, serialise/parse), **invariance** (sorting preserves multiset), **idempotence**, **oracle** (agrees with a slow reference implementation), **metamorphic**

(relationship between two runs), and **never crashes** on any valid input.

Composite strategies build domain objects; `st.builds(MyModel, ...)` and `st.from_type` integrate with annotations. `assume(...)` discards uninteresting inputs (too many discards is a signal your strategy is wrong). `@settings(deadline=None)` for slow tests; `@example(...)` pins a known-tricky case permanently.

Stateful testing (`RuleBasedStateMachine`) generates sequences of operations against a model and checks invariants after each — the right tool for caches, state machines, connection pools, and anything with an ordering bug hiding in it.

10.3 Mocking — and mocking less

The order of preference:

1. **Real object** — fastest to trust.
2. **Fake** — an in-memory implementation of the same interface (a dict-backed repository). Best ratio of realism to speed.
3. **Stub/spy** — a Protocol implementation you wrote by hand.
4. `unittest.mock` — last resort.

Rules for when you do use `mock`:

- **Patch where it is used, not where it is defined:** `mock.patch("myapp.service.requests.get")`, not `"requests.get"`. The `from x import y` in the module under test made a second reference.
- **Always `autospec=True`** (or `create_autospec`). Without it, a mock happily accepts calls with wrong arguments and returns another mock, so your test passes after you rename the function.
- Assert on behaviour, not call counts, wherever possible. `assert_called_once_with` is coupling to implementation.
- ⚠ `assert mock.called_once_with(...)` — a typo'd attribute on a `Mock` returns a truthy `Mock`, so the assertion always passes. `MagicMock` with `autospec`, or `mock.assert_called_once_with`, avoids this. This bug has shipped more broken code than any other testing mistake.

Time and randomness are dependencies too: inject a clock, or use `freezegun/time-machine`; seed RNGs explicitly.

The deeper point: heavy mocking is a design smell. If a unit needs six patches, it has six hard-wired dependencies; inject them and the mocks disappear.

10.4 Coverage, mutation, and what they mean

```
[tool.coverage.run]
branch = true
source = ["src"]
parallel = true
[tool.coverage.report]
fail_under = 85
exclude_also = ["if TYPE_CHECKING:", "raise NotImplementedError", "\\\\.\\.\\.\\."]
```

Branch coverage (not just line) is the minimum useful metric. But coverage measures *execution*, not *verification*: a test suite with no assertions can hit 100%. Use it to find untested regions, never as a quality score. [3.12+] coverage.py uses `sys.monitoring`, so the overhead is far lower than it used to be.

Mutation testing (`mutmut`, `cosmic-ray`) closes that gap: it introduces small changes (`< → <=`, `+ → -`, remove a call) and reports mutants your suite failed to kill. Slow, so run it on the core domain logic only, on a schedule rather than every PR. A surviving mutant is a concrete missing assertion.

Other layers worth naming: **contract tests** (Pact) for service boundaries; **snapshot tests** for large stable outputs (review diffs carefully, they rot); **golden/characterisation tests** when refactoring legacy code you do not understand yet; **fuzzing** (`atheris`) for parsers.

10.5 Matrices and CI

`tox` and `nox` run the suite across interpreters and dependency sets in isolated environments. `nox` uses Python for configuration, which is easier once the matrix gets conditional.

```
# noxfile.py
import nox

@nox.session(python=["3.11", "3.12", "3.13", "3.14", "3.14t"])
def tests(session):
    session.install("-e", ".[dev]")
    session.run("pytest", "-n", "auto", *session.posargs)

@nox.session
def typecheck(session):
    session.install("-e", ".[dev]")
    session.run("mypy", "src")
```

A sane CI gate: ruff (lint + format check), mypy strict, pytest with coverage across the supported matrix, `pip-audit`, and a build of sdist+wheel that gets installed and imported in a clean environment. Add the free-threaded build to the matrix early if you ship anything with a C dependency.

10.6 Exercises for Parts IX–X

1. Convert a `setup.py` project to `pyproject.toml` with hatchling, ship `py.typed`, and verify types resolve for a consumer.
2. Build a plugin system with entry points; write a second distribution that registers into it without either package importing the other.
3. Write a Hypothesis stateful test for an LRU cache. Find a real bug (there is usually an eviction-order one).
4. Take a heavily-mocked test module and rewrite it with a fake. Compare line count and how each responds to a refactor.
5. Run mutation testing on your most important module and fix every surviving mutant.

Part XI — Correctness-Critical Standard Library

These are the modules where "it works on my machine" turns into a financial or legal incident.

11.1 Numbers

Floats

`float` is IEEE-754 binary64. `0.1` is not 0.1; it is the nearest representable double. Therefore:

```
0.1 + 0.2 == 0.3      # False
0.1 + 0.2              # 0.30000000000000004
```

Rules:

- **Never use floats for money.** Ever. Use integer minor units (cents) or `Decimal`.
- Compare with `math.isclose(a, b, rel_tol=1e-9, abs_tol=0.0)`, choosing `abs_tol` explicitly when values may be near zero.
- Summation loses precision catastrophically over many terms; `math.fsum` is exactly rounded. `statistics.fmean` likewise.
- Watch for catastrophic cancellation: subtracting two nearly-equal large numbers destroys significant digits. Rearrange the algebra.
- `float("nan") != float("nan")`, but `[nan] == [nan]` is `True` because list comparison checks identity first. `nan` in a sorted list or a dict key breaks invariants; reject it at the boundary.
- `round()` uses banker's rounding (ties to even): `round(2.5) == 2`. This is correct for statistics and wrong for invoices.
- `repr` round-trips exactly `[3.1+]`; `float(repr(x)) == x` always.

Decimal

```
from decimal import Decimal, getcontext, ROUND_HALF_UP, localcontext

Decimal("0.1") + Decimal("0.2") == Decimal("0.3")      # True

with localcontext() as ctx:
    ctx.prec = 28
    ctx.rounding = ROUND_HALF_UP
    total = (price * qty).quantize(Decimal("0.01"))
```

- **Always construct from strings or ints**, never floats — `Decimal(0.1)` faithfully preserves the binary error.
- Context is thread-local and affects arithmetic; use `localcontext()` rather than mutating the global context in library code.
- `quantize` at the boundary (display, storage) with an explicit rounding mode agreed with your finance team. `ROUND_HALF_EVEN` is the default; most jurisdictions want `ROUND_HALF_UP`.
- `Decimal` preserves significance: `Decimal("1.10") != Decimal("1.1")` under `compare_total` but `==` is numeric. Know which you are testing.
- `Fraction` for exact rational arithmetic (unit conversions, tax splits); slower but never lies.

11.2 datetime

The rules that prevent 90% of date bugs:

1. **Store and compute in UTC, aware.** Convert to local only for display.
2. **Never use `datetime.utcnow()`** — it returns a *naive* datetime that lies about being local. Use `datetime.now(timezone.utc)`.
3. **Aware vs naive never mix.** Comparing them raises `TypeError`; arithmetic between naive datetimes silently ignores DST.
4. **Use `zoneinfo`** [3.9+, IANA database] for real time zones. Fixed offsets (`timezone(timedelta(hours=-5))`) are not time zones — they cannot represent DST.

```
from datetime import datetime, timezone, timedelta
from zoneinfo import ZoneInfo

now = datetime.now(timezone.utc)
local = now.astimezone(ZoneInfo("America/New_York"))
```

DST arithmetic is where the subtlety lives. Adding a `timedelta` to an aware datetime does *absolute* time arithmetic in some cases and wall-clock in others:

```
ny = ZoneInfo("America/New_York")
d = datetime(2025, 3, 8, 12, 0, tzinfo=ny)
d + timedelta(days=1)           # wall clock: 2025-03-09 12:00 EDT (23 real
                                # hours later)
(d.astimezone(timezone.utc) + timedelta(days=1)).astimezone(ny)  # 24 real hours
                                # later
```

Decide which you mean: "same time tomorrow" (wall clock — calendars, alarms) or "24 hours from now" (absolute — timeouts, SLAs). For wall-clock arithmetic across a DST boundary, normalise afterwards with `.astimezone(tz)` to resolve the offset.

PEP 495 `fold` disambiguates the repeated hour when clocks go back: `fold=0` is the first (DST) occurrence, `fold=1` the second (standard). During the *skipped* hour in spring, the datetime does not exist at all — `ZoneInfo` will give you something, but your business logic must decide what a 2:30 AM appointment means on that date.

Other essentials: `datetime.fromisoformat` / `isoformat` [3.11 parses the full ISO 8601 profile including `Z`]; `date` vs `datetime` (a birthday is a `date`; conflating them creates timezone bugs out of nothing); monotonic durations use `time.monotonic()`, never wall clock, because NTP can step backwards; `time.perf_counter()` for benchmarks.

11.3 Files and the filesystem

```
from pathlib import Path
p = Path("/var/data") / "reports" / f"{name}.json"
```

```
p.parent.mkdir(parents=True, exist_ok=True)
p.write_text(json.dumps(payload), encoding="utf-8")
```

Always pass `encoding=`. The default is locale-dependent, which means your code behaves differently on a Windows CI runner. [3.15 direction / 3.10+] `encoding="locale"` and `PYTHONWARNDEFAULTENCODING=1` help you find the omissions; UTF-8 mode (`-X utf8`) forces sanity.

Atomic writes. A plain `write_text` on a file that another process reads can be observed half-written; a crash mid-write destroys the old content. The correct pattern:

```
import os, tempfile
def atomic_write(path: Path, data: str) -> None:
    fd, tmp = tempfile.mkstemp(dir=path.parent) # same filesystem – rename
    try:                                       # across devices is not atomic
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            f.write(data)
            f.flush()
            os.fsync(f.fileno())              # data hits the disk
        os.replace(tmp, path)                 # atomic on POSIX and Windows
    except BaseException:
        os.unlink(tmp)
        raise
```

`os.replace` is atomic; `os.rename` differs on Windows when the target exists. For full durability you must also `fsync` the *directory* after the rename on POSIX.

Other traps: file locking is not portable (`fcntl.flock` vs `msvcrt.locking`, and neither works reliably over NFS); `os.path.exists` before opening is a TOCTOU race — just open it and handle the exception; case-insensitive filesystems on macOS and Windows make `a.txt` and `A.txt` the same file; path length limits on Windows; `Path.resolve()` follows symlinks and is your defence against traversal (Part XII).

11.4 Logging

The mental model: **loggers** (hierarchical, named by module) → **handlers** (destinations) → **formatters** → **filters** (on both loggers and handlers). Records propagate up the dotted hierarchy to ancestors' handlers.

```
import logging
log = logging.getLogger(__name__) # module-level, hierarchical, correct
```

Rules for libraries vs applications:

- **Libraries** get a logger and never configure handlers or levels. Add a `NullHandler` to your top package to suppress "no handler" warnings. Configuring logging from a library steals the application's decisions.
- **Applications** configure once at startup, ideally with `dictConfig`.

```
logging.config.dictConfig({
    "version": 1,
    "disable_existing_loggers": False,      # almost always what you want
    "formatters": {"json": {"()": "pythonjsonlogger.jsonlogger.JsonFormatter"}},
    "handlers": {"stdout": {"class": "logging.StreamHandler", "formatter":
"json"}},
    "root": {"level": "INFO", "handlers": ["stdout"]},
})
```

Performance and correctness details:

- **Use %s lazy formatting:** `log.info("user %s failed %d times", user, n)`. An f-string is formatted even when the level is disabled — measurable in hot paths, and it defeats log aggregation by producing unique message templates.
- `log.exception(...)` inside an `except` block includes the traceback; `log.error(..., exc_info=True)` elsewhere.
- `extra={...}` attaches structured fields; a `LoggerAdapter` or a `Filter` injects context (request id from a `ContextVar`) into every record.
- ⚠ Logging from multiple **processes** to one file corrupts it. Use `QueueHandler` + `QueueListener` (one writer), or log to stdout and let the supervisor handle aggregation — the container-native answer.
- `QueueHandler` is also the fix for logging from an asyncio loop: file and network handlers block, and a blocking handler stalls every task.
- Never log secrets, tokens, full request bodies, or PII. Add a `Filter` that redacts, and test it.

11.5 Serialisation

pickle serialises almost any Python object, including code references. Protocol 5 [3.8] adds out-of-band buffers for zero-copy of large arrays.

⚠ **Unpickling untrusted data is arbitrary code execution.** `__reduce__` names a callable and its arguments; that callable can be `os.system`. There is no safe subset, no sandbox, no validation that helps. Treat a pickle file exactly like an executable. Use it only for trusted, same-version, internal data (multiprocessing does, which is why worker inputs must be trusted).

Portability: pickles are not stable across Python versions or across your own class refactors — moving or renaming a class breaks every stored pickle. Never use it as a storage or wire format.

Alternatives by need: **JSON** (universal, but no int/float distinction beyond IEEE, no bytes, no dates — and `json` accepts `NaN/Infinity` by default which is invalid JSON; pass `allow_nan=False`); **orjson/msgspec** (much faster, native dataclass and datetime support); **msgpack** or **CBOR** (compact binary); **Protobuf/Avro** (schema evolution, cross-language); **Parquet/Arrow** (columnar analytics).

`marshal` is CPython's internal format for `.pyc` files — undocumented, version-specific, not for you.

11.6 CLI, config and subprocess

`argparse` is stdlib and sufficient: subparsers, `type=`, `choices=`, mutually exclusive groups. `click` for composable decorator-based CLIs, `typer` for annotation-driven ones. Whatever you choose, respect the

conventions: `--help`, non-zero exit codes on failure, errors to stderr, machine-readable output behind a `--json` flag, and no colour when not a TTY.

`tomllib` [3.11] reads TOML from the stdlib (read-only; use `tomli-w` or `tomlkit` to write). Config precedence that users expect: defaults < config file < environment variables < CLI flags. `ChainMap` models this in three lines.

`subprocess` — one rule dominates: **never `shell=True` with user input.**

```
import subprocess
res = subprocess.run(
    ["git", "log", "--oneline", "-n", str(n)],
    capture_output=True, text=True, timeout=30, check=True,
)
```

`check=True` raises on non-zero; `timeout=` prevents an inherited hang; a list argv means no shell metacharacter parsing. If you must build a shell string, `shlex.quote` every component — and reconsider.

11.7 Exercises for Part XI

1. Compute a 15% tax on 1,000,000 line items with `float` and with `Decimal`. Show the divergence and quantify it in currency.
2. Schedule a recurring 2:30 AM job across a spring-forward date. Decide and implement what should happen; write the test.
3. Implement `atomic_write` and prove durability by killing the process mid-write in a loop.
4. Configure logging so that a request id set in a middleware appears in every log line emitted by any library, without touching those libraries.
5. Craft a pickle payload that executes a command (in a container, on purpose) so you never again treat pickle as data.

Part XII — Security

12.1 The trust boundary

Draw it explicitly: everything crossing it — HTTP requests, queue messages, uploaded files, config from a shared store, third-party API responses, filenames — is hostile until validated. Validation means parsing into a typed model that rejects anything unexpected (`extra="forbid"`), not `isinstance` checks.

12.2 Deserialisation and code execution

- `pickle` / `shelve` / `dill` / `joblib.load` / `torch.load` (historically) / `numpy.load(allow_pickle=True)` — all arbitrary code execution on untrusted input. `yaml.load` without a loader is the same; use `yaml.safe_load`.
- `eval` / `exec` **cannot be sandboxed**. Stripping `__builtins__` is defeated in one line by walking `().__class__.__mro__[1].__subclasses__()` to reach an object that can open files or spawn processes. Do not try. If you need user expressions, use a restricted evaluator over a whitelisted AST

(`ast.parse` + a `NodeVisitor` that rejects everything except literals, names in a known set, and a fixed operator list), or embed a real sandboxed language.

- **Format strings:** `"{0.__class__.__init__.__globals__[SECRET]}".format(obj)` leaks module globals. Never call `.format()` on a user-supplied template. f-strings are compile-time and safe; `str.format` with user templates is not. [3.14, PEP 750] **t-strings** give you the template pieces as data (`Template` with `Interpolation` values) so libraries can escape correctly before any string is built — the structural fix for injection in templating.
- **XML:** entity expansion (billion laughs), external entities (XXE), and quadratic blowup. Use `defusedxml`, or a parser with entity resolution disabled.
- **Archives:** zip/tar bombs (compression ratio limits) and path traversal via `../` or absolute paths in member names. [3.12] `tarfile.extractall(filter="data")` is the safe filter and became the default in later versions; for zip, validate each resolved member path stays inside the destination.

```
dest = Path(target).resolve()
for member in zf.namelist():
    out = (dest / member).resolve()
    if not out.is_relative_to(dest):          # [3.9]
        raise SecurityError(f"path traversal: {member}")
```

12.3 Injection

- **SQL:** parameterised queries only. `cursor.execute("SELECT * FROM t WHERE id = ?", (id,))`. String formatting into SQL is indefensible; ORMs help but `text()` escapes the protection. Identifiers (table/column names) cannot be parameterised — whitelist them against a known set. `LiteralString` (Part II) lets a type checker enforce that only literal-derived strings reach your query function.
- **Shell:** argv lists, never `shell=True`.
- **Path traversal:** resolve and containment-check, as above. Never trust an uploaded filename; generate your own.
- **SSRF:** if users supply URLs, resolve the hostname and reject private ranges (`ipaddress.ip_address(...).is_private`), disallow redirects to them, and prefer an allow-list. Note DNS rebinding — resolve once and connect to the resolved IP.
- **Regex (ReDoS):** nested quantifiers on user input cause exponential backtracking. Keep patterns simple, bound input length, or use `re2`.

12.4 Cryptography

Rules, without exception:

1. **Do not implement cryptography.** Use `cryptography` (or `PyNaCl`). `hashlib/hmac` for primitives.
2. **Passwords:** `argon2-cffi` or `bcrypt`, or `hashlib.scrypt/pbkdf2_hmac` with a high iteration count. Never a bare SHA-256, never MD5, always per-password salt (these libraries handle it).
3. **Randomness:** `secrets` for anything security-relevant (`token_urlsafe`, `token_hex`, `choice`). `random` is a Mersenne Twister — predictable from ~624 outputs — and is fine only for simulations.
4. **Comparison:** `hmac.compare_digest` for tokens, signatures and MACs. `==` on secrets leaks length and content through timing.

5. **Encryption**: authenticated modes only (AES-GCM, ChaCha20-Poly1305, or **Fernet** if you want no choices). Never ECB. Never reuse a nonce.
6. **TLS**: verify certificates. **verify=False** in **requests/httpx** is a vulnerability, not a workaround; fix the CA bundle instead.

```
import secrets, hmac
token = secrets.token_urlsafe(32)
if not hmac.compare_digest(provided, expected):
    raise AuthError
```

12.5 Secrets and configuration

Environment variables or a secret manager — never source control, never a logged config dump. Exclude them from **repr** (**field(repr=False)**, Pydantic **SecretStr**) so they cannot leak into a traceback or an error report. Rotate. Scope credentials narrowly. Scan the repository history (**gitleaks**, **trufflehog**) — a secret committed and then deleted is still a secret in the history.

⚠ Tracebacks and debug pages are a leak vector: a framework's debug mode renders local variables, including your database password. Never enable it in production; configure exception reporters to redact.

12.6 Dependencies and runtime hardening

Covered in Part IX: hash-pinning, trusted publishing, **pip-audit** in CI, dependency review. Additionally:

- Run with least privilege; do not run as root in containers.
- **sys.addaudithook** (PEP 578) for a runtime allow-list or forensic trail on **exec**, **open**, **socket.connect**, **subprocess**.
- Set resource limits (**resource.setrlimit**) around untrusted work, and prefer a separate process (or container) for anything you cannot fully validate.
- Keep the interpreter patched; CPython security releases matter, and old versions leave support silently.

12.7 A pre-deploy checklist

- ☐ Every external input parsed into a strict model, unknown fields rejected
- ☐ No **pickle/yaml.load/eval** on anything crossing the boundary
- ☐ All SQL parameterised; identifiers whitelisted
- ☐ No **shell=True** with interpolated data
- ☐ Archive extraction path-checked and size-bounded
- ☐ Passwords hashed with argon2/bcrypt/scrypt; tokens from **secrets**; comparisons constant-time
- ☐ TLS verification on everywhere
- ☐ Secrets outside the repo, redacted in logs and **repr**
- ☐ Debug mode off; tracebacks not user-visible
- ☐ Dependencies pinned with hashes, audited in CI
- ☐ Logging free of PII and credentials

Part XIII — Domain-Specific Languages and Code Generation

Python is unusually good at hosting little languages. Operator overloading, decorators, context managers, descriptors, `__getattr__`, pattern matching, and full runtime access to the AST mean you can build an API that reads like a language of its own — or an actual parser for one.

13.1 Internal vs external DSLs

An **internal** (embedded) DSL is Python that reads like something else: SQLAlchemy's query API, pytest's fixtures, Django's model fields, `attrs`' declarations. An **external** DSL has its own syntax and needs a parser: a query language, a config format, a rules language for non-programmers.

Choose internal unless you need one of three things: non-programmers must write it, it must be storable and transmittable as data, or it must be restricted so it cannot do arbitrary things. If you need an external DSL for the third reason, remember Part XII: parsing is your security boundary.

13.2 Building an internal DSL

The toolkit, and what each gives you:

Operator overloading for expression trees. Build structure instead of computing:

```
class Expr:
    def __and__(self, other): return And(self, other)
    def __or__(self, other):  return Or(self, other)
    def __invert__(self):     return Not(self)

class Field(Expr):
    def __init__(self, name): self.name = name
    def __eq__(self, value):  return Cmp(self.name, "=", value)
    def __gt__(self, value):  return Cmp(self.name, ">", value)

query = (Field("age") > 21) & (Field("country") == "SE")
```

⚠ Overloading `__eq__` to return a node rather than a bool makes the class unhashable and breaks `in`, `==` in tests, and dict keys. Every ORM makes this trade deliberately; make it deliberately too, and document it.

Fluent interfaces with `Self` for chaining:

```
class Query:
    def where(self, cond) -> Self: ...
    def order_by(self, *fields) -> Self: ...
    def limit(self, n: int) -> Self: ...

Query(User).where(age > 21).order_by("-created").limit(10)
```

Return new instances rather than mutating, so partially-built queries can be shared safely.

Descriptors and `__set_name__` for declarative class bodies (Part I). This is how every ORM field, every dataclass, and every schema library works.

`__getattr__` for open-ended namespaces — but only where the vocabulary is genuinely unbounded, because it defeats autocompletion and typo detection.

Context managers for scoping: `with transaction():`, `with figure():`, `with tag("div"):`.

Decorators as registration syntax: `@app.route("/x")`, `@task(retries=3)`, `@rule(priority=10)`.

`match` [3.10] for interpreting the tree you built — structural patterns map cleanly onto AST-like node classes.

Design rules: keep the vocabulary small; produce good error messages in the DSL's own terms, not Python's; make the object graph inspectable (a `__repr__` that shows the built structure); and make sure a type checker can still see through it, or your users lose their tooling.

13.3 External DSLs: parsing

Three levels of effort:

- **re + a hand-written recursive-descent parser** — right for small grammars. Tokenise, then one function per grammar rule.
- **A parser library** — `lark` (EBNF, several algorithms), `parsimonious` (PEG), `pyparsing` (combinators). `lark` is usually the best default.
- **Reuse Python's own parser** — `ast.parse` a restricted subset, then walk it with a `NodeVisitor` that rejects everything outside a whitelist.

The safe-expression pattern, which you will need more than once:

```
import ast, operator

ALLOWED_NODES = (ast.Expression, ast.BinOp, ast.UnaryOp, ast.Constant,
                  ast.Name, ast.Load, ast.Compare, ast.BoolOp)
OPS = {ast.Add: operator.add, ast.Sub: operator.sub,
       ast.Mult: operator.mul, ast.Div: operator.truediv}

def safe_eval(source: str, names: dict[str, float]) -> float:
    tree = ast.parse(source, mode="eval")
    for node in ast.walk(tree):
        if not isinstance(node, ALLOWED_NODES):
            raise ValueError(f"forbidden syntax: {type(node).__name__}")
    return _eval(tree.body, names)
```

⚠️ Whitelist node types; never blacklist. And never fall back to `eval()` "just for the hard cases" — Part XII explains why there is no safe subset of `eval`.

13.4 Code generation

Generating and `execing` source is a legitimate, widely used technique: `dataclasses`, `attrs`, `namedtuple`, and `Pydantic` all build `__init__` as text because a generated function with fixed parameters is far faster to *call* than a generic loop over field descriptors.

```
def make_init(fields: list[str]):
    args = ", ".join(fields)
    body = "\n".join(f"    self.{f} = {f}" for f in fields)
    src = f"def __init__(self, {args}): \n{body} \n"
    ns: dict = {}
    exec(compile(src, "<generated>", "exec"), ns)
    return ns["__init__"]
```

Rules for generated code: pass a real `filename` to `compile` so tracebacks are readable; never interpolate untrusted input into the source; keep the generated text retrievable for debugging (`attrs` and `dataclasses` can show you theirs); and prefer generating *once at class creation* rather than per call.

Alternatives to text generation: build an AST directly and `compile` it (safer for structured cases, harder to read); or `functools.partial` and closures when the shape is fixed.

13.5 Import hooks as language extension

Part V covered the import system's finders and loaders. A meta-path finder that compiles a non-Python source file into a module lets you `import` your DSL directly:

```
class DslFinder:
    def find_spec(self, fullname, path, target=None):
        location = locate_dsl_file(fullname, path)
        if location is None:
            return None
        return importlib.util.spec_from_file_location(
            fullname, location, loader=DslLoader(location))

class DslLoader(importlib.abc.Loader):
    def create_module(self, spec): return None
    def exec_module(self, module):
        source = compile_dsl_to_python(Path(module.__spec__.origin).read_text())
        exec(compile(source, module.__spec__.origin, "exec"), module.__dict__)

sys.meta_path.insert(0, DslFinder())
```

This is how tools like Cython's pure-Python mode, notebook loaders, and various typed-template systems work. [△](#) It also makes your codebase unreadable to anyone who has not read this chapter — document it loudly, and prefer an explicit build step for anything a stranger will maintain.

13.6 Exercises

1. Build a query DSL producing parameterised SQL — never string-interpolated — with `__and__`, `__or__`, and comparison overloads.

2. Write a `lark` grammar for a small rules language, then an evaluator over its tree using `match`.
3. Implement `namedtuple`-style code generation and compare call performance with a `__getattr__`-based version.
4. Write an import hook that loads `.rules` files as modules, with correct tracebacks pointing at the original file and line.

Part XIV — Advanced Async: Custom Loops, Backpressure, Structured Concurrency

Part III covered `asyncio`'s model. This part covers what you need when you are building the infrastructure rather than using it.

14.1 The loop, from the inside

An event loop is three things: a **selector** (`epoll`/`kqueue`/IOCP via the `selectors` module), a **ready queue** of callbacks, and a **timer heap**. One iteration:

1. Compute the timeout: zero if callbacks are ready, else the time until the next timer, else block indefinitely.
2. Poll the selector for I/O readiness; schedule the corresponding callbacks.
3. Move expired timers into the ready queue.
4. Run every callback currently in the ready queue (a bounded snapshot, so newly-scheduled callbacks wait for the next turn — this is what prevents starvation).

`loop.call_soon`, `call_later`, `call_at`, `add_reader`, `add_writer` are the primitive surface. Tasks are built on top: a `Task` steps its coroutine, and each `await` on a pending future registers a callback that steps it again.

Implications you can act on:

- **A callback that blocks blocks everything**, including timers. This is why `debug=True` warns about slow callbacks — set `loop.slow_callback_duration` and log them in production.
- Timer resolution is bounded by loop-iteration time under load, so `asyncio.sleep(0.001)` is a suggestion, not a guarantee.
- `await asyncio.sleep(0)` yields one turn to the loop — the correct way to cooperate inside a long computation you cannot move off-loop.

Alternative and custom loops

`uvloop` (libuv) is a drop-in replacement that is substantially faster for socket-heavy work:

`asyncio.run(main(), loop_factory=uvloop.new_event_loop)`. Writing a *whole* loop from scratch is rarely justified; writing a **custom event loop policy**, a **custom task factory**, or a **loop subclass** for instrumentation is common and useful.

```
class InstrumentedLoop(asyncio.SelectorEventLoop):
    def call_soon(self, callback, *args, context=None):
        METRICS["callbacks"] += 1
        return super().call_soon(callback, *args, context=context)
```

[3.12+] `asyncio.eager_task_factory` starts a coroutine synchronously and only creates a `Task` if it actually suspends — a real win when most calls hit a cache and return without awaiting anything.

Implementing `__await__` directly is how you integrate a non-asyncio primitive:

```
class Signal:
    def __init__(self): self._waiters = []
    def __await__(self):
        fut = asyncio.get_running_loop().create_future()
        self._waiters.append(fut)
        return fut.__await__()          # delegate; yields until the future
    resolves
    def fire(self, value):
        for fut in self._waiters:
            if not fut.done():
                fut.set_result(value)
        self._waiters.clear()
```

14.2 Backpressure

Backpressure is the property that a slow consumer slows the producer instead of accumulating work. Without it, every system fails the same way: unbounded memory growth, then an OOM kill, at the worst possible moment.

Where it is automatic: a pull-based generator pipeline (Part VI) — the consumer sets the pace by definition.

Where you must build it:

```
queue = asyncio.Queue(maxsize=1000)          # NEVER leave maxsize unbounded
await queue.put(item)                        # blocks the producer when full – good
```

An unbounded `asyncio.Queue` is not a queue, it is a memory leak with a friendly API.

Other mechanisms:

- **Semaphores** cap in-flight work against a dependency: `async with sem:` around each request.
- **Connection pool limits** (`httpx.Limits(max_connections=50)`) are backpressure at the transport layer.
- **Streaming protocol flow control:** asyncio's `StreamWriter.drain()` exists precisely so you await when the write buffer is full. Ignoring `drain()` is the most common way to leak memory in a TCP server.
- `asyncio.Protocol`'s `pause_writing/resume_writing` are the low-level hooks; high-water and low-water marks configure them.

```

async def send_all(writer, chunks):
    for chunk in chunks:
        writer.write(chunk)
        await writer.drain()      # yields until the peer keeps up

```

Load shedding is the other half. When the queue is full and blocking is not acceptable (an HTTP handler cannot make the client wait forever), reject fast:

```

try:
    queue.put_nowait(item)
except asyncio.QueueFull:
    METRICS["shed"] += 1
    raise HTTPException(503, "overloaded")

```

Design rule: **every buffer in the system must be bounded, and every bound must have a defined behaviour when reached** — block, shed, or spill to disk. "It won't fill up" is not a behaviour.

14.3 Structured concurrency in depth

The principle: **a task must not outlive the scope that created it**. Guarantees that follow: no leaked tasks, errors always propagate somewhere, cancellation is complete, and shutdown is deterministic.

TaskGroup [3.11] implements it. On exit it awaits children; on any child failure it cancels the siblings and raises an **ExceptionGroup**. **anyio** and **trio** call the same thing a nursery and extend it with cancel scopes:

```

import anyio

async def main():
    async with anyio.create_task_group() as tg:
        tg.start_soon(worker, 1)
        tg.start_soon(worker, 2)
        with anyio.move_on_after(5):      # cancel scope, composable
            await slow_thing()

```

anyio is the right dependency for a library that must work on both **asyncio** and **trio** — you write once and your users choose.

Cancellation, rigorously

- **CancelledError** derives from **BaseException** so **except Exception** cannot swallow it. Never catch it without re-raising.
- Cleanup in **finally** that awaits can be interrupted by a *second* cancellation. Wrap essential cleanup in **asyncio.shield()** plus a short **asyncio.timeout()**.
- [3.11] **Task.uncancel()** and the cancellation counter let nested scopes distinguish "my timeout fired" from "my caller cancelled me" — this is what makes **asyncio.timeout()** composable where **wait_for** was not.

- Cancellation is **cooperative**: a task in a tight CPU loop with no `await` cannot be cancelled. Yield with `await asyncio.sleep(0)` or move the work to a thread or process.

Graceful shutdown

```
async def main():
    stop = asyncio.Event()
    loop = asyncio.get_running_loop()
    for sig in (signal.SIGINT, signal.SIGTERM):
        loop.add_signal_handler(sig, stop.set)

    async with asyncio.TaskGroup() as tg:
        tg.create_task(serve(stop))
        tg.create_task(consume(stop))
        await stop.wait()
    # TaskGroup exit awaits children; they observe `stop` and finish in-flight
    work
```

The sequence that matters in production: stop accepting new work → drain in-flight work with a deadline → flush buffers and metrics → close connections → exit. A hard timeout around the whole thing prevents a hung dependency from blocking the shutdown forever.

14.4 Exercises

1. Build a bounded pipeline: N producers, a bounded queue, M consumers, with metrics on queue depth. Slow one consumer and show producers slowing rather than memory growing.
2. Implement an awaitable primitive (a rate limiter or a latch) via `__await__` and use it inside a TaskGroup.
3. Write a TCP echo server with `StreamWriter.drain()`, then remove the `drain()` and demonstrate the memory growth.
4. Implement graceful shutdown with a 30-second drain deadline; prove no request is lost and no task hangs.
5. Benchmark the default loop against `uvloop` on 10,000 concurrent connections.

Part XV — Distributed Systems: Task Queues, Brokers, Scaling Workers

At some point work must outlive the request that created it, survive a deploy, and run on a different machine. That is a distributed system, with all of the failure modes that implies.

15.1 When to reach for a queue

Good reasons: work that takes longer than a request should (video encoding, report generation, bulk email); work that must be retried across process restarts; work that must be rate-limited against a third party; scheduled and periodic jobs; fan-out to many consumers.

Bad reasons: hiding slow code that should be fast; avoiding a database index; "we might need to scale later." A queue adds a broker, a worker fleet, serialisation, idempotency requirements, and a whole new class of partial failure. Do not take that on before you need it.

Simpler options first: a database table as a queue (`SELECT ... FOR UPDATE SKIP LOCKED` is genuinely good and transactional with your data); `asyncio` background tasks for in-process work that may be lost; a cron job for scheduled work.

15.2 Brokers

	Model	Delivery	Good for	Watch out for
Redis	in-memory lists/streams	at-most / at-least once	fast, simple, small jobs	durability depends on config; memory-bound
RabbitMQ	AMQP exchanges + queues	at-least once, acks	routing, priorities, delayed messages	operational complexity, queue backlogs
SQS	managed queue	at-least once (FIFO available)	AWS-native, no ops	visibility timeouts, 256 KB limit
Kafka	partitioned log	at-least once, ordered per partition	event streams, replay, high throughput	not a task queue; consumer group rebalancing

The one distinction to hold onto: a **queue** deletes a message when it is acknowledged; a **log** keeps it and each consumer tracks its own offset. If you need replay, audit, or multiple independent consumers of the same events, you want a log.

15.3 Celery, in practice

```
# tasks.py
from celery import Celery

app = Celery("myapp", broker="redis://localhost:6379/0",
             backend="redis://localhost:6379/1")

app.conf.update(
    task_acks_late=True,                # ack AFTER completion – survives worker
    task_reject_on_worker_lost=True,   # death
    worker_prefetch_multiplier=1,      # fair dispatch for long tasks
    task_time_limit=600,                # hard kill
    task_soft_time_limit=540,           # raises SoftTimeLimitExceeded first
    task_serializer="json",            # ⚠ never pickle
    accept_content=["json"],
    result_expires=3600,
    broker_transport_options={"visibility_timeout": 3600},
)
```

```
@app.task(bind=True, max_retries=5, autoretry_for=(TransientError,),
          retry_backoff=True, retry_jitter=True, acks_late=True)
def process_upload(self, upload_id: str) -> None:
    upload = Upload.objects.get(id=upload_id)    # pass an ID, not the object
    ...
```

```
celery -A tasks worker --loglevel=INFO --concurrency=8
celery -A tasks beat      # periodic scheduler
celery -A tasks flower    # monitoring UI
```

Rules learned the hard way:

- **Pass IDs, not objects.** Arguments are serialised; a serialised model is stale the moment it is queued and huge if it has relations.
- **task_serializer="json", never pickle.** A broker that accepts pickle is remote code execution for anyone who can write to it (Part XII).
- **acks_late=True** for anything that must not be lost — combined with idempotency, since late acks mean a task can run twice.
- **worker_prefetch_multiplier=1** for long tasks, or one worker grabs a hundred jobs and the others idle.
- **Always set time limits.** An unbounded task holds a worker slot forever.
- **Separate queues by latency class** (-Q **fast,slow**) so a batch of hour-long jobs cannot starve interactive work.

Alternatives worth knowing: **RQ** (simpler, Redis-only, good for small systems), **Dramatiq** (cleaner design, saner defaults), **arq** (asyncio-native), **Temporal** (durable workflows with real state machines — the right tool once your jobs have multi-step, long-running logic), and **APScheduler** for in-process scheduling.

15.4 The distributed-systems rules

Idempotency is not optional

Every delivery guarantee that is achievable in practice is *at-least once*. Your task **will** run twice — after a network partition, a worker OOM, a broker redelivery, or a retry. Design for it:

```
@app.task(acks_late=True)
def charge_customer(order_id: str, idempotency_key: str) -> None:
    with transaction.atomic():
        if Charge.objects.filter(key=idempotency_key).exists():
            return # already done
        charge = payment_api.charge(order_id, idempotency_key=idempotency_key)
        Charge.objects.create(key=idempotency_key, external_id=charge.id)
```

Use a natural key where one exists, a stored key otherwise, and push the key through to any external API that supports one (Stripe and most payment providers do).

Exactly-once is a marketing term

You cannot have exactly-once delivery across a network. You can have at-least-once delivery plus idempotent processing, which is observationally equivalent and is what everyone who claims exactly-once is actually doing.

The dual-write problem

Writing to your database and publishing to a broker are two operations that cannot be made atomic. If the process dies between them, the system is inconsistent. The fix is the **transactional outbox**: write the event to an **outbox** table inside the same database transaction, and have a separate relay publish rows from it and mark them sent. Duplicate publishes are fine, because consumers are idempotent.

Retries and the failure budget

- Exponential backoff **with jitter**. Without jitter, every client retries at the same instant and you have built a synchronised thundering herd.
- Cap retries and route exhausted tasks to a **dead letter queue** with enough context to replay them.
- Distinguish **transient** (timeout, 503, deadlock) from **permanent** (validation error, 404) failures. Retrying a permanent failure five times wastes a worker and delays the alert.
- Use a **circuit breaker** for a failing dependency: after N failures, fail fast for a cooldown period instead of piling up.

Scaling workers

- Scale on **queue depth and oldest-message age**, not CPU. A backlog with idle CPU means you are I/O bound; more workers help. A backlog with saturated CPU means you need more machines or faster code.
- Concurrency model must match the workload: threads or `--pool=gevent` for I/O-bound tasks, processes (`--pool=prefork`) for CPU-bound.
- **Poison messages** — a task that crashes the worker every time — will consume your entire fleet on redelivery. Cap redeliveries and dead-letter them.
- Deploys: workers must handle SIGTERM by finishing the current task and refusing new ones, with a drain timeout shorter than the orchestrator's kill timeout.
- **Schema evolution**: during a rolling deploy, old workers consume new messages and vice versa. Add fields, never remove or repurpose them; version the payload; deploy the consumer before the producer.

Observability

You cannot debug a distributed system from logs alone. You need:

- **Correlation IDs** propagated from the originating request through every queue message and every log line (a `ContextVar` plus a logging filter, Part XI).
- **Distributed tracing** — OpenTelemetry, with context propagated in message headers.
- **The four metrics that matter for a queue**: depth, oldest-message age, processing rate, failure rate. Alert on oldest-message age; it is the only one that reliably means "users are affected."
- **Structured logs** (JSON) to stdout, aggregated centrally. Never log secrets or payloads.

15.5 Exercises

1. Build a Celery pipeline: upload → validate → process → notify, with retries, a dead letter queue, and idempotency keys. Kill workers mid-task and prove nothing is lost or duplicated in effect.
2. Implement a transactional outbox with a relay process; demonstrate consistency across a simulated crash between the write and the publish.
3. Implement a circuit breaker around a flaky dependency and show the difference in tail latency under failure.
4. Instrument the pipeline with OpenTelemetry and trace one request end to end across three services.
5. Load-test until the queue backs up; write the autoscaling rule you would actually deploy, and the alert threshold.

Part XVI — Contributing to CPython and Major Open Source

16.1 Why do it

The practical reasons: you will read more expert code in a month than in a year of your own project; you will learn how large systems are actually reviewed; and the feature you need will exist. The honest one: most of what you rely on is maintained by a small number of people, several of them unpaid, and the bug you found will not fix itself.

You do not need to be an expert first. The people who become experts are the people who started sending small patches.

16.2 Contributing to CPython

Setting up.

```
git clone https://github.com/python/cpython
cd cpython
./configure --with-pydebug --with-trace-refs # debug build: assertions on,
slower
make -j$(nproc)
./python -m test -j0 # the test suite
./python # your build's REPL
```

On Windows, `PCbuild\build.bat`. The Developer's Guide at devguide.python.org is the authoritative source and is genuinely good — read it before anything else.

Finding something to work on. The `easy` label on the issue tracker; failing tests on buildbots; documentation gaps (a real contribution, and the fastest way to learn the workflow); reproducing and triaging open issues, which maintainers value highly and which almost nobody volunteers for.

The workflow.

1. An issue on GitHub first, unless it is trivial. Discussion before code saves everyone time.
2. Branch, implement, and add tests in `Lib/test/`. **A patch without a test will not be merged.**

3. Add a news entry: `blurb add` creates the file in `Misc/NEWS.d/next/`.
4. Sign the Contributor Licence Agreement (once).
5. Open a PR referencing the issue. CI runs a large matrix; expect to iterate.
6. Wait, and be patient. Core developers are volunteers with a backlog. A polite ping after a couple of weeks is fine.

Where things live. `Lib/` pure-Python stdlib; `Modules/` C extension modules; `Objects/` the core types; `Python/` the compiler, eval loop, and runtime; `Include/` public headers; `Doc/` documentation (reStructuredText, Sphinx); `Tools/`; `Misc/NEWS.d/` the changelog.

Culture. Backwards compatibility is close to sacred — an argument that starts "this API is badly designed" needs a deprecation path, not a change. New language features go through the PEP process and Discourse discussion, not a PR. Performance claims need benchmarks (`pypperformance`). Reviews can be blunt; they are about the code.

16.3 Contributing to libraries

Often a better first step than CPython: faster feedback, smaller surface, more welcoming.

1. **Read `CONTRIBUTING.md`.** It exists so you do not have to guess.
2. **Reproduce the bug in the smallest possible example** — this alone is a valuable contribution and often reveals the fix.
3. **Match local conventions**, even where you disagree. Run their formatter and linter, not yours.
4. **One change per pull request.** A PR that fixes a bug and reformats forty files will not be merged.
5. **Write the test first**, so the maintainer can see it fail before your fix and pass after.
6. **Describe the *why*** in the PR body: the problem, the approach, the alternatives you rejected.
7. **Accept review feedback gracefully.** Maintainers carry the code forever; you carry it for a week.

Good on-ramps in every project: documentation, type annotations for an untyped module, tests for uncovered code paths, triaging old issues, and reproducing bug reports.

16.4 Maintaining your own

If you publish something people use, you have taken on obligations:

- **A README that shows the thing working in the first ten lines.**
- **A changelog**, and semantic versioning you actually honour (Volume III, Chapter 31).
- **CI on every supported Python version**, and a stated support policy — the common one is "the three most recent releases."
- **Issue templates** and a response, even if the response is "not planned."
- **A LICENSE file.** Without one, nobody may legally use your code.
- **A `SECURITY.md`** with a private disclosure route.
- **Deprecation over removal**, always.
- **Say no.** Every accepted feature is maintenance forever. "This is out of scope, but here is how to do it in your own code" is a complete and kind answer.

And plan for the day you stop caring: add co-maintainers early, document the release process, and if you must step away, say so publicly and hand over or archive. Silent abandonment is worse for users than an honest "unmaintained" banner.

16.5 Exercises

1. Build CPython from source with `--with-pydebug` and run the test suite.
2. Add a line to a CPython docstring or documentation page and submit the PR — go through the whole workflow once on something trivial.
3. Find an open issue in a library you use, reproduce it minimally, and post the reproduction.
4. Add type annotations to one untyped module of a library you depend on and submit it.
5. Take one of your own repositories and add everything from §16.4 that is missing.

Appendix A — Version Cheat Sheet

What arrived when, for the features this book depends on. Verify against release notes before relying on any single line.

3.8 — walrus `:=`, positional-only `/`, `TypedDict`, `Protocol`, `Final`, `Literal`, pickle protocol 5, `functools.cached_property`, `CancelledError` becomes `BaseException`.

3.9 — `dict` merge `|`, builtin generics (`list[int]`), `zoneinfo`, `str.removeprefix/removesuffix`, PEG parser, `Path.is_relative_to`.

3.10 — structural pattern matching (`match`), `X | Y` unions (PEP 604), `zip(strict=True)`, `dataclass(slots=, kw_only=)`, parameter specification variables, much better error messages, `itertools.pairwise`.

3.11 — big speed release (~1.25× on average), zero-cost exceptions, fine-grained error locations, `ExceptionGroup` + `except*`, `asyncio.TaskGroup`, `asyncio.timeout`, `Self`, `LiteralString`, `Required/NotRequired`, `tomllib`, `StrEnum`, `contextlib.chdir`, `fromisoformat` full ISO 8601.

3.12 — PEP 695 type parameter syntax and `type` aliases, PEP 669 `sys.monitoring`, PEP 684 per-interpreter GIL, PEP 688 buffer protocol for Python classes, PEP 701 f-string grammar, `@override`, `Unpack` for `**kwargs`, `itertools.batched`, perf trampolines, removal of `distutils`.

3.13 — experimental free-threaded build, experimental copy-and-patch JIT, new interactive REPL, PEP 696 `TypeVar` defaults, PEP 705 `ReadOnly` for `TypedDict`, PEP 742 `TypeIs`, PEP 702 `@deprecated`, PEP 594 "dead batteries" removed, improved `locals()` semantics.

3.14 — < cite index="4-1,6-1">free-threading officially supported (PEP 779), PEP 649/749 deferred annotation evaluation with `annotationlib`, PEP 750 template strings (t-strings), PEP 734 multiple interpreters in the stdlib, < cite index="1-1">a new `Zstd` compression module, an opt-in tail-call interpreter, and a zero-overhead external debugger interface with remote `pdb` attach, PEP 758 `except` without parentheses.

3.15 — in development at the time of writing; a stable ABI for free-threaded extensions (`abi3t`) is among the work in progress. Check the release notes.

Appendix B — Choosing a Track

After Volume II, the advanced topics you actually need diverge sharply. Pick one track, go deep, and come back for the others later. Everyone needs the **core** row regardless.

Core (all tracks). Chapters 24 (memory), 29 (performance), 30 (design/SOLID), 32 (CI and tooling), 33 (dataclasses and validation); Volume IV Parts VII (errors and API design), X (testing), XI (stdlib correctness), XII (security).

Web / backend

Stack. FastAPI (async-first, annotation-driven, OpenAPI for free) or Django (batteries included, admin, ORM, mature auth). Flask if you want minimal and synchronous. Postgres via SQLAlchemy 2.0 or the Django ORM; Alembic for migrations. [httpx](#) for outbound calls. Uvicorn/Gunicorn, Docker, Redis.

Book chapters that matter most. 28 (asyncio) and Volume IV Parts III and XIV — a blocking call in an async handler is the defining failure mode of this track. Part XV (queues) as soon as you have work that outlives a request. Part XII (security) is not optional: you are the trust boundary. Chapter 33 for request/response validation.

What actually goes wrong. N+1 queries; missing indexes; a synchronous driver inside an async endpoint; unbounded queues; no timeouts on outbound calls; sessions or connections shared unsafely across tasks; secrets in the image.

Learn in this order. One framework end to end (auth, CRUD, migrations, tests) → async correctness → deployment and observability → background jobs → caching and query tuning.

Data / ML

Stack. NumPy, pandas or **Polars** (faster, lazy, saner API — worth learning first if you are starting now), PyArrow/Parquet, DuckDB for local analytics, scikit-learn, PyTorch, JupyterLab or marimo. dbt or an orchestrator (Dagster, Airflow, Prefect) once pipelines are real.

Book chapters that matter most. 24 (memory — you will hit the ceiling), 29 (performance and vectorisation), Volume IV Part IV (buffers, strides, views vs copies, profiling), Part VI (streaming with generators), Part XI (floats, decimals, datetimes — silent numerical wrongness is this track's characteristic bug), Part III for parallelism.

What actually goes wrong. Views versus copies confusion; [SettingWithCopyWarning](#) ignored; loading what you should stream; floats where you needed decimals; naive datetimes; training/serving skew from two different preprocessing code paths; notebooks as production.

Learn in this order. One dataframe library deeply → vectorised thinking (no [iterrows](#)) → memory and dtypes → Parquet and columnar formats → pipelines and orchestration → reproducibility (seeds, environments, data versioning).

Automation / DevOps / scripting

Stack. [pathlib](#), [subprocess](#), [argparse](#) or Typer, [httpx](#), [paramiko](#)/Fabric, Ansible, [rich](#) for output, [schedule](#) or systemd timers, [pytest](#) with [tmp_path](#), Docker, the cloud SDK you actually use ([boto3](#), [google-](#)

cloud-*).

Book chapters that matter most. Volume I Chapters 10–11 and Volume II Chapter 20 are the daily tools. Part XI (filesystem semantics, atomic writes, logging) is the heart of this track. Part XII (never `shell=True`, path traversal, secrets). Chapter 27 for parallelising slow batch work. Chapter 31 so your tools are installable rather than pasted around.

What actually goes wrong. `shell=True` with interpolated input; no timeouts on subprocesses; non-atomic writes losing data on a crash; a script that half-succeeded and is not safe to re-run; exit codes ignored; credentials in the script.

Learn in this order. Robust CLIs (arguments, exit codes, `--dry-run`) → idempotent operations → error handling and logging → packaging so colleagues can install it → scheduling and monitoring → parallelism.

A note on picking. You are not choosing forever; you are choosing what to be good at first. The core skills — testing, design, error handling, security — transfer completely. The stack does not, and trying to learn three stacks at once is how people spend a year and end up junior at all of them.

Appendix C — Study Plans

Beginner: 8 weeks

Five hours a week. Type every example; do not copy-paste.

Week	Read	Build
1	Ch 1–2	Working setup, venv, a script that greets you by name
2	Ch 3–5	A unit converter and a tip calculator
3	Ch 6	FizzBuzz, a guessing game, a multiplication table
4	Ch 7	A contact book using nested dicts and lists
5	Ch 8	Refactor weeks 2–4 into functions with docstrings
6	Ch 9	Make every earlier program survive bad input
7	Ch 10	A CSV expense tracker that persists between runs
8	Ch 11	A command-line tool combining files, dates, and randomness

Done when you can write a 100-line program from a description, and when a traceback tells you where to look instead of making you anxious.

Intermediate: 12 weeks

Week	Read	Build
1	Ch 12–13	Rewrite old loops as comprehensions; a flexible <code>log()</code>

Week	Read	Build
2	Ch 14	Multi-key sorting and reporting over real data
3–4	Ch 15	Model a real domain with classes, properties, dunder
5	Ch 16	A streaming pipeline over a file too big for memory
6	Ch 17	<code>@timed</code> , <code>@retry</code> , <code>@cache</code> written from scratch
7	Ch 18	An exception hierarchy and two context managers
8	Ch 19	Split your project into a proper package with <code>src/</code>
9	Ch 20	Replace hand-rolled helpers with stdlib equivalents
10	Ch 21	Tests for everything; find one real bug
11	Ch 22	Annotate one module; get mypy clean
12	Ch 23	Git history, <code>pyproject.toml</code> , installable project

Done when someone else can clone your repository, install it, run the tests, and understand the layout without asking you.

Advanced: 12 weeks

Week	Read	Deliverable
1	Ch 24	Halve a program's memory; explain why it worked
2	Ch 25	A generic, protocol-based module, mypy strict clean
3	Ch 26	A validated model using descriptors; a plugin registry
4	Ch 27	Benchmark threads vs processes vs serial on one workload
5–6	Ch 28	An async service with timeouts, caps, and clean shutdown
7	Ch 29	Profile something real; ship a 10× improvement
8–9	Ch 30	Refactor a tangled module: inject dependencies, delete mocks
10	Ch 31	Publish to TestPyPI with a changelog and a deprecation
11	Ch 32	Full ruff + mypy + pre-commit + CI matrix on a real repo
12	Ch 33	Validate every external input in one service

Done when you can explain a design decision in terms of what it costs, not just what it does.

Expert: 16 weeks

Two to six hours a week. Each week: read the part, do at least two exercises, and write one paragraph explaining the mechanism to an imagined colleague. The writing is not optional — it is the test.

Week	Focus	Deliverable
------	-------	-------------

Week	Focus	Deliverable
1	Part I §1.1–1.4	Implement <code>property</code> , <code>classmethod</code> , <code>staticmethod</code> from scratch
2	Part I §1.5–1.9	A plugin registry with <code>__init_subclass__</code> and a metaclass; compare
3	Part II §2.1–2.5	Type a decorator library with zero <code>Any</code>
4	Part II §2.6–2.10	Get an existing module to mypy <code>--strict</code> clean
5	Part III §3.1–3.3	Benchmark suite across GIL / free-threaded / process
6	Part III §3.4	A crawler with <code>TaskGroup</code> , timeouts, clean shutdown
7	Part III §3.5–3.8	Port the crawler's parsing stage to a process pool
8	Part IV §4.1–4.3	Read <code>dis</code> for five idioms; cut a program's RSS in half
9	Part IV §4.4–4.7	Profile a real service; make one 10× improvement
10	Part V §5.1–5.4	An AST-based lint rule wired into CI
11	Part V §5.5–5.7 + Part VI	A lazy-import layer; a constant-memory pipeline
12	Part VII	Redesign a module's exceptions; deprecate an API properly
13	Part VIII	Model a real external API with strict validation
14	Part IX + Part X	Publish with trusted publishing; add Hypothesis and mutation testing
15	Parts XI–XII	Run the pre-deploy checklist against something you own
16	Parts XIII–XVI	Build a small DSL, or submit one upstream contribution

Appendix D — Quick Reference: The Mistakes That Cost the Most

Ranked by how often they cause real incidents.

Correctness

1. Mutable default arguments — `def f(x=[])`. Use `None`.
2. `except Exception: pass` — swallows the bug you are trying to find.
3. Floats for money. Use integer cents or `Decimal`.
4. `datetime.utcnow()` and naive datetimes. Use `datetime.now(timezone.utc)`.
5. Modifying a list while iterating over it.
6. `if x:` when `0` or `""` is a valid value. Use `is not None`.
7. Shallow copy of nested data, then mutating the shared inner objects.
8. `open()` without `encoding="utf-8"`.
9. `groupby` on unsorted input.
10. Assuming an iterator can be consumed twice.

Concurrency

11. A blocking call inside an async handler — freezes every task.
12. `create_task` without keeping a reference — the task can vanish.
13. Swallowing `CancelledError`.
14. Unbounded queues — a memory leak with a friendly API.
15. Assuming the GIL makes `counter += 1` safe. It does not.
16. `fork` in a process that has threads.
17. Never calling `.result()` on futures — exceptions disappear silently.
18. No timeout on any network call.

Performance

19. `x in big_list` in a loop. Use a set.
20. $N+1$ queries.
21. String building with `+=` in a loop. Use `join`.
22. `list.pop(0)`. Use `deque`.
23. Loading a file that should be streamed.
24. `@cache` on a method — pins every instance in memory forever.
25. Optimising before profiling.

Design

26. Inheritance where composition was meant.
27. Constructing dependencies inside a class instead of injecting them.
28. Abstractions invented for one implementation.
29. Error messages without the offending value.
30. Functions that both compute and perform I/O.

Operations and security

31. `shell=True` with interpolated input.
32. Unpickling or `yaml.load`-ing untrusted data.
33. String-formatted SQL.
34. `verify=False` on TLS.
35. `random` instead of `secrets` for tokens.
36. Secrets in the repository, the image, or the logs.
37. `pip install` outside a virtual environment.
38. No lock file for an application.
39. Debug mode on in production.
40. Extracting an archive without checking member paths.

Appendix E — Capstone Projects

Pick one. Each forces you across most of the book.

1. **A miniature ORM.** Descriptors for columns, `__init_subclass__` for table registration, a generated `__init__` via `exec`, typed query builder using `Self` and overloads, parameterised SQL, a connection pool, transaction context managers, and a Hypothesis stateful test for the unit-of-work.

2. **An async job runner.** Priority queues with backpressure, TaskGroup-based workers, per-job timeouts and correct cancellation, retries with exponential backoff and jitter, structured logging with `ContextVar` correlation ids, graceful shutdown on SIGTERM, and a process-pool executor for CPU stages.
 3. **A profiler.** Built on `sys.monitoring`, with per-function statistics, flamegraph output, sub-1% overhead, remote attach, and a pytest plugin that fails the build on regressions.
 4. **A high-performance parser.** Streaming, `mmap` + `memoryview` zero-copy, generator pipeline, a Cython or Rust inner loop with the GIL released, fuzzed with `atheris`, benchmarked in CI, distributed as abi3 wheels for four platforms.
 5. **A plugin framework.** Entry points, namespace packages, protocol-based interfaces, sandboxed subprocess execution for untrusted plugins, audit hooks, versioned API with a real deprecation policy, and full type stubs.
-

Appendix F — Reading List

Primary sources (read these first).

- The CPython source: `Objects/typeobject.c` (attribute lookup, MRO), `Python/ceval.c` (the eval loop), `Objects/dictobject.c`, `Modules/_asynciomodule.c`.
- The Python Language Reference, especially the Data Model chapter — the most under-read authoritative document in the ecosystem.
- The PEPs cited throughout this book. 3119, 3129, 484, 492, 525, 544, 557, 561, 563, 572, 578, 585, 604, 612, 634, 646, 649, 654, 659, 669, 683, 684, 688, 692, 695, 696, 703, 734, 742, 749, 750, 751, 779.
- `python-dev`/Discourse threads for the features you rely on — the rationale is usually more instructive than the spec.

Books.

- *Fluent Python*, Ramalho — the best single companion to Part I and VI.
- *CPython Internals*, Shaw — a guided tour of the source.
- *High Performance Python*, Gorelick & Ozsvald — Part IV in depth.
- *Architecture Patterns with Python*, Percival & Gregory — Parts VII–VIII at system scale.
- *Robust Python*, Viafore — the typing culture argument.

Code to read.

- `cpython/Lib/` — `dataclasses.py`, `functools.py`, `contextlib.py`, `enum.py`, `asyncio/`. Standard library source is unusually good teaching material.
 - `attrs`, `pydantic`, `httpx`, `pytest`, `rich`, `polars` — well-engineered, different design philosophies, all worth studying.
-

Closing Note

Expertise here is not memorising the list. It is knowing, when something behaves unexpectedly, *which layer to look at*: is this an attribute-lookup rule, a variance rule, a scheduling rule, a refcount rule, an encoding rule? The parts of this book are those layers. When you can name the layer within a few seconds of seeing a bug, you are done with this book — and ready to start reading the source, which is where the rest of it lives.